

Yres Documentation

Yres

11 November, 2025

Table of Contents

1	Where to Begin:	11
2	Key Features:.....	12
3	Frontend features	13
3.1	Introduction	13
3.2	General UI overview	13
3.2.1	Yres General Navigation	13
3.2.2	Core Navigation Areas:.....	13
3.2.3	Tips for Efficient Navigation:.....	13
3.3	Dashboard	14
3.3.1	Navigation	14
3.3.2	Components.....	15
3.4	Admin.....	17
3.4.1	Announcements.....	17
3.4.2	Audit Logs	19
3.4.3	Database overview.....	20
3.4.4	Firewall	20
3.4.5	PowerBI Models	20
3.4.6	Rebuild.....	21
3.4.7	Settings.....	22
3.4.8	Secrets.....	23
3.4.9	Shared integration runtimes.....	23
3.4.10	Update Environment	24
3.4.11	Users & Roles	25
3.4.12	Environment admin.....	25
3.5	Projects.....	32
3.5.1	Changes.....	33
3.5.2	Install Changes	33
3.5.3	Project Overview	34
3.5.4	Scripted Objects.....	34

3.5.5	Release Changes	35
3.6	Data Sources	36
3.6.1	Sources	36
3.6.2	Added Sources	38
3.7	Load Management	39
3.7.1	Run Pipelines	39
3.7.2	Triggers	40
3.7.3	Monitoring	41
3.7.4	Master Pipelines	41
3.8	Data Engineering	45
3.8.1	View Persistence	45
3.8.2	Database Objects	46
3.9	Account	49
3.9.1	Account settings	49
3.9.2	Feedback	49
4	Setup	50
4.1	Installing Yres	50
4.1.1	Preparations	50
4.1.2	Installation	60
4.2	Getting started with Yres	66
4.2.1	Logging in to the Yres webapp	66
4.2.2	UI overview	67
4.2.3	Creating a new organization	67
4.3	Users and their roles	68
4.3.1	Managing users	69
4.3.2	Managing roles	70
4.4	Utilizing different environments	71
4.4.1	Updating environments	71
4.4.2	Utilizing projects and changes	71
4.5	Linking data sources	71
4.6	Loading data sources	73
4.6.1	Refreshing data source metadata	74

4.6.2	Load types	74
4.6.3	Key columns	74
4.7	Generating views	74
4.7.1	Generating persisted views	75
4.8	Setting up pipelines and triggers	76
5	Troubleshooting	79
5.1	1. Deployment rights error	79
5.2	2. Unable to load local files	79
6	Data Source Requirements	81
6.1	Databases	81
6.1.1	MySQL	81
6.1.2	DB2	82
6.1.3	SQL Server	84
6.1.4	Azure SQL Database	85
6.1.5	Oracle	87
6.1.6	PostgreSQL	88
6.1.7	Snowflake	89
6.2	Azure	91
6.2.1	Azure Blob Storage	92
6.2.2	SharePoint	94
6.3	File	96
6.3.1	File Server	96
6.4	Services & Apps	97
6.4.1	Monday	97
6.4.2	AFAS	98
6.4.3	Exact Online	99
6.4.4	Simplicate	100
6.4.5	SalesForce	101
6.4.6	SAC	102
6.4.7	OneStream (preview)	103
6.4.8	PowerBI	104
6.5	Generic protocol	105

6.5.1	OData	105
6.5.2	OData OAuth.....	106
6.5.3	RestService.....	108
7	SQL Interaction	111
7.1	StoredProcedures	111
7.1.1	Procedure: [Change].[spAddScriptedObject].....	111
7.1.2	Procedure: [Change].[spAddTable]	111
7.1.3	Procedure: [Change].[spCopyTableContent]	112
7.1.4	Procedure: [Change].[spImport]	112
7.1.5	Procedure: [Change].[spInstall]	113
7.1.6	Procedure: [Change].[spMaintain].....	113
7.1.7	Procedure: [Change].[spMaintainProject].....	114
7.1.8	Procedure: [Change].[spRelease]	114
7.1.9	Procedure: [Config].[spAddFrameWorkColumns].....	115
7.1.10	Procedure: [Config].[spCompareMetadata]	115
7.1.11	Procedure: [Config].[spCreateExternalTablesFromDictionary]	116
7.1.12	Procedure: [Config].[spCreateTablesFromDictionary].....	116
7.1.13	Procedure: [Config].[spDeleteTablesFromDB]	117
7.1.14	Procedure: [Config].[spEnableColumnstore].....	117
7.1.15	Procedure: [Config].[spEnableMemoryOptimization]	118
7.1.16	Procedure: [Config].[spFillServices_SAC]	118
7.1.17	Procedure: [Config].[spGetDependenciesSQL]	118
7.1.18	Procedure: [Config].[spSetDatabaseParameter].....	119
7.1.19	Procedure: [Config].[spSetDatabaseServiceTier].....	119
7.1.20	Procedure: [Config].[spUpdateRefreshToken]	119
7.1.21	Procedure: [Config].[spUpdateTablesFromDictionary].....	120
7.1.22	Procedure: [Config].[spWriteCrash]	121
7.1.23	Procedure: [Config].[spWriteDump]	121
7.1.24	Procedure: [Config].[spWriteError]	122
7.1.25	Procedure: [Config].[spWriteFullLog]	123
7.1.26	Procedure: [Config].[spWriteLog]	124
7.1.27	Procedure: [Config].[spWriteMessage].....	125

7.1.28 Procedure: [Config].[spWriteMessage].....	125
7.1.29 Procedure: [Config].[spWriteWarning]	126
7.1.30 Procedure: [dbo].[spAdaptiveIndexDefrag_CurrentExecStats]	126
7.1.31 Procedure: [dbo].[spAdaptiveIndexDefrag_Exceptions]	127
7.1.32 Procedure: [dbo].[spAdaptiveIndexDefrag_PurgeLogs]	127
7.1.33 Procedure: [dbo].[spCopyDB]	128
7.1.34 Procedure: [dbo].[spJsonToTable].....	128
7.1.35 Procedure: [dbo].[spLongPrint]	128
7.1.36 Procedure: [dbo].[spMSForEachTable].....	129
7.1.37 Procedure: [dbo].[spMSForEachWorker]	129
7.1.38 Procedure: [dbo].[spRunSQL]	130
7.1.39 Procedure: [LoadManagement].[spCheckKeyAndRowHash].....	130
7.1.40 Procedure: [LoadManagement].[spFillDictionary_AFAS]	131
7.1.41 Procedure: [LoadManagement].[spFillDictionary_oDATA]	131
7.1.42 Procedure: [LoadManagement].[spFillTable_Json]	131
7.1.43 Procedure: [LoadManagement].[spFillTable_Monday].....	132
7.1.44 Procedure: [LoadManagement].[spFindTablesBehindSQL]	132
7.1.45 Procedure: [LoadManagement].[spGenerateTypeMapping]	132
7.1.46 Procedure: [LoadManagement].[spGetColumnMapping].....	133
7.1.47 Procedure: [LoadManagement].[spGetRowCount]	133
7.1.48 Procedure: [LoadManagement].[spHIS_InsertAndUpdate]	134
7.1.49 Procedure: [LoadManagement].[spHIS_TruncateTable]	134
7.1.50 Procedure: [LoadManagement].[spLoadDWH]	135
7.1.51 Procedure: [LoadManagement].[spMaintainFiles].....	135
7.1.52 Procedure: [LoadManagement].[spMaintainFilesInDictionary].....	136
7.1.53 Procedure: [LoadManagement].[spMaintainPersistView].....	136
7.1.54 Procedure: [LoadManagement].[spMaintainRestService].....	137
7.1.55 Procedure: [LoadManagement].[spMaintainSource].....	137
7.1.56 Procedure: [LoadManagement].[spMaintainTable]	138
7.1.57 Procedure: [LoadManagement].[spMaintainTrigger].....	138
7.1.58 Procedure: [LoadManagement].[spMaintainTrigger].....	138
7.1.59 Procedure: [LoadManagement].[spMaterializeViews].....	139

7.1.60 Procedure: [LoadManagement].[spMaterializeViewToTable]	140
7.1.61 Procedure: [LoadManagement].[spPrepareCopy]	140
7.1.62 Procedure: [LoadManagement].[spRegenerateSurrogateKeys]	140
7.1.63 Procedure: [Monitoring].[spWriteLoadStatus]	141
7.1.64 Procedure: config.spGenerateDbreader	142
7.2 Functions	144
7.2.1 Function: [Change].[fxGetChangelsOpen]	144
7.2.2 Function: [Change].[fxGetLaspageChangeIdFor]	144
7.2.3 Function: [Change].[fxGetLapageOpenChange]	144
7.2.4 Function: [Change].[fxGetReleasedJson]	145
7.2.5 Function: [Change].[fxGetTableDefinition]	145
7.2.6 Function: [Change].[fxGetTableTypeDefinition]	145
7.2.7 Function: [Config].[fxAddTryCatch]	146
7.2.8 Function: [Config].[fxGetSchemaName]	146
7.2.9 Function: [Config].[fxGetSession]	147
7.2.10 Function: [Config].[fxGetSetting]	147
7.2.11 Function: [dbo].[fxGeneratePassword]	147
7.2.12 Function: [dbo].[fxGetJsonCollection]	148
7.2.13 Function: [dbo].[fxRemoveNonAlphaCharacters]	148
7.2.14 Function: [dbo].[fxStripCharacters]	148
7.2.15 Function: [dbo].[fxToDecimal]	149
7.2.16 Function: [dbo].[fxToProper]	149
7.2.17 Function: [dbo].[fxUNIXtoDateTime]	149
7.2.18 Function: [dbo].[fxUTC2]	150
7.2.19 Function: [LoadManagement].[fxGetActualTablename]	150
7.2.20 Function: [LoadManagement].[fxGetDataType]	151
7.2.21 Function: [LoadManagement].[fxGetKeyColumns]	151
7.2.22 Function: [LoadManagement].[fxGetKeyHashColumns]	152
7.2.23 Function: [LoadManagement].[fxGetOptimized]	152
7.2.24 Function: [LoadManagement].[fxGetRowColumns]	152
7.2.25 Function: [LoadManagement].[fxGetRowHashColumns]	153
7.2.26 Function: [LoadManagement].[fxGetStoreType]	153

7.2.27 Function: [LoadManagement].[fxGetSurrogate].....	154
7.2.28 Function: [LoadManagement].[fxPredictKeyColumns].....	154
7.2.29 Function: [LoadManagement].[fxPredictTableName]	155
7.2.30 Function: [oData].[fxBaseResponse].....	155
7.2.31 Function: [oData].[fxMetadataResponse].....	155
7.2.32 Function: [Metadata].[fxGetViewSources].....	156
7.2.33 Function: [Metadata].[fxGetViewSourcesRecursive]	156
7.2.34 Function: [Monitoring].[fxGetTableLoads].....	156
7.2.35 Function: [LoadManagement].[fxExtractor].....	157
7.3 Logs	158
7.4 Views	159
7.4.1 View: [Monitoring].[vwLoadMonitor].....	159
7.4.2 View: [LoadManagement].[vwDictionary]	160
7.4.3 View: [LoadManagement].[vwUsedTables]	161
7.4.4 View: [Monitoring].[vwUsedTables]	162
7.4.5 View: [Monitoring].[vwMonitor]	163
7.4.6 View: [Config].[vwUserlog].....	164
7.4.7 View: [Config].[vwUserLogJSON].....	164
7.4.8 View: [Config].[vwDictionaryVsHis].....	165
7.4.9 View: [Config].[vwDictionaryVsStage].....	165
7.4.10 View: [LoadManagement].[vwExtractor].....	165
7.4.11 View: [LoadManagement].[vwArchivingExtractor].....	166
7.4.12 View: [LoadManagement].[vwUnusedTables].....	166
7.4.13 View: [LoadManagement].[vwUsedColumns]	166
7.4.14 View: [LoadManagement].[vwUsedODSTablesAndColumns].....	166
7.4.15 View: [LoadManagement].[vwViewPersistence].....	167
7.4.16 View: [LoadManagement].[vwViewsAndColumns]	167
7.4.17 12. View: [Monitoring].[vwLoads].....	167
7.4.18 View: [Monitoring].[vwWorkflow]	168
8 Release notes.....	169
8.1 Version 1.55 - September 2025	169
8.1.1 Yres DWH – Version 1.55 (September 2025)	169

8.2	Version 1.54 - July 2025	170
8.2.1	Yres DWH – Version 1.54 (July 2025)	170
8.3	Version 1.53 - May 2025	172
8.3.1	Yres DWH – Version 1.53 (May 2025)	172
8.4	Version 1.52 - January 2025	175
8.4.1	Breaking change (reminder)	175
8.4.2	Improvements	175
8.5	Version 1.51 - September 2024	177
8.5.1	Breaking change	177
8.5.2	Improvements	177
8.6	Version 1.50 - August 2024	178
8.7	Version 1.49 - June 2024	178
8.8	Version 1.48 - April 2024	179
8.9	Version 1.47 - Januari 2024	179

Welcome to the Yres Documentation

The Yres documentation provides a comprehensive guide for using Yres, the data platform automation tool specifically designed to simplify the deployment and management of Azure-based data warehouses. Here, you will find information on setting up, managing, and leveraging the features Yres offers.

1 Where to Begin:

- **General Navigation** ([see page 13](#)): Learn how to navigate the Yres interface and perform basic actions.
- **Installation and Configuration**: Follow step-by-step instructions to correctly set up Yres in your Azure environment.
- **Troubleshooting**: Get assistance with common challenges and limitations within the Yres ecosystem.
- **Administrative Tools**: Discover how to manage users, roles, and environments using the Admin panel.
- **Data Source Management**: Add new data sources, load data, and create tables for analysis and storage.

2 Key Features:

- **Single Sign-On (SSO):** Simplify and secure access to Yres with Azure SSO.
- **Log Management:** Gain insights into actions and events within the data platform.
- **Life Cycle Management:** Manage the evolution of your data warehouse with minimal complexity.
- **Flexibility and Scalability:** Leverage Azure resources for scalable and secure data storage and processing.

Yres reduces complexity into an easy-to-use interface, powered by Azure's robust capabilities. Start today and discover how you can optimize your data ecosystem!

For more detailed information, navigate to the desired section via the sidebar or use the search bar at the top.

3 Frontend features

3.1 Introduction

Welcome to the documentation for using the Yres frontend. This section aims to give more information about utilizing every part of the frontend, including best practices, more in-depth explanations of features and more.

This section is grouped equivalent to the groups used in the frontend. Each subsection is sorted alphabetically.

3.2 General UI overview

3.2.1 Yres General Navigation

This section provides a complete overview of navigating the Yres interface and performing essential tasks. Yres is designed to streamline your experience, enabling you to efficiently utilize Azure resources for data warehouse management.

3.2.2 Core Navigation Areas:

- **Admin Panel:** Manage users, roles, environments, and global settings directly from this central hub.
- **Projects:** Organize your data-related tasks into manageable projects, track changes, and ensure smooth workflows.
- **Data Sources:** Add, edit, or manage connections to various supported data sources, and create tables for further processing.
- **Load Management:** Monitor and manage data pipelines, triggers, and persistence tasks to ensure efficient data operations.
- **Account Settings:** Customize your preferences, manage account security, and adjust personal settings.

3.2.3 Tips for Efficient Navigation:

- Familiarize yourself with the **Admin Panel** for managing users and environments; it serves as the foundation for most administrative tasks.
- Use the **Data Sources** section to streamline the process of linking, loading, and managing external data connections.

- Explore the **Load Management** area to keep your pipelines running smoothly and ensure data persistence for further analysis.

Yres simplifies the management of complex data ecosystems, allowing users to focus on insights and decision-making rather than operational overhead.

For more detailed guidance, refer to the specific sections available in the sidebar or use the search functionality for quick access.

3.3 Dashboard

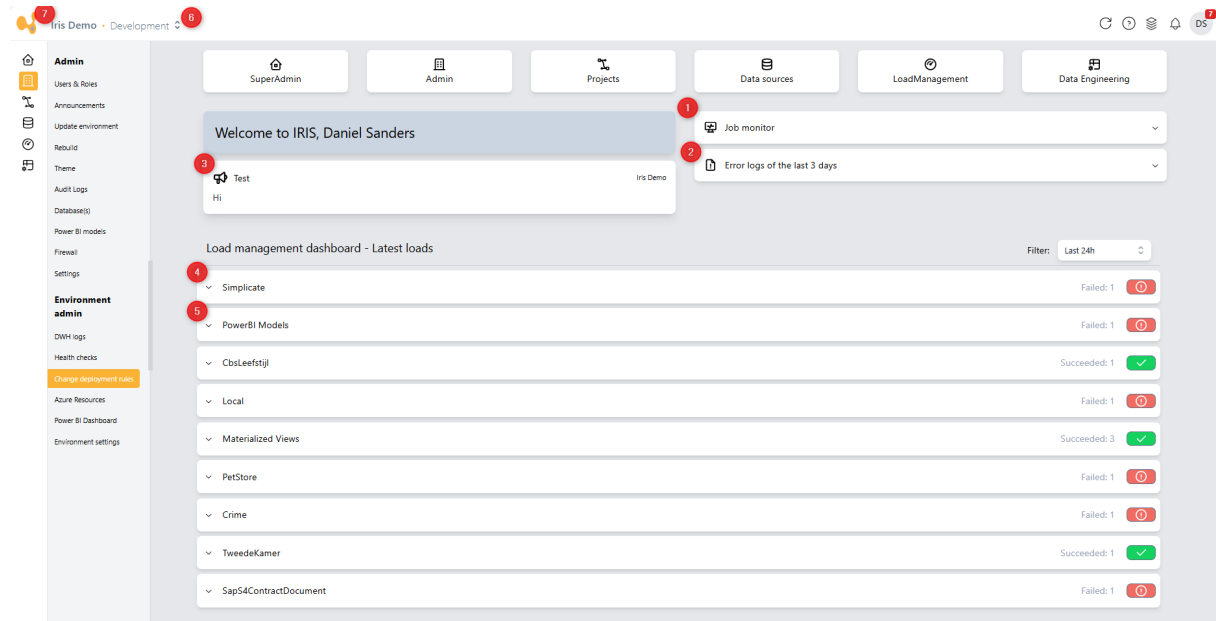
3.3.1 Navigation

When logging into Yres you will be directed to the Dashboard page where you can find the most important metrics for your environment.

- Announcements
- Error logs
- Job statuses
- Load statuses
- Persisted Views
- PowerBI Model refreshes

If you navigate to other pages you can always return to the dashboard by clicking on the logo in the left top of the webpage.

3.3.2 Components



1. This is where you can find all the jobs in the selected environment with their status and logs. Of actions are performed by ADF this log will also include a direct link to the monitor in ADF.

Job monitor

Delete table Persoon	Batch	Daniel Sanders	14-07-2025 22:19:41	14-07-2025 22:19:42	dev	Success	i
Delete table SoortMisdrijf	Batch	Daniel Sanders	14-07-2025 21:49:44	14-07-2025 21:49:45	dev	Success	i
Delete table PersoonGeschenk	Batch	Daniel Sanders	14-07-2025 20:38:14	14-07-2025 20:38:14	dev	Success	i
Update table Persoon	Batch	Daniel Sanders	14-07-2025 20:38:02	14-07-2025 20:38:04	dev	Success	i
MasterPipeline - RefreshPowerBI	Pipeline	Daniel Sanders	14-07-2025 17:59:49	14-07-2025 18:02:57	dev	Failed	i 🔗

Showing 1 to 15 of 579 results

< 1 2 3 ... 39 >

2. These are all the error logs produced by the database in the last 3 days. Here you can find error on processes like creating tables, installing changes, updating sources, etc. But also detailed log on load steps.

 Error logs of the last 3 days

general.errorLogs				
Process	MessageType	Timestamp	User	Actions
splInstall	Error	14-07-2025 17:31:34		
splInstall	Error	14-07-2025 17:15:32		
splInstall	Error	14-07-2025 17:13:45		
splInstall	Error	14-07-2025 17:00:33		
splInstall	Error	14-07-2025 16:55:20		
splImport	Dump	14-07-2025 16:00:05	Daniel Sanders	

3. This is where you will find announcements created by your admin and general announcements by Yres. Announcements can be used to communicate on upcoming releases, downtime and other information users of Yres should be aware of.

4. Here you can find loads grouped by source system, only the lapage result per source object is shown. At the right top you can select the period over which you would like to see the logs.

The log

Load management dashboard - Latest loads

Filter: This week(mon-s) 

^ Simpicate Succeeded: 2 | Failed: 1 

Simpicate									
Source system	Target	DateTime	LoadType	Runtime sec: Pipeline	Copied rows	New rows	Delta rows	Status	Actions
Simpicate	Simpicate_hours_hours	15-07-2025 09:42:23	FULL		6	0	0	FAILED	 
Simpicate	Simpicate_sales_revenuegroup	14-07-2025 10:10:43	FULL	842	6	0	0	SUCCEEDED	 
Simpicate	Simpicate_sales_salesreason	14-07-2025 10:10:39	FULL	858	22	0	0	SUCCEEDED	 

Under the  icon you'll find the detailed steps of the load and with the link icon you can navigate straight to the ADF monitor for that specific load.

5. Data processes that are not source system specific like loading PowerBI models and saving Views to tables are also found in this overview

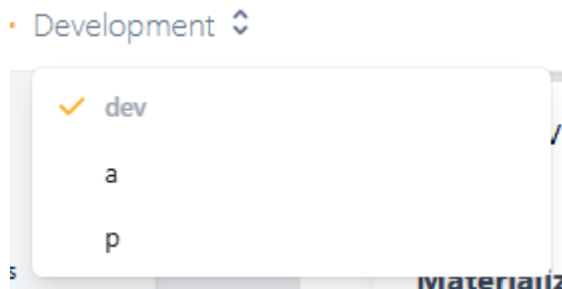
The screenshot shows the Yres dashboard with the following sections:

- PowerBI Models**: Failed: 1 (Red status icon). Contains a table with one row:

Source system	Target	DateTime	LoadType	Runtime sec: Pipeline	Copied rows	New rows	Delta rows	Status	Actions
PowerBI Models	PowerBI - Plainwater	15-07-2025 07:19:33	FULL	191				FAILED	[Info] [Refresh]
- CbsLeefstijl**: Succeeded: 1 (Green status icon).
- Local**: Failed: 1 (Red status icon).
- Materialized Views**: Succeeded: 3 (Green status icon). Contains a table with three rows:

Source system	Target	DateTime	LoadType	Runtime sec: Pipeline	Copied rows	New rows	Delta rows	Status	Actions
Materialized Views	dbo.Ernt24hrs	14-07-2025 10:56:43	FULL	0	1			SUCCEEDED	[Info] [Refresh]
Materialized Views	dbo.AvgFragLst30days	14-07-2025 10:56:43	FULL	0	54			SUCCEEDED	[Info] [Refresh]
Materialized Views	DBO.EmployeeSales	14-07-2025 10:56:43	FULL	0	0			SUCCEEDED	[Info] [Refresh]

6. In your Yres installation has multiple environments you can switch between environments using the dropdown in the left top



7. Using the logo at the left top (this can be your own company logo) you can always return to this screen.

3.4 Admin

The Admin panel allows the user to manage other users and environments. This includes role management, viewing data warehouse logs and managing announcements.

[Announcements \(see page 17\)](#) | [Audit Logs \(see page 19\)](#) | [Azure Resources \(see page 25\)](#) | [Data Warehouse Logs \(see page 27\)](#) | [Environment Settings \(see page 28\)](#) | [Firewall \(see page 20\)](#) | [Health Checks \(see page 31\)](#) | [PowerBI Dashboard \(see page 32\)](#) | [PowerBI Models \(see page 20\)](#) | [Rebuild \(see page 21\)](#) | [Update Environment \(see page 24\)](#) | [Users & Roles \(see page 25\)](#)

3.4.1 Announcements

Admins can utilize the announcements feature to send messages to all users, or all users within a specific organization. This is useful for easy and direct communication with the rest of the Yres users for things like scheduled maintenance. These announcements will show up on the home page.

Announcements can be accessed either via the Superadmin panel, or the organization admin panel. Organization admins can only send announcements to their respective organization.

3.4.1.1 Managing announcements

Title

Start date



Start time

End date



End time

Receiver



Notify users



Priority



Announcement

B **I**  |   Title 1 Title 2
    

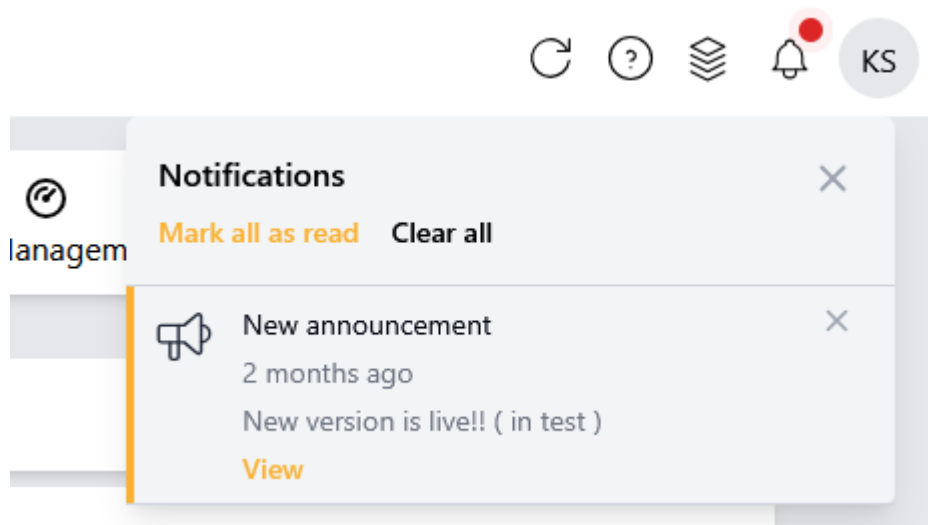
This is just a test announcement

¹ An example of a filled out announcement form.

There are a couple things of note when creating a new announcement:

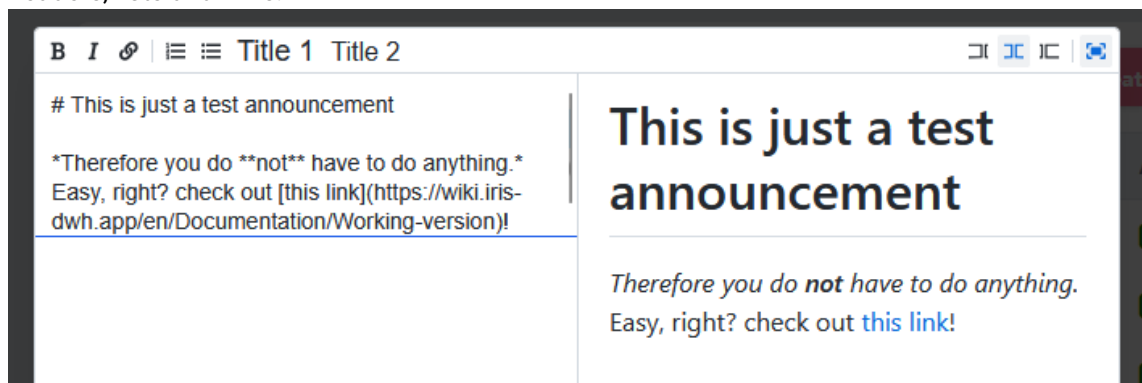
- The start and end date are not required. If these fields are left empty the announcement will be immediately sent out and will stay up until it is deleted.

- You can choose whether or not the announcement shows up in the notifications tab at the top with the Notify Users option:



2 An example of a notification in the topbar.

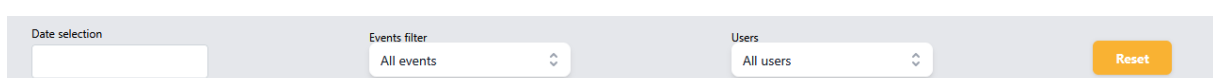
- With Priority, you can make an announcement take priority over others. The announcement with the higher priority will be shown at the top of the list in the home screen.
- The body of the announcement supports Markdown text, which allows you to insert things like headers, lists and links:



3 A side-by-side comparison of the pure markdown and the rendered result.

3.4.2 Audit Logs

Audit logs are a way to monitor the actions of organization users. These logs come with info such as user agent data, the client IP address and extra properties depending on the log type.



These logs can be filtered based on a specific date range, event severity or specific users.

3.4.3 Database overview

The databases page allows organization admins to view all databases in use for the current organization. If you've added a custom database, you can directly update it via this screen.

3.4.4 Firewall

The Firewall tab allows organization to grant or deny access to specific IP addresses. This allows for greater security by letting admins control where users can access Yres from.

Firewall Create					
Name	Value	Type	Grant	Created_at	Actions
Lars Caspers ip address		ip	allow	2024-02-22 10:05:19	 
larsTest		ip	allow	2024-02-22 10:05:19	 

3.4.5 PowerBI Models

At the PowerBI models page, admins can manage unified models for each Yres environment. This allows the correct models to be called when working with PowerBI data in Yres, such as refreshing models via the Master pipeline.

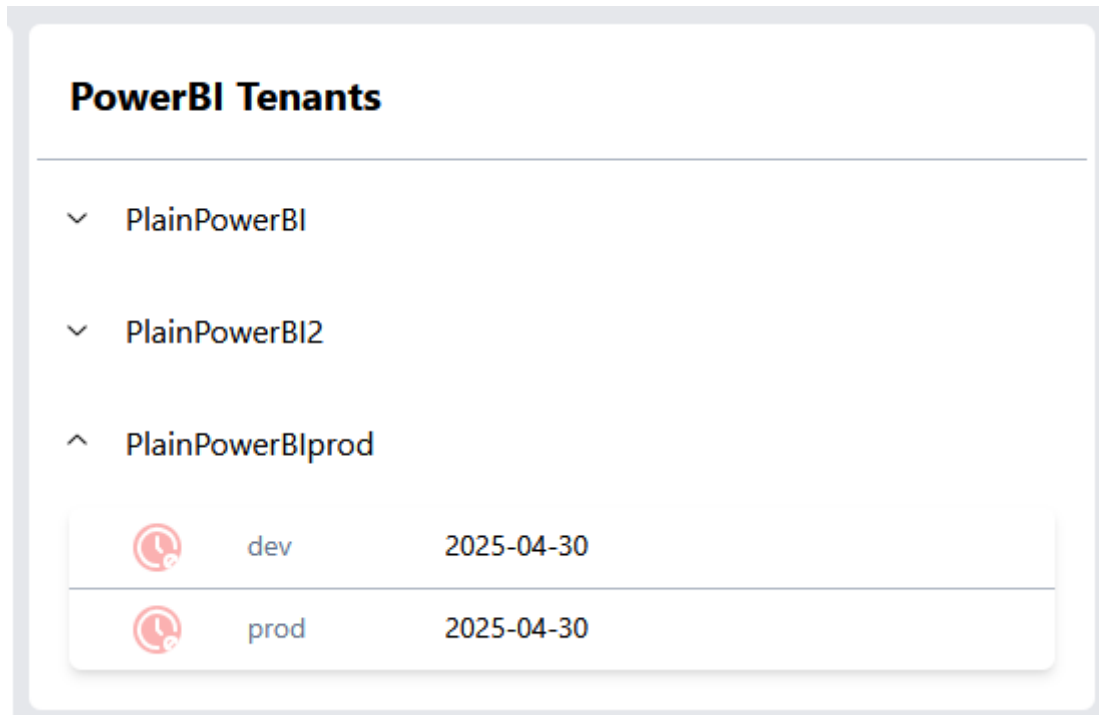
^ MyNewModel  			
Environment	App registration	Workspace name	Model name
dev	PowerBI	Plainwater BI	Plainwater
a	PowerBI	Plainwater BI	UrenControle
p	PowerBI	Plainwater BI	JaarOverzicht

4 An example of a PowerBI model – when called within Yres for an action in the dev environment, it'll use the Plainwater model. If it's called within the production (p) environment, it'll use JaarOverzicht.

3.4.5.1 Setup

Before adding models, a tenant is required, which will allow you to fetch its respective workspaces and models. This is done by adding a data source with the type PowerBI. More information on credentials for this source type can be found [here](#). (see page 104)

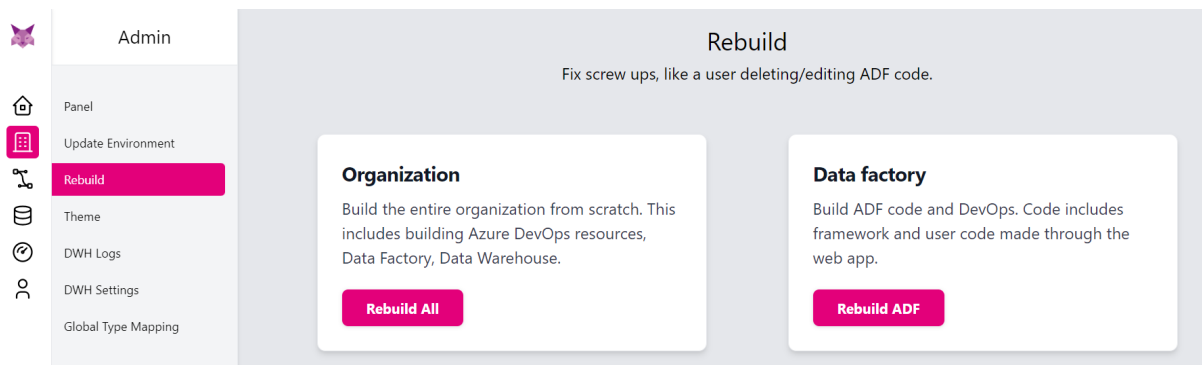
Once the source has been created, it'll show up at the tenants section:



5 An example of the tenants section on the right of the PowerBI models page, showing the expiry date for each set of credentials

3.4.6 Rebuild

The rebuild tab allows the user to reset data to its factory settings. The user is able to choose between rebuilding the entire organization or the Azure data factory.





This action will overwrite ALL existing configuration not done with the Yres Frontend. Use this with caution.

3.4.7 Settings

This page contains settings that will impact the entire application (all environments).

Settings

Label	Value	Actions
IRIS Dynamic Workflow batch count	6	

Danger Zone

Iris Demo

Delete this organization
Once deleted, it will be gone forever. Please be certain.

Delete organization

In the Settings table you will find the setting that dictates with how many parallel processes the system will load sources into the Database and/or Datalake. This setting can be any integer value between 1 and 50.

Update setting

IRIS Dynamic Workflow batch count

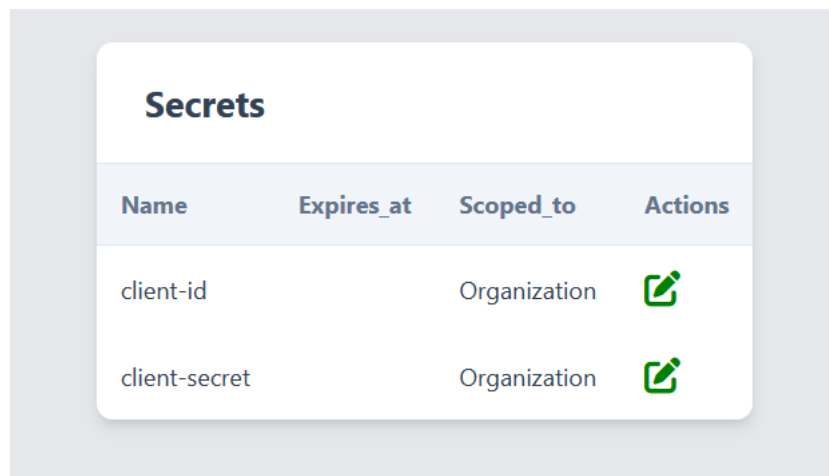
Dynamic workflow batch count be at most 50

Under the Settings table you'll find the "Danger Zone". Here you can cancel your Yres subscription by deleting the organization. This will not impact your Azure environment, it will only remove the Organization from the Yres portal.

This action can not be undone

3.4.8 Secrets

The admin secrets menu is used to view secrets stored in the corresponding key vault, along with its scope and, if given, an expiry date. Users with the right permissions can also use this menu to update a secret value if needed.



Secrets			
Name	Expires_at	Scoped_to	Actions
client-id		Organization	
client-secret		Organization	

6 The secrets table, showing a couple records and the option to update them

3.4.9 Shared integration runtimes

In the Shared integration runtimes tab, admins can manage self-hosted integration runtimes for use with ADF.

Integration runtimes created via Yres require a name and description. After creating an integration runtime, admins can click on the info icon to download the Microsoft integration runtime tool and use the keys displayed to register it via the tool wizard.

Shared Integration Runtimes			Create
Name	Environment_type	Adf	Actions
myNewIR	dev	adf-shared-dev-iris-demo	 

7 The integration runtimes table, showing a custom integration runtime along with the options to delete and view the installation steps.

3.4.10 Update Environment

The update environment tab is used to update environments to the latest version of Yres.

Update environment

Update IRIS DWH and publish data factory

Dev

Already up to date

Last update:
IRIS DWH version:1.53

Prod

Click to update prod to 1.53

Last update:
IRIS DWH version:1.48

8 Update options per environment



It is advised to page the new version on a development environment before updating other environments like a production environment to ensure stability.



For more information on update content, see [Release notes \(see page 169\)](#)

3.4.11 Users & Roles

The users & roles page allows admins to manage users and their roles. This gives admins safe and scalable control over who can manage which parts of Yres in a simple interface.

3.4.11.1 Users

Users can be created with just an email, their role and whether logging in requires a Microsoft Azure single sign-on (SSO). Added users will receive an email with a default password, which, along with their name, can be changed at the account settings.

3.4.11.2 Roles

Yres comes with a set of default roles, but admins can easily create custom ones that suit their needs. Roles usually follow a create-read-update-delete (CRUD) structure per permission type. Permissions that do not fall under this structure, such as building ADF code, are listed at the next step, under “Other”.

Once a role has been added, it can be assigned to a user at the roles dropdown.

3.4.12 Environment admin

Not all settings and configurations are applied on the entire installation, some are Environment specific. Those items can be found in this menu.

3.4.12.1 Azure Resources

The Azure Resources tab allows users to view their linked Azure resources at a glance. The table shows the resource name, type, location and includes a direct hyperlink to the resource if available.

Type	Location	Actions
Sql Database	westeurope	
Web	westeurope	
KeyVault	westeurope	
Storage	westeurope	
Web	westeurope	
Sql Database	westeurope	
CognitiveServices	global	
Sql Database	westeurope	
Storage	westeurope	
Storage	westeurope	
Sql Server	westeurope	
DataFactory	westeurope	
Logic	westeurope	

3.4.12.2 Change deployment rules

Change deployment rules allow you to have sources with different naming between Dev, Test and Prod.

I.e. dev = ERP_DEV.Product, page = ERP_TST.Product & Prod = ERP.Product

3.4.12.2.1 Overview

This feature offers you a way to translate the object names between Dev and the environment you are importing the changes into.

Change overwrites

OLD			NEW		
Source	Schema	Table	Source	Schema	Table
MyERP	Development	Orders	MyERP	Test	Orders 
MyCRM	CRM	Customers_DEV	MyERP	CRM	Customers 



This feature is only available in environments that are not Development.

This setting needs to be applied before importing a change into the environment.

Usage of this feature is often combined with using the Source, Schema or Table overwrite functionality so the table name will remain stable in all environments.

3.4.12.3 Data Warehouse Logs

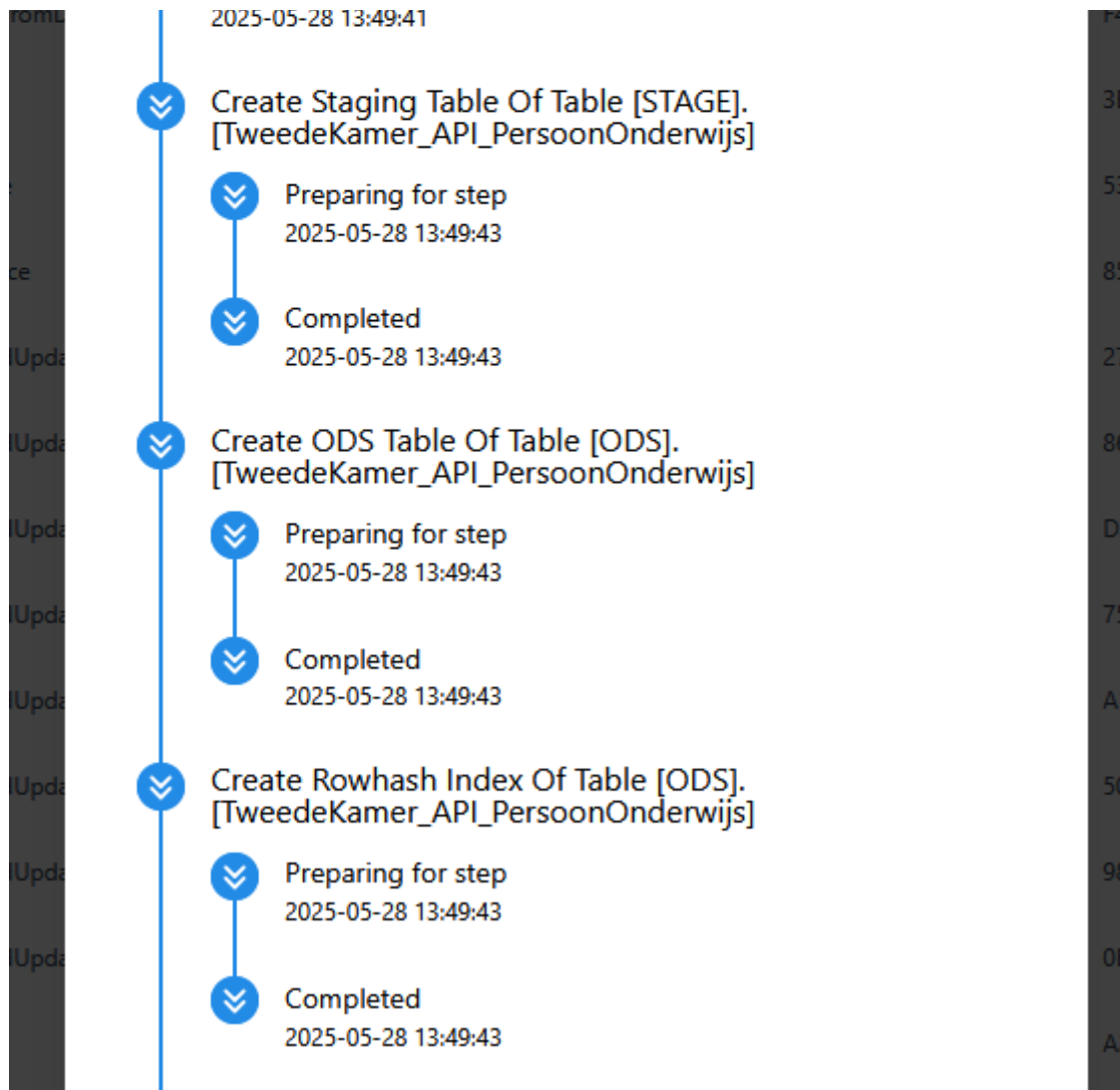
This page allows the user to view and filter logs, which contain information and step by step actions on executed stored procedures. These logs can be filtered by a specific date range, the type of stored procedure and the severity of the log.

Clicking on the info icon shows a step-by-step breakdown of the specified log:

Log Steps

spCreateTablesFromDictionary @EXECUTE: 1
 @SelectedTable: PersoonOnderwijs @SelectedSchema: API
 @SelectedSource: TweedeKamer

- 
INIT
 Start procedure
 2025-05-28 13:49:41
- 
READ WORK PACKAGE
 Gather tables to process
 2025-05-28 13:49:41



3.4.12.4 Environment Settings

The environment tab allows the user to change various Data Warehouse settings per environment, such as Azure service tiers or storing methods.

3.4.12.4.1 List of settings

Setting	Description	Notes
DefaultStore	Choose between row oriented (default) or column oriented data storage. Row storage stores data in rows and is optimized for reading and writing data efficiently. Column storage stores data in columns and is more suited for running queries. This may depend on the type of linked database (Row storage is used by MySQL and column storage is used by Snowflake, for example)	
DefaultKeepStage	To allow for delta loading, a staging table is used to store new changes to allow for a faster loading experience. This setting allows the user to choose between truncating the staging table (NO) or keeping records (YES) after they've been moved to the main table	
DefaultODSMemOptimized	Sets whether or not to fully store ODS tables in memory in a memory-optimized format	
DefaultStageMemOptimized	Sets whether or not to fully store STAGE tables in memory in a memory-optimized format	
DefaultServiceTier	Sets the preferred Azure database subscription tier	
Yres_VERSION	Sets the Yres version to use for the database	This can only be changed by updating the environment
SchemaHIS	Sets the name to be referred to as the HIS schema	
SchemaStage	Sets the name to be referred to as the STAGE schema	
HighServiceTier	Sets the service tier to scale up to when under a higher workload	
StorageSize	Sets the storage size in GB	This is dependent on the DefaultServiceTier

Setting	Description	Notes
AllowUpdatesInYresSchema	Sets whether Yres users can make changes to objects like views and tables maintained by Yres directly from the database	
DefaultSurrogate	Sets whether Yres should automatically create surrogate keys for the natural table key	- These keys are stored in [Loadmanagement].[SurrogateKeys] - This setting can be overwritten for specific tables
AllowDeletesFromDB	Sets whether Yres users can delete scripted objects directly from the database	 If you decide to turn this setting on, be mindful of current changes and scripted objects. Deleting scripted objects in a change may make the change unusable
BIDashboardUrl	Sets the embed URL of a PowerBI report to be used at the PowerBI dashboard tab	
BIDashboardHeight	Sets the height of the displayed PowerBI dashboard	
AllowSettingsUpdates	Sets whether Yres users are prevented from updating the settings directly from the database	
AllowLogManipulation	Sets whether Yres users are allowed to edit logs directly from the database	
AutomaticDatabaseScaling	Sets whether Yres automatically scales the database for managed loads	
UsePagination	Sets whether Yres should chunk large amounts of data into pages	Turning this feature on will override DefaultKeepStage
PageSize	Sets the page size	

Setting	Description	Notes
RetryCount	Sets the amount of times Yres should retry a request	
EnvironmentType	Sets the type of the current environment	

3.4.12.5 Health Checks

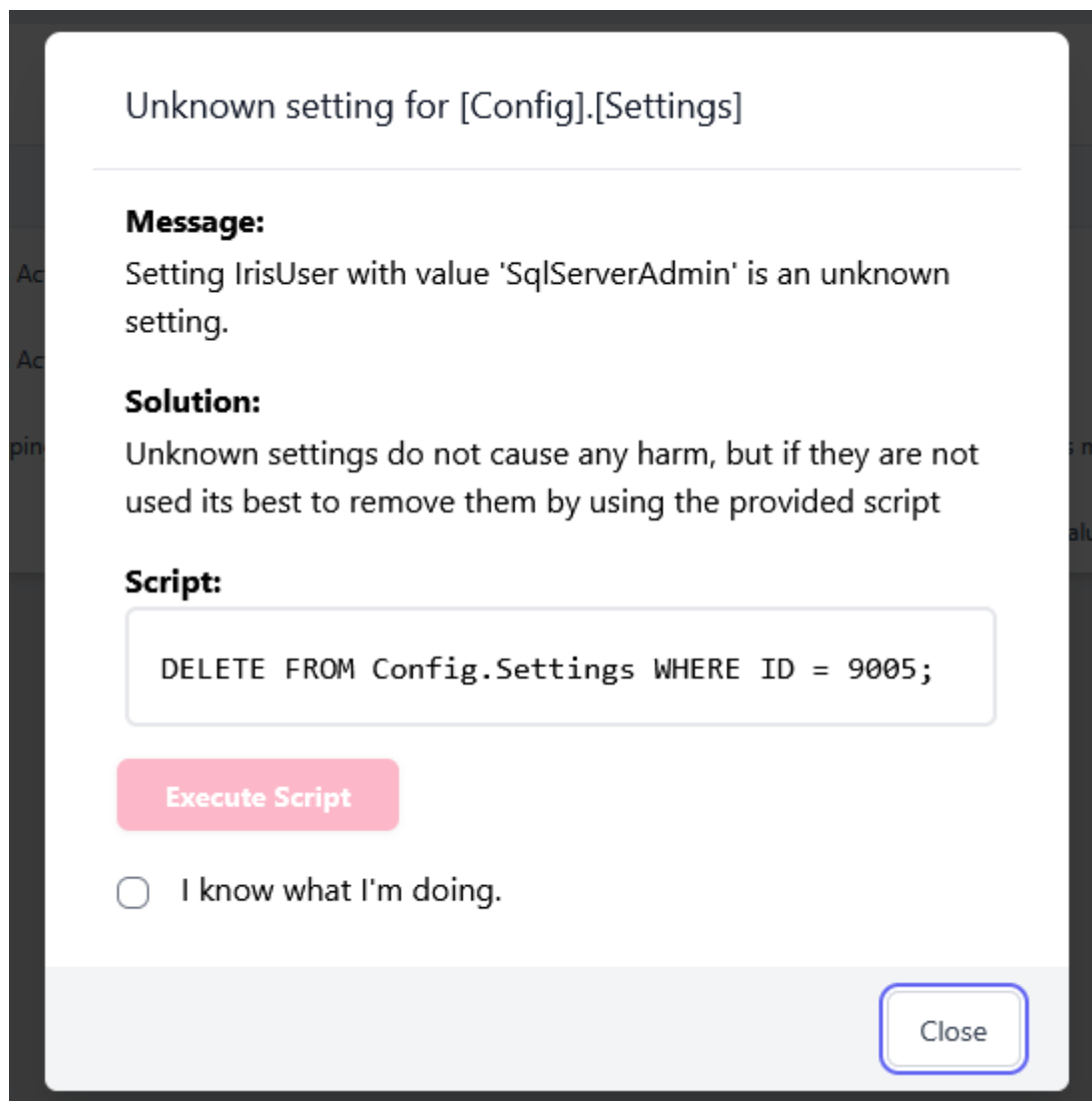
The health checks page shows admins the status of the Data Warehouse (or DWH) side of Yres. This gives an easy insight from the webapp, and in some cases allows the admin to execute a SQL script to fix issues.

3.4.12.5.1 Overview

The health checks overview gives the admin insight of any common issues that have been found by the stored procedure [Maintenance].[vwYresChecks]. These oftentimes come with an error message, and can in some cases come with a script to execute.



Always be careful when executing a script. Consult an Yres admin if you're unsure about the outcome.



9 An example of an entry in health checks, including a script to execute.

3.4.12.6 PowerBI Dashboard

The PowerBI dashboard allows users to link and show a PowerBI dashboard within the Yres webapp. It requires an embed URL from a PowerBI report.

3.5 Projects

The project panel allows the user to manage projects and their changes, as well as managing scripted objects.

Projects are used to categorize changes, which in turn are used to group the work an user has done on Yres.

For more information, see [Changes \(see page 33\)](#).

[Project Overview \(see page 34\)](#) | [Changes \(see page 33\)](#) | [Scripted Objects \(see page 34\)](#) | [Release Changes \(see page 35\)](#) | [Install Changes \(see page 33\)](#)

3.5.1 Changes

Changes are used to categorize edits made to the Yres data warehouse, such as updating tables or adding scripted objects. These changes can then be released and subsequently installed to different environments to ensure environments are running with the same data.

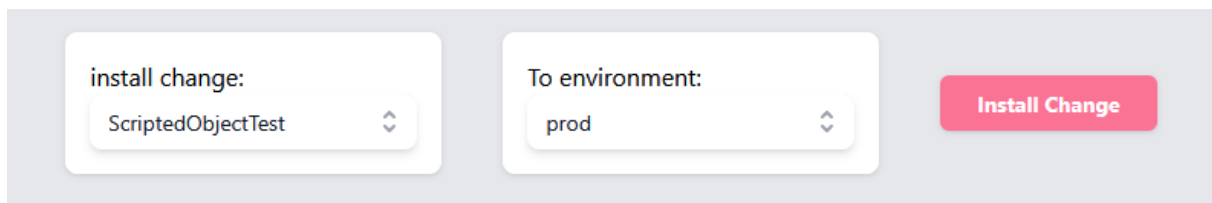
Changes must be given a due date. This due date must be equal to or before the due date of their respective projects.

3.5.2 Install Changes

The install changes page allows the user to install changes to specific environments (e.g. from pageing to production).

3.5.2.1 Installing a change

To install a change to the desired environment, utilize the dropdowns at the top of the page:



10 A user example, which will install the change named ScriptedObjectTest to the prod environment

3.5.2.2 Reimporting or reinstalling an installed change

Below these dropdowns are two tables showing the released changes and the changes installed to the specified environment respectively. Inside the table showing installed changes, users can choose to reimport () or reinstall () already installed changes.

3.5.2.2.1 Reimporting a change

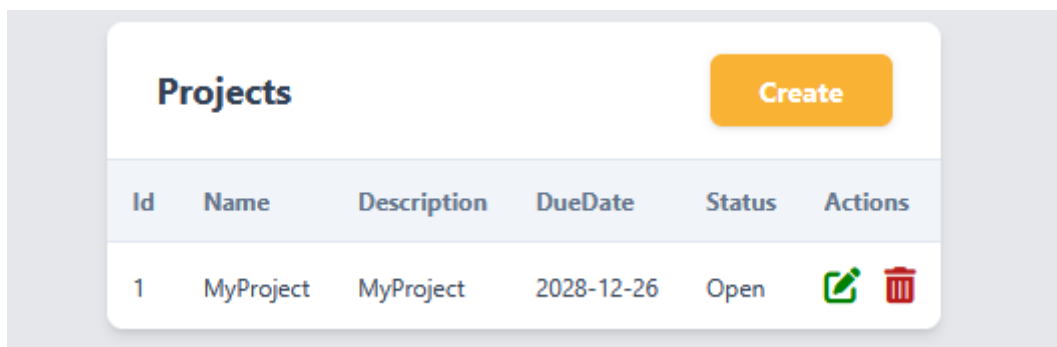
Choosing to reimport a change calls for the stored procedure [Change].[spImport] and refetches the change and project data, including its dependencies and content.

3.5.2.2.2 Reinstalling a change

Reinstalling a change calls for the stored procedure [Change].[spInstall] and installs the change to the environment.

3.5.3 Project Overview

Projects are used to manage changes made to the data warehouse within an Yres environment, such as added tables or custom objects. Users are able to release changes to selected environments under the aforementioned projects.



Projects					
Id	Name	Description	DueDate	Status	Actions
1	MyProject	MyProject	2028-12-26	Open	 

11 The projects menu, showing an open project called MyProject.

Projects can be given a name and a description. Projects also have due dates. The due dates of any changes made under a project must also be before its due date.

Projects cannot be deleted if they still contain open changes.

3.5.4 Scripted Objects

The Scripted objects tab can be used to manage custom SQL objects not generated by Yres, such as custom tables, stored procedures or functions. These scripted objects are added under a change and can therefore be released and installed to other environments.

Scripted Objects					Create
Actions	Change	ObjectName	ObjectType	IfExists	
 	MyChange	[CustomIris].[Extractor]	Custom table (Not generated by IRIS)	UPDATE	
 	MyChange	[dbo].[Shipments]	Custom table (Not generated by IRIS)	UPDATE	
 	2ndChange	[dbo].[AddShipment]	SQL stored procedure	UPDATE	
 	2ndChange	[dbo].[Shipments]		UPDATE	
 	MyChange	[dbo].[AddShipment]	SQL stored procedure	UPDATE	
 	MyChange	[dbo].[OrderDetails]		UPDATE	
 	MyChange	[dbo].[Orders]		UPDATE	
 	MyChange	[dbo].[Products]		UPDATE	

12 An example list of scripted objects, like SQL stored procedures and custom tables.

When adding a custom table as a scripted object, you can choose whether it will be added with or without its contents by selecting the “With table content?” option.

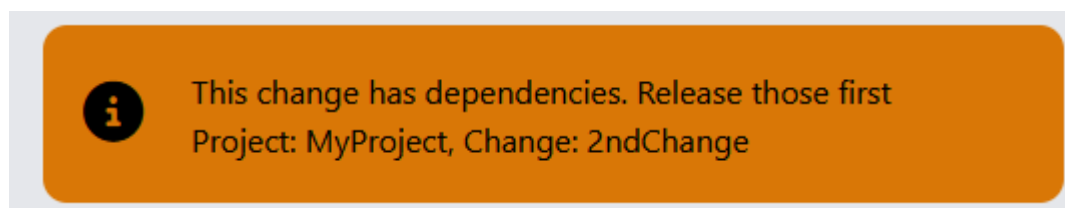
3.5.5 Release Changes

The release changes page allows users to release a change in order to make it available to install to other environments, and allows change content to be viewed in two different formats.

3.5.5.1 Releasing a change

There are a couple things of note about releasing a change:

1. Changes cannot be edited after they’ve been released.
2. Yres blocks releasing a change when a change contains content that another change is dependent on. Yres will give an error if this is the case, citing the dependent change or changes:



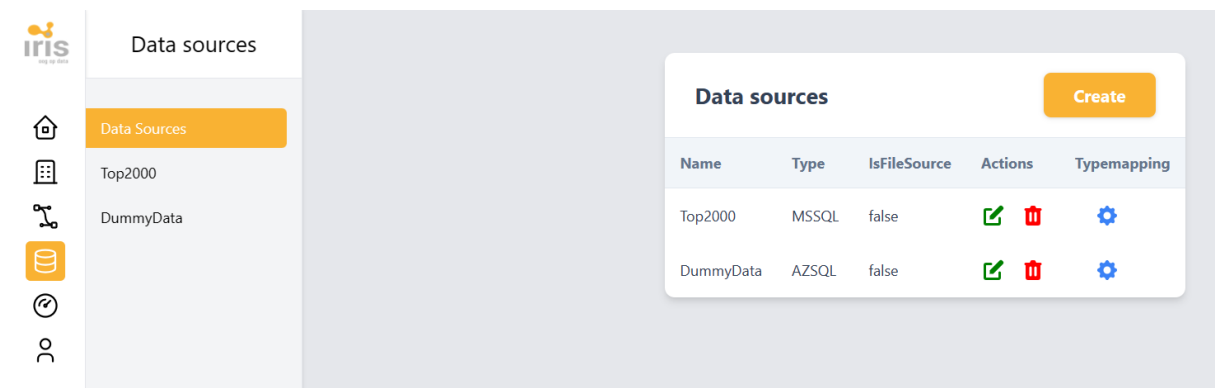
3.5.5.2 Change content

Viewing content of a change can be done via a diagram or a table. The diagram option shows the source, schema and table in a tree view per source, which can be expanded when clicking on a node. It’s recommended for when you have a change with a lot of different schemas and tables inside one change and need a quick overview.

The table option shows a table with every object within a change, along with more information like its load type, the delta column in case of a delta load and what happens if the object already exists.

3.6 Data Sources

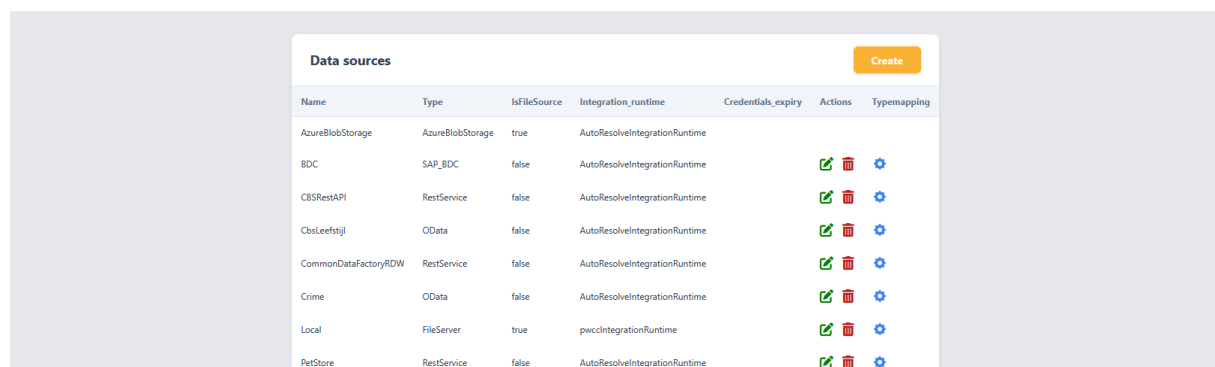
Data sources are the heart of Yres; they are necessary for it to function. The data sources panel allows the user to add and manage data sources, as well as generating typemapping for sources and refreshing their metadata.



[Sources \(see page 36\)](#) | [Added Sources \(see page 38\)](#)

3.6.1 Sources

The data sources page allows users to link a variety of different data sources for use in loads. Yres supports a host of different sources, each with different requirements. To find more information on this topic, see [Data Source Requirements \(see page 81\)](#).



13 The data sources page, showing added sources

3.6.1.1 Overview

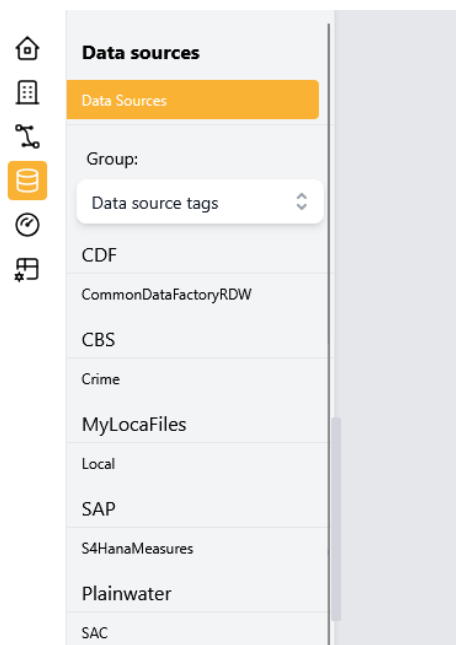
The page itself shows a list of added data sources, where you can manage its properties. Along with basic information like the name, type or credential expiry, there are a couple of options worth going into detail:

3.6.1.1.1 Integration runtimes

Users can specify the integration runtime used for a specific source; the default is `AutoResolveIntegrationRuntime`. If needed, users can create custom integration runtimes in ADF or via the [Shared integration runtimes \(see page 23\)](#) page. Yres will automatically detect these and allow them for use with data sources.

3.6.1.1.2 Tags

Tags can be added to a source to make sorting it easier. At the sidebar, users can group data sources by type and by tag.



14 An example of data sources grouped by tags

Tags are comma-separated (e.g. "sales, salesforce"), so you can add multiple tags to a source to make it show up in multiple groups.

3.6.1.2 Global typemapping

Clicking on the blue cogwheel opens the global typemapping page. Global typemapping allows the user to unify specific data types and properties, which might vary from source to source. This allows for a smoother experience when creating views which combine different data source types.

ID	Source Type	Source Data Type	Target Data Type	Has Length	Has Precision
1	MSSQL_ADF	AccountNumber	nvarchar	1	0
2	MSSQL_ADF	datetime	datetime	0	0
3	MSSQL_ADF	decimal	decimal	0	1
4	MSSQL_ADF	Flag	bit	0	0
5	MSSQL_ADF	int	int	0	0
6	MSSQL_ADF	money	decimal	0	1
7	MSSQL_ADF	Name	nvarchar	1	0
8	MSSQL_ADF	NameStyle	bit	0	0
9	MSSQL_ADF	nchar	nchar	1	0
10	MSSQL_ADF	numeric	numeric		1
11	MSSQL_ADF	nvarchar	nvarchar	1	0
12	MSSQL_ADF	OrderNumber	nvarchar	1	0
13	MSSQL_ADF	Phone	nvarchar	1	0
14	MSSQL_ADF	smallint	smallint	0	0
15	MSSQL_ADF	sysname	nvarchar	1	0

3.6.2 Added Sources

After adding a new source, a couple of things happen inside the DWH:

- A couple of SQL tables get filled with information about the added source, such as the name, type and if needed some basic connection information.
- Every source except file sources and REST services gets stored inside a dictionary containing source metadata. This metadata contains useful information such as tables, available fields and table keys and relations.



It is advised to manually refresh source metadata once the source has been created to ensure optimal compatibility.

Last run: 19-05-2025 12:07:21 @

Status: Succeeded

Refresh Metadata

3.6.2.1 Adding tables

After the source has been created and metadata has been refreshed, you can start to add tables or data integrations to a source. The creation wizard allows for useful features like typemapping and load types to be set per table, allowing for more control over data structure and loading.

For file sources like Azure blob storage, users can upload files directly and add table files.

3.6.2.2 Actions

Depending on the type of added tables, users can perform actions other than the usual edit and delete:

Type	Available for	Description
Compare metadata	All sources except file sources and RestService	Shows if there are any discrepancies in the metadata. Allows for comparison in HIS and STAGE tables
Column info	All sources except file sources and RestService	Shows the table columns, along with their source and target data types
Activate/deactivate	All sources	Determines whether the table is included in future loads or not

3.7 Load Management

The Load Management panel allows the user to manage loading of database information via pipelines, their respective triggers as well as monitoring the pipeline targets and managing persisted views.

[Master Pipelines](#) (see page 41) | [Monitoring](#) (see page 41) | [Run Pipelines](#) (see page 39) | [Triggers](#) (see page 40)

3.7.1 Run Pipelines

In the Run Pipelines tab, users are able to manually start multiple pipelines, such as loading a certain table or retrieving metadata. Custom pipelines made in ADF will also show up in the menu on the left.

Using the filters on the top right, users can filter records by a date range and status. Additionally, the Dynamic Yres Workflow supports filters for source, schema and table.

The screenshot displays the 'Run Pipelines' interface within the 'Load Management' section. On the left, a sidebar contains navigation icons for Home, Load Management (selected), Master pipelines, Triggers, and Monitoring. The 'Load Management' panel shows a 'Select Pipeline' dropdown menu with options like 'Dynamic Workflow IRIS', 'Refresh All Metadata', 'AdaptiveIndexDefragmentation', 'GarbageCollection', and several 'MasterPipeline' entries. The main area features a table of pipeline runs with filters for 'Run end', 'Status', 'Source', 'Schema', and 'Table'. The table header is 'Runs: Dynamic Workflow IRIS'. The table contains 12 rows of data, each with columns for Source, Schema, Table, Status, RunStart, RunEnd, Runtime, and Actions. A 'Set parameters to start' button is located in the top right corner of the table area.

Source	Schema	Table	Status	RunStart	RunEnd	Runtime	Actions
Tweedelamer	API	Commissie	InProgress	15-07-2025 13:46:21	15-07-2025 14:23:46	37m 25s	
Simplicite	hours/hours	AUTO	Failed	15-07-2025 09:41:00	15-07-2025 09:51:06	10m 6s	
CosLeefstijl	API	TypesDataSet	Succeeded	14-07-2025 23:50:00	15-07-2025 03:14:25	3h 24m 25s	
Local	LocalFiles	IRIS	Failed	14-07-2025 23:00:01	14-07-2025 23:07:34	7m 33s	
ALL	ALL	ALL	Failed	14-07-2025 10:07:41	14-07-2025 10:57:25	49m 44s	
Local	LocalFiles	IRIS	Failed	13-07-2025 23:00:00	13-07-2025 23:08:53	8m 53s	
Local	LocalFiles	IRIS	Failed	12-07-2025 23:00:00	12-07-2025 23:04:00	4m 0s	
Local	LocalFiles	IRIS	Failed	11-07-2025 23:00:01	11-07-2025 23:06:16	6m 15s	
Local	LocalFiles	IRIS	Failed	10-07-2025 23:00:00	10-07-2025 23:07:13	7m 13s	
Local	LocalFiles	IRIS	Failed	10-07-2025 09:03:00	10-07-2025 09:08:36	5m 36s	
Local	LocalFiles	IRIS	Failed	09-07-2025 23:00:01	09-07-2025 23:04:15	4m 14s	

3.7.2 Triggers

Triggers allow the user to schedule pipelines with a specific repetition pattern. This allows the pipeline to be triggered repeatedly.

3.7.2.1 Creating a new trigger

1. Select a pipeline in the menu on the left.
2. Click on the Create button.
3. Fill out the form with the desired data. Then, click on the Submit button.

Form.modal.createTrigger

✓

2

Timezone UTC(+1) will be used

Name

Every

Frequency

Week(s) 

On

Monday 

Time

16:00 

Next Step

3.7.3 Monitoring

The monitoring page allows users to view the load history of a specific table, as well as view detailed log steps and rollback or reset tables.

Targets			CommonDataFactoryRDW_Masterdata_Bedrijven								Reset Table
Target	Load Date	Latest Load	Actions	Status	DateTime	Load type	Runtime	Copied	New	Delta	
BDC		SUCCEEDED		FAILED	2025-05-13 17:07:37	FULL	143	0	0	0	
CbsLeefstijl		SUCCEEDED		FAILED	2025-05-13 15:56:10	FULL	135	0	0	0	
API		SUCCEEDED		FAILED	2025-05-12 20:49:01	FULL	106	0	0	0	
TypedDataSet	2025-05-28 10:34:32	SUCCEEDED		FAILED	2025-05-12 20:40:06	FULL	141	0	0	0	
CommonDataFactoryRDW		FAILED		FAILED	2025-05-11 16:18:19	FULL	112	0	0	0	
Masterdata		FAILED		FAILED	2025-05-09 07:34:20	FULL	124	0	0	0	
Bedrijven	2025-05-13 17:07:20	FAILED		FAILED	2025-05-08 08:37:58	FULL	116	0	0	0	
Crime		SUCCEEDED		FAILED	2025-05-07 06:55:19	FULL	178	0	0	0	
Local		FAILED		FAILED	2025-05-06 00:27:12	FULL	392	30477	0	0	
PetStore		FAILED		FAILED	2025-05-05 23:40:26	FULL	323	0	0	0	
RickAndMorty		SUCCEEDED		FAILED	2025-05-02 22:16:31	FULL	217	0	0	0	
S4HanaMeasures		SUCCEEDED		FAILED	2025-05-02 21:22:35	FULL	100	0	0	0	
SAC		SUCCEEDED		SUCCEEDED	2025-04-22 02:31:46	FULL	408	30470	0	0	
SapS4ContractDocument		FAILED		SUCCEEDED	2025-04-22 02:21:53	FULL	414	30470	0	0	
Simplicate		FAILED		SUCCEEDED	2025-04-22 01:31:30	FULL	635	30470	30349	0	
SimplicateDefault		FAILED		FAILED	2025-04-17 21:21:44	FULL		30628	0	0	
Snowflake		FAILED		FAILED	2025-04-16 14:00:00	FULL		30628	1000	0	
TweedeKamer		SUCCEEDED		FAILED	2025-04-03 10:24:48	FULL	472	30570	1000	0	
TweedeKamerRest		SUCCEEDED		SUCCEEDED	2024-12-31 14:01:00	FULL	249	30302	30177	0	

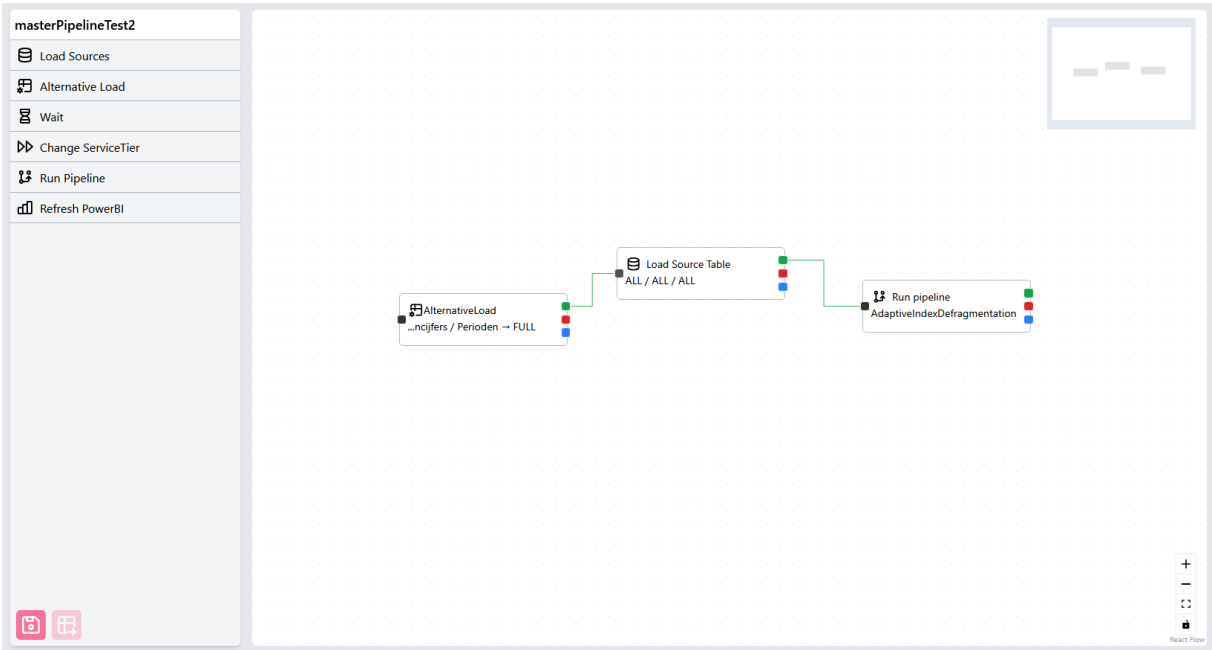
The menu tree on the left allows users to navigate to specific tables following a source > schema > table structure. Tables show their latest load date and status. The worst status bubbles up to the schema and source folders to easily discern failed loads at a glance.

Clicking on a source fetches its load history. Users can view the detailed steps of a load, along with any error messages and a link to the corresponding ADF pipeline. Runs can also be rolled back with the clock icon, or the table can be reset in its entirety.

3.7.4 Master Pipelines

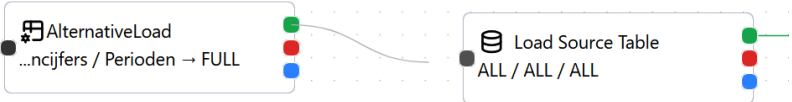
Master pipelines are a way for users to design ADF pipelines with custom flows from the frontend.

To create a pipeline, simply enter a name for it and click on the edit button.



3.7.4.1 Navigation and basic usage

The master pipeline interface utilizes [reactflow](https://reactflow.dev/)¹ in its UI, and therefore uses the standard controls that come with the package. Additionally, there are some simple navigation controls to take note of as well.

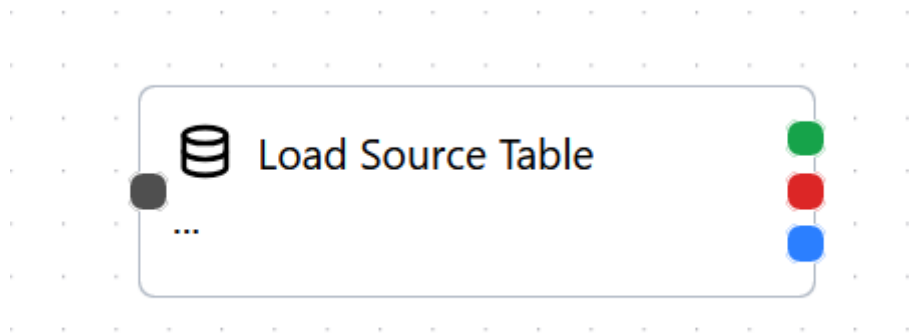
Nodes	
Adding a node	Drag a node type from the left side of the screen and drop it onto the grid
Moving a node	Drag the node with the mouse, or click on it and use the arrow keys
Deleting a node	Click on the node you wish to delete and press the Backspace key
Connections <i>(edges in reactflow)</i>	
Adding a connection	Select one of the points on the right side of the node and drag it to the dark point on the left side of another node 

1. <https://reactflow.dev/>

Nodes	
Deleting a connection	Click on the connection you wish to delete and press the Backspace key

The save button on the bottom left saves the master pipeline as a draft. The publish button becomes available after the save button has been used and publishes the pipeline to ADF.

3.7.4.2 Nodes



Nodes are actions that you can execute based on the connection from a previous node. These three connections on the right side of a node, from top to bottom, are:

- On success
- On failure
- On completion

This allows you to build logic-based pipeline flows with a variety of node types.

3.7.4.2.1 Node types

Type	Description	Notes
 Load Sources	Executes the Dynamic Workflow Yres pipeline. Requires a source, schema and table	
 Alternative load	Executes the Dynamic Workflow Yres pipeline with the added option of altering the load type. Requires a source, schema, table and load type	Only FULL, IMAGE, OVERWRITE and RELOAD are supported
 Wait	Forces the pipeline to wait until a specific time	This accounts for the timezone specified in the user settings

Type	Description	Notes
 Change ServiceTier	Changes the service tier for the pipeline to run at	Service tiers below S3 do not support a Columnstore index. Service tiers below P1 do not support storing tables in memory
 Run Pipeline	Executes other Yres pipelines, including custom pipelines and master pipelines	
 Refresh PowerBI	Refreshes a PowerBI model created in Yres	For more information on creating PowerBI models, consult the PowerBI Models (see page 20) section
 Persist View	Executes the Materialize Views pipeline. Requires a source, schema and table	



Note that this list is not final and may be expanded in the future.

3.7.4.3 Publishing a master pipeline

Once the pipeline is ready to be published, save it and click on the export button next to the save button. This translates the flow built to an ADF pipeline flow and publishes the pipeline to ADF, under the MasterPipelines folder:

 PW - Iris	36
 Dynamic Workflow IRIS	
 Refresh All Metadata	
 Custom	2
 MasterPipelines	2
 MasterPipeline - masterpipelineTest	
 MasterPipeline - masterPipelineTest2	

The pipeline can then be executed at the [Run Pipelines \(see page 39\)](#) page.

3.8 Data Engineering

The data engineering panel allows users to manage database objects and view persistence, which allows for greater control over database objects directly from the Yres frontend.

[Database Objects](#) (see page 46) | [View Persistence](#) (see page 45)

3.8.1 View Persistence

In the View Persistence tab, users are able to manage persisted views, which store the results of a view query and allow the user to view the results of tables without having to resend a query, which can take significantly longer.

Using the create form, users can define a source for the persisted view and its destination table, as well as specifying object and load type. Users can also define the level of the persisted view, which regulates in which order it will be loaded. This is useful, for example when a persisted view depends on another persisted view. In this case, users can give the one view a level of 0, and the view that depends on another view a level of 1. This ensures the view with level 0 will be prioritized, ensuring up-to-date data.

Create persisted view

SourceSchemaName
Metadata

SourceViewName
vwUserObjects

DestinationSchemaName

DestinationTableName

Level
0

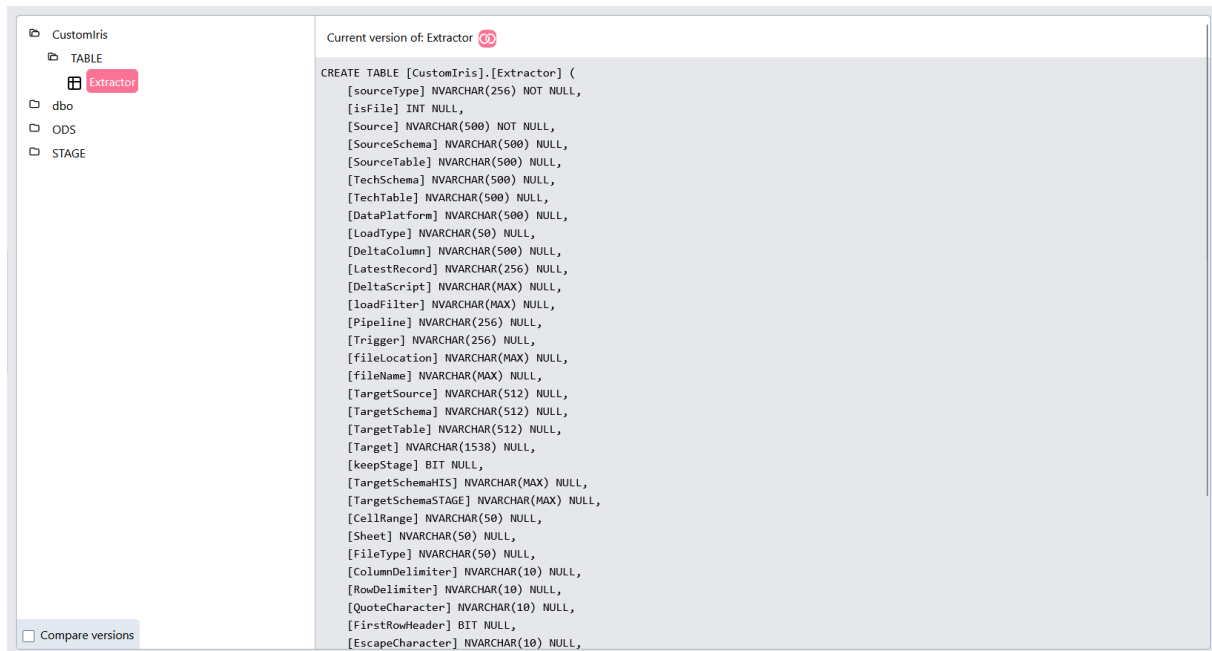
Cancel Next Step

15 A screenshot of the create persisted view modal.

3.8.2 Database Objects

The database objects page allows users to quickly view all database objects created by either a user or Yres itself. It allows for easy access to the definitions of database objects, their version history and dependencies and allows for convenient scripted object management.

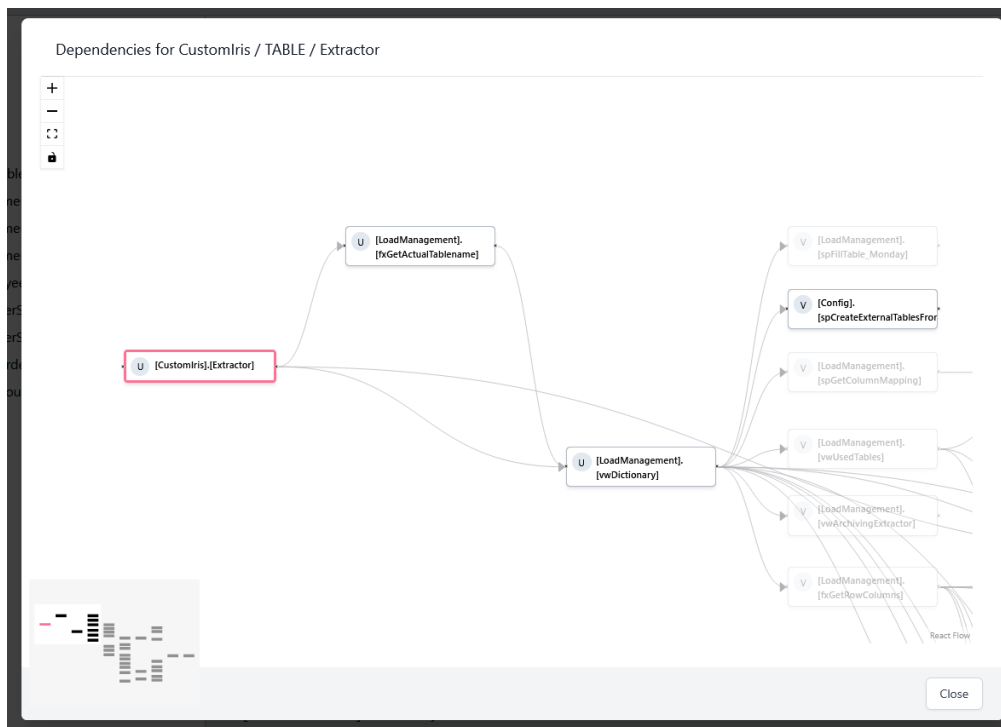
3.8.2.1 Overview



Using the object tree on the left, users can select the object to view its current version.

The button at the bottom left allows for comparing between the current and previous definitions of an object. You can select the versions to compare with the dropdowns at the top.

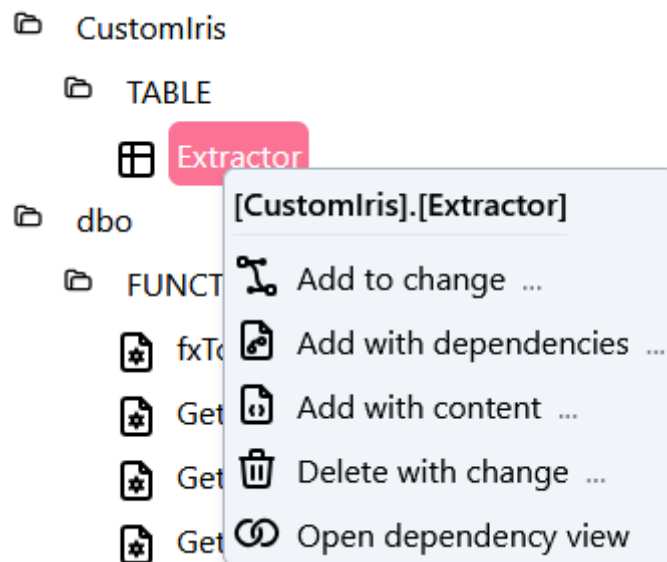
The link icon next to the object name at the top shows all object dependencies, including the objects that the current object depends on and the objects depending on the current object. To give some visual clarity in large dependency trees, you can click on one of the objects to highlight its respective linked objects:



16 The opened dependency modal, highlighting the dependencies for [Config].[spCreateExternalTablesFromDictionary] when clicked

3.8.2.2 Adding objects to a change

To conveniently add objects to a specific change, users can right-click on the object in the object tree to open a context menu:

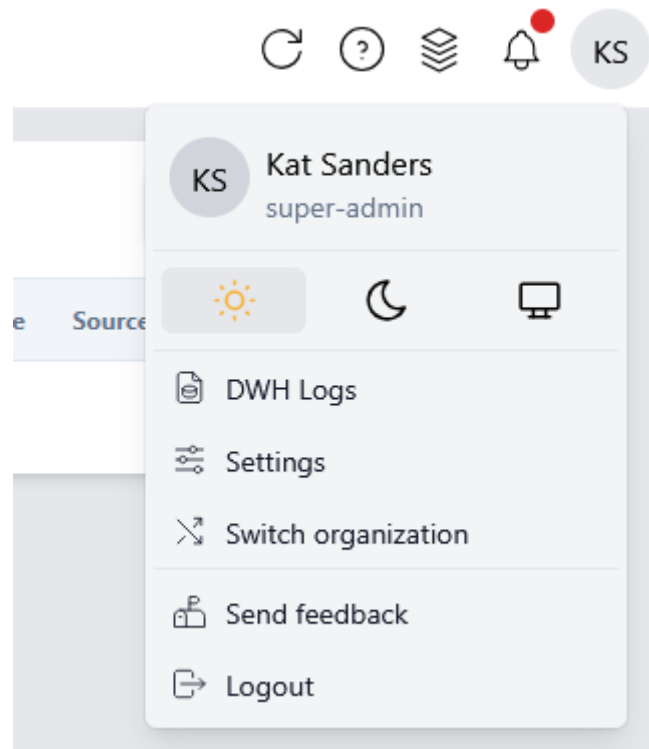


17 An opened context menu, showing several options to add to a change

This gives the user control over how they want to add the specified object to a change, allowing adding it to a change with or without its dependencies, or with its content. Additionally, users can delete an object with a change.

3.9 Account

The account page can be accessed from the topbar, at the icon displaying your username initials. From here, you can do things like switch between a light and dark mode, access your account settings and send feedback.



18 A screenshot of the account panel when clicked.

3.9.1 Account settings

On the account settings page, users can change their timezone, date format and language preferences. Users can also choose to change their password.

3.9.2 Feedback

Users can send feedback straight from the Yres frontend. This sends an email to [feedback@Yres-dwh.nl](mailto:feedback@yres-dwh.nl)², and any replies to the feedback mail will be sent to the email linked with your account.

2. <mailto:feedback@yres-dwh.nl>

4 Setup

This section should guide the user through all the necessary steps to properly set up the Yres environment. Use the sidebar on the left or the search bar on the right to navigate through sections.

4.1 Installing Yres

Yres exists out of a set of Azure resources and templates. During the installation process the Azure resources are created and configured according to the Yres templates. For Yres to gain access to your Azure subscription(s) a Application registration needs to be created with the required roles.

4.1.1 Preparations

To create an Application registration use [this link](#)³ to your Entra environment. If you do not have access please acquire access or get support by someone who does have access before continuing.

Using the link mentioned in the previous paragraph start to create the Application Registration. Name the application as you wish and according to your own naming convention. In this example the application is named Yres-dataplatform-user.

Leave the account type setting on Single tenant and leave the redirect setting empty. Click on "Register" to continue.

3. https://portal.azure.com/#view/Microsoft_AAD_RegisteredApps/CreateApplicationBlade/isMSAApp~/false

Register an application ...

* Name

The user-facing display name for this application (this can be changed later).



Supported account types

Who can use this application or access this API?

- ☒ Accounts in this organizational directory only (I only - Single tenant)
- ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)
- ☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)
- ☐ Personal Microsoft accounts only

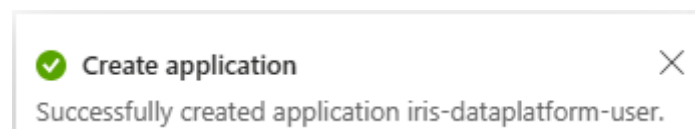
[Help me choose...](#)

Redirect URI (optional)

We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.



After clicking “Register” you will be redirected to the Azure portal main screen while receiving a message that the application has been created successfully.



After this you can find your application in your Entra admin portal or by going to [this link](#)⁴ and searching for the application name used at creation. If you cannot find the application, please make sure you have selected “All applications” above the search bar.

4. https://portal.azure.com/#view/Microsoft_AAD_IAM/ActiveDirectoryMenuBlade/~/_/RegisteredApps

All applications Owned applications Deleted applications

 iris-datapl 

 Add filters

1 applications found

Display name ↑↓

	iris-dataplatform-user
---	------------------------

Once the application is found, please open it by clicking on the name and then navigate to the API Permissions in the menu on the left.

 **iris-dataplatform-user** | API permissions  ...

 Search  Refresh  Got feedback?

- Overview
- Quickstart
- Integration assistant
- Manage
 - Branding & properties
 - Authentication
 - Certificates & secrets
 - Token configuration
 - API permissions**
 - Expose an API
 - App roles
 - Owners
 - Roles and administrators
 - Manifest
- Support + Troubleshooting

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

 Add a permission  Grant admin consent for Plainwater

API / Permissions name	Type	Description	Admin consent requ...	Status
 Microsoft Graph (1) 				
User.Read	Delegated	Sign in and read user profile	No	

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

Click on “Add permissions” and add the “Azure Service Management” permission. To do so, locate the Azure Service Management tile in the list of services and select it.

Request API permissions



Microsoft Graph

Take advantage of the tremendous amount of data in Office 365, Enterprise Mobility + Security, and Windows 10. Access Microsoft Entra ID, Excel, Intune, Outlook/Exchange, OneDrive, OneNote, SharePoint, Planner, and more through a single endpoint.



Azure Batch

Schedule large-scale parallel and HPC applications in the cloud



Azure Communication Services

Rich communication experiences with the same secure CPaaS platform used by Microsoft Teams



Azure Data Catalog

Programmatic access to Data Catalog resources to register, annotate and search data assets



Azure Data Explorer

Perform ad-hoc queries on terabytes of data to build near real-time and complex analytics solutions



Azure DevOps

Integrate with Azure DevOps and Azure DevOps server



Azure Key Vault

Manage your key vaults as well as the keys, secrets, and certificates within your Key Vaults



Azure Rights Management Services

Allow validated users to read and write protected content



Azure Service Management

Programmatic access to much of the functionality available through the Azure portal



Azure Storage

Secure, massively scalable object and data lake storage for unstructured and semi-structured data



Data Export Service for Microsoft Dynamics 365

Export data from Microsoft Dynamics CRM organization to an external destination



Dynamics 365 Business Central

Programmatic access to data and functionality in Dynamics 365 Business Central



Dynamics CRM

Access the capabilities of CRM business software and ERP systems

Once selected a new screen will open, letting you select the related permissions. Select "User impersonation", at the time of writing this is the only option.

Permission	Admin consent required
Permissions (1) ☒ user_impersonation ⓘ Access Azure Service Management as organization users	No

Now click “add permission” to complete. Repeat this process for the Azure keyvault and Azure DevOps permissions.

Commonly used Microsoft APIs



Microsoft Graph
Take advantage of the tremendous amount of data in Office 365, Enterprise Mobility + Security, and Windows 10. Access Microsoft Entra ID, Excel, Intune, Outlook/Exchange, OneDrive, OneNote, SharePoint, Planner, and more through a single endpoint.



Azure Batch
Schedule large-scale parallel and HPC applications in the cloud



Azure Communication Services
Rich communication experiences with the same secure CPaaS platform used by Microsoft Teams



Azure Data Catalog
Programmatic access to Data Catalog resources to register, annotate and search data assets



Azure Data Explorer
Perform ad-hoc queries on terabytes of data to build near real-time and complex analytics solutions



Azure DevOps
Integrate with Azure DevOps and Azure DevOps server



Azure Key Vault
Manage your key vaults as well as the keys, secrets, and certificates within your Key Vaults

Once done, your list of permissions should look like this.

+ Add a permission ✓ Grant admin consent for Plainwater

API / Permissions name	Type	Description	Admin consent req...	Status
▼ Azure DevOps (1)				...
user_impersonation	Delegated	Have full access to Visual Studio Team Services REST APIs	No	...
▼ Azure Key Vault (1)				...
user_impersonation	Delegated	Have full access to the Azure Key Vault service	No	...
▼ Azure Service Management (1)				...
user_impersonation	Delegated	Access Azure Resource Manager as organization users	No	...
▼ Microsoft Graph (1)				...
User.Read	Delegated	Sign in and read user profile	No	...

Once all permissions are in the list, continue to navigate to the Certificates & Secrets page in the menu on the left.

 iris-dataplatform-user | Certificates & secrets ✕ ...

Search ◊ <<  Got feedback?

Overview
Quickstart
Integration assistant
Manage
Branding & properties
Authentication
Certificates & secrets
Token configuration
API permissions
Expose an API
App roles
Owners

Credentials enable confidential applications to identify themselves to the authentication service when receiving tokens at a web addressable location (using an HTTPS scheme). For a higher level of assurance, we recommend using a certificate (instead of a client secret) as a credential.

Application registration certificates, secrets and federated credentials can be found in the tabs below.

Certificates (0) **Client secrets (0)** Federated credentials (0)

A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

+ New client secret

Description	Expires	Value	Secret ID
No client secrets have been created for this application.			

A secret needs to be generated by clicking on “New client secret” and giving the secret a name (i.e. WebApp) . A validity needs to be selected of a maximum of 24 months. We recommend setting this to 24 months, but company policies might apply within you organization.

Add a client secret

Description

WebApp

Expires

730 days (24 months)

A new secret will be generated in the list, please copy the **value** and save it in a secure place. this value is needed later on in the installation. *The value is only visible now and cannot be found after you leave this page! if you failed to save it, please delete the secret and generate a new one.*

Certificates (0) **Client secrets (1)** Federated credentials (0)

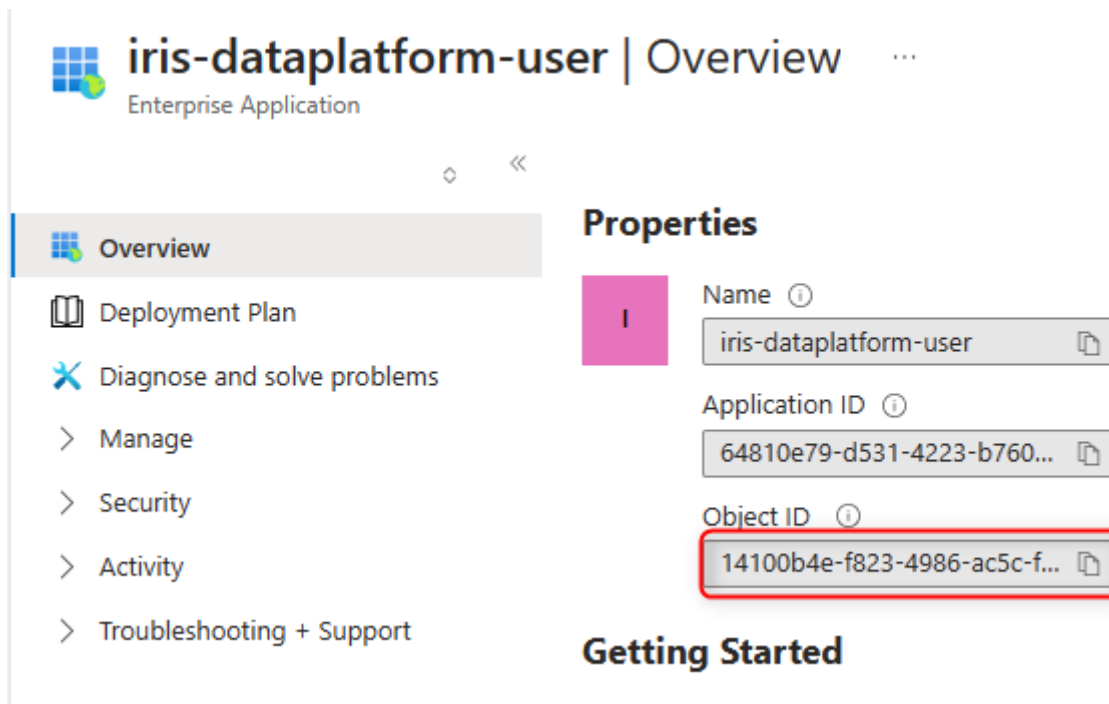
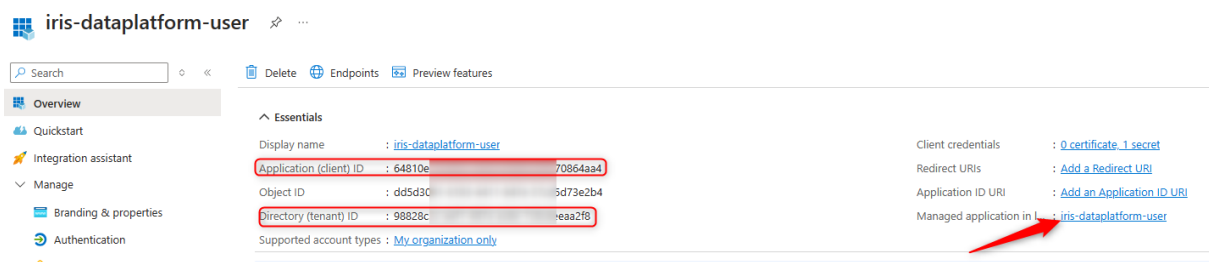
A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

+ New client secret

Description	Expires	Value	Secret ID
WebApp	4/29/2026	JEY8...	5Yo... 4ad4c... acde

The installation requires other values, so lets acquire them. Go to the “Overview” page of the application using the menu on the left.

Copy the Application (client) ID and the Directory (tenant) ID and save them in a secure storage for later. Now click on the Managed Application and locate the ObjectID (*this is not the same as the Object ID in the first screen!*). Also copy that value and store it for later.



The application registration is done, now we need to create the subscription(s) and resource groups.

Yres can handle many different types of setups in Azure. We now need to decide what the setup will be for this installation.

The questions to answer are:

1. How many environments do we need? (dev, page, acceptance, quality, prod)
2. Does it all land in a single subscription, does each environment have its own subscription or are some combined and others alone?
3. Does it all land in a single resource group, does each environment have its own resource group or are some combined and others alone?

We recommend to use 2 or 3 environments, have each environment have its own resource group and place the resource groups in a single subscription or late each have its own subscription.



If required, create one or more subscription(s), please refer to the [Microsoft documentation](#)⁵ for more details. After creating the subscriptions, please check the resource providers according to [this document](#) (see page 79).

If you have dedicated subscriptions for Yres and are willing to add the application as contributor of the subscription, you can skip the following steps and continue to the installation. Yres will create the resource groups for you.

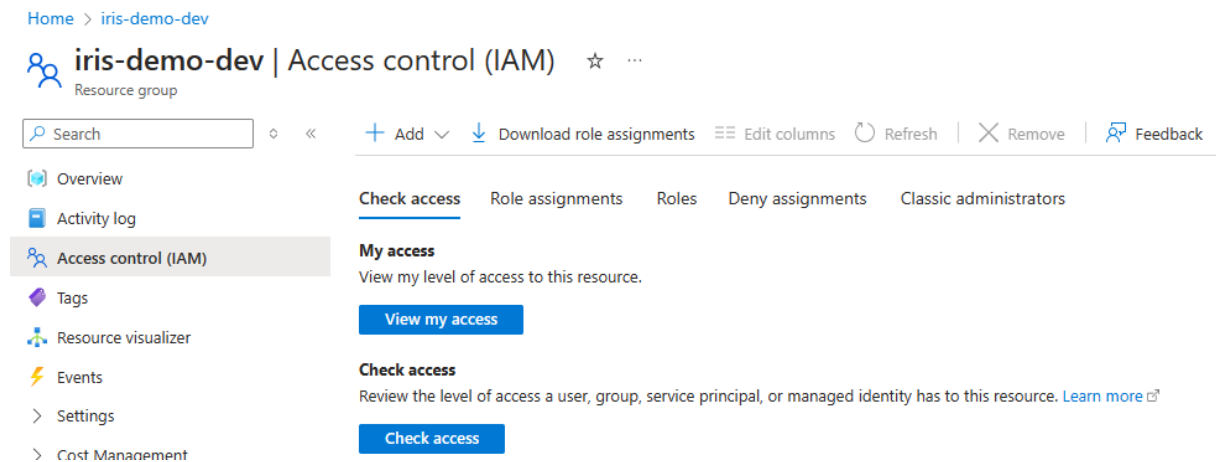
If required, create one or more resource groups, please refer to the [Microsoft documentation](#)⁶ for more details. Please use the environment names in the name of the resource group for development and production we require you to use “dev” and “prod”. Other environments can be named freely although we advise using the following:

dev
 page
 acc
 quality
prod

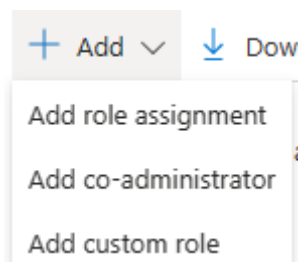
5. <https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/create-enterprise-subscription>

6. <https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/manage-resource-groups-portal>

Once the subscriptions and resource groups have been created, please add the application created earlier as Contributor to the resource groups. Do this by opening the resource group and clicking on Access control in the menu on the left.



Now click on Add and select Add role assignment from the list



Select "Privileged administrator roles" and select Contributor from the list.

Add role assignment ...

[Role](#) [Members](#) Conditions [Review + assign](#)

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

[Job function roles](#) [Privileged administrator roles](#)

Grant privileged administrator access, such as the ability to assign roles to other users.

 Can a job function role with less access be used instead?

contributor	×	Type : All	Category : All
Name ↑↓	Description ↑↓		
Contributor	Grants full access to manage all resources, but does not allow you to as		

Showing 1 - 1 of 1 results.

Click next to continue. In the next screen select “Select members” and find your application in the list.

[Role](#) [Members](#) Conditions [Review + assign](#)

Selected role Contributor

Assign access to ☒ User, group, or service principal
☐ Managed identity

Members [+ Select members](#)

Select members ×

Select ⓘ

 iris-dataplatform-user

Once you've found it, select it and click "Select" at the bottom of the page. you have now added you application to the members of the Contributor role.

Add role assignment ...

Role **Members** Conditions Review + assign

Selected role Contributor

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members [+ Select members](#)

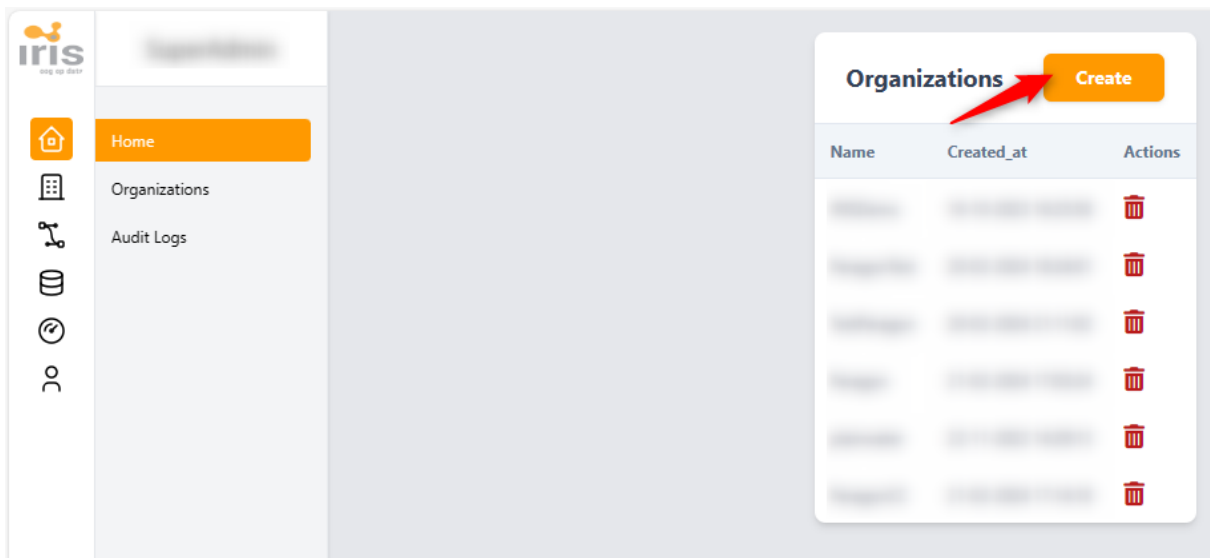
Name	Object ID	Type
iris-dataplatform-user	14100b4e-f823-4986-ac5c-feac5fa09556	App

Description Optional

Finish this step by clicking "Review + assign" at the bottom, twice.

4.1.2 Installation

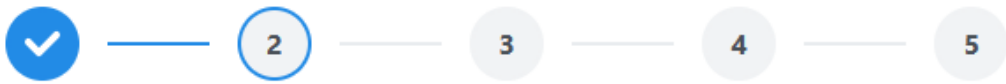
Go to Yres web and [create a new organization](#)⁷.



The Yres installation starts with the Organization name, this name needs to be at least 2 characters long and can only contain letters and spaces ("^[a-zA-Z]*\$").

7. <https://www.yres-web.app/adminpanel/organizations/create>

Create new organization



Name

Next Step

Yres has 2 or more environments. The 2 mandatory environments are called "dev" and "prod". Depending on your license, you can place several environments between dev and prod. For example, consider "page" & "acceptance".

Create new organization



UpdateEnvironments

Define the environments you want to use to for IRIS. Resources like sql-server, database, ADF will be created per environment. Dev and prod environments are required.

Environment

dev

Environment

Remove

Add environment

Production environment

prod

Back

Next Step

The names of your resources can be fully generated or semi generated. The fully generated process creates resource names according to the [Microsoft naming convention](https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/azure-best-practices/resource-naming)⁸. If a name is not available it will be postfixed with a unique string.

The semi generated process lets you change the names, please remember to always use the \$ in the name because this will be replaced with the environment. If the environment is not part of the name, naming conflict will occur between dev, prod and all other environments. This will be solved automatically by applying a postfix to the name, but it makes recognizing the different resources difficult.

8. <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/azure-best-practices/resource-naming>

If you have already created the resource groups, make sure the names match the resource groups created in the preparations.

Create new organization

✓ — ✓ — ✓ — 4 — 5

Resource names

Fill in \$ where the environment name should be placed. e.g. sqlsrv-iris-\$ will become sqlsrv-iris-dev. If a name is already taken, a random string will be put after the defined name.

Generate azure resource names?

Yes ▼

Resource group

rg-iris-\$

Key vault kv-iris-\$	Sql server sqlsrv-iris-\$
Sql server database sqlldb-iris-\$	Azure data factory adf-iris-\$
Azure storage account st1iris1\$	Data lake dls1iris1\$

Back
Next Step

For the final step we need the values saved in the preparations. Fill the first 4 fields with the following values:

Active directory ID → Directory (Tenant) ID

Application Object ID → Object ID

Client ID → Application (client) id

Client secret → Value from Secret

Then choose if you choose to have a single or multiple subscriptions. If you have multiple subscriptions (no matter in what configuration), select Yes.

Fill out the subscription ID for each environment, if you selected no in the previous step the ID will be copied over from dev automatically.

✓

✓

✓

✓

5

Setup azure

Create an active directory app with the following permissions: DevOps, ServiceManagement and KeyVault. Generate a client secret. Application object id can be found by selecting enterprise application.

Active directory id	Application object id
Client id	Client secret

Subscriptions

Different subscriptions for each environment?

Environment	Suscription id
dev	
Environment	Suscription id
test	
Environment	Suscription id
prod	

Back

Next Step

Click next to continue.

Check if you created the resource groups correctly and if the correct roles have bene assigned.

Create new organization



Setup azure 2: permissions

If you have a dedicated subscription only for IRIS, give the AD app owner rights on the subscription. If not create the following resource groups and give the AD app owner rights.

Resource group 1

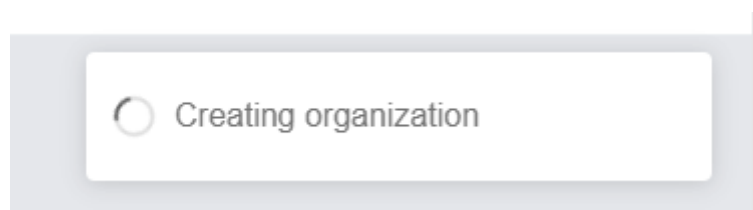
rg-iris-testdaniel-dev

Resource group 2

rg-iris-testdaniel-prod

Back
Submit

Click submit to finish the process, a status will be shown to indicate the process is running



4.2 Getting started with Yres

4.2.1 Logging in to the Yres webapp

Using the password provided to you by Yres, log into your account at [the Login page](#)⁹. After logging in successfully, the Yres webapp should be visible.

4.2.2 UI overview



For more information regarding specific UI elements and how to navigate them, utilize the [Frontend features \(see page 13\)](#) documentation section.

4.2.3 Creating a new organization



It is recommended for new Azure users to regard the [Setting up Azure for use with IRIS¹⁰](#) section first.

Organizations are the basis of Yres and act as a contained space where users can utilize Yres services. Every organization comes with a developer (dev) and production (prod) environment, which are used to allow for separate development with Yres (dev) without it affecting the live version (prod).

To create a new organization, navigate to the SuperAdmin panel.

1. **Enter a name for your organization.**
2. **Add environments.** A dev and prod environment will be created by default.
3. **Generate resource names.** This can be achieved in two ways:
 - **Manually**, by generating the resources specified. Please read the instructions given in Yres carefully in order to avoid screw ups.
If one were to add a keyvault in the dev resource group named “company-keyvault-dev”, the field in Yres should say “company-keyvault-\$”. Additionally, the keyvault generated in a different resource group e.g. prod should be named “company-keyvault-prod”.
 - **Automatically**, by selecting Yes in the *Generate Azure resource names?* field, letting Azure generate the appropriate resources with their default names.
4. **Entering needed Azure IDs.** For this, a Microsoft Entra ID with the permissions ServiceManagement and KeyVault is needed.

The following information should be filled into the appropriate fields:

- The active directory ID (or Tenant ID).
- A subscription ID.
- An application object ID.
- The client ID and client secret.

5. **Setting up Azure permissions.** This will create role groups for both dev and prod environments. Names will be automatically generated based on the resource group (“rg-Yres-dev” and “rg-Yres-prod” by default). When manually creating resource names, the corresponding resource groups

9. <https://www.yres-web.app/login>

10. <https://plainwater.atlassian.net/wiki/pages/createpage.action?fromPageId=2630189060&linkCreation=true&spaceKey=IRIS&title=Setting+up+Azure+for+use+with+IRIS>

should share the same name. For the purposes of demonstration, we have created two resource groups, ending in “-dev” and “-prod”.

Create new organization

✓ — ✓ — 3 — 4 — 5

UpdateEnvironments
 Define the environments you want to use to for IRIS. Resources like sql-server, database, ADF will be created per environment. Dev and prod environments are required.

Develop environment

dev

Add environment
Remove

Production environment

prod

Back
Next Step



Please note that creating a new organization may take a while to finish. This may result in the corresponding frontend elements not working correctly.

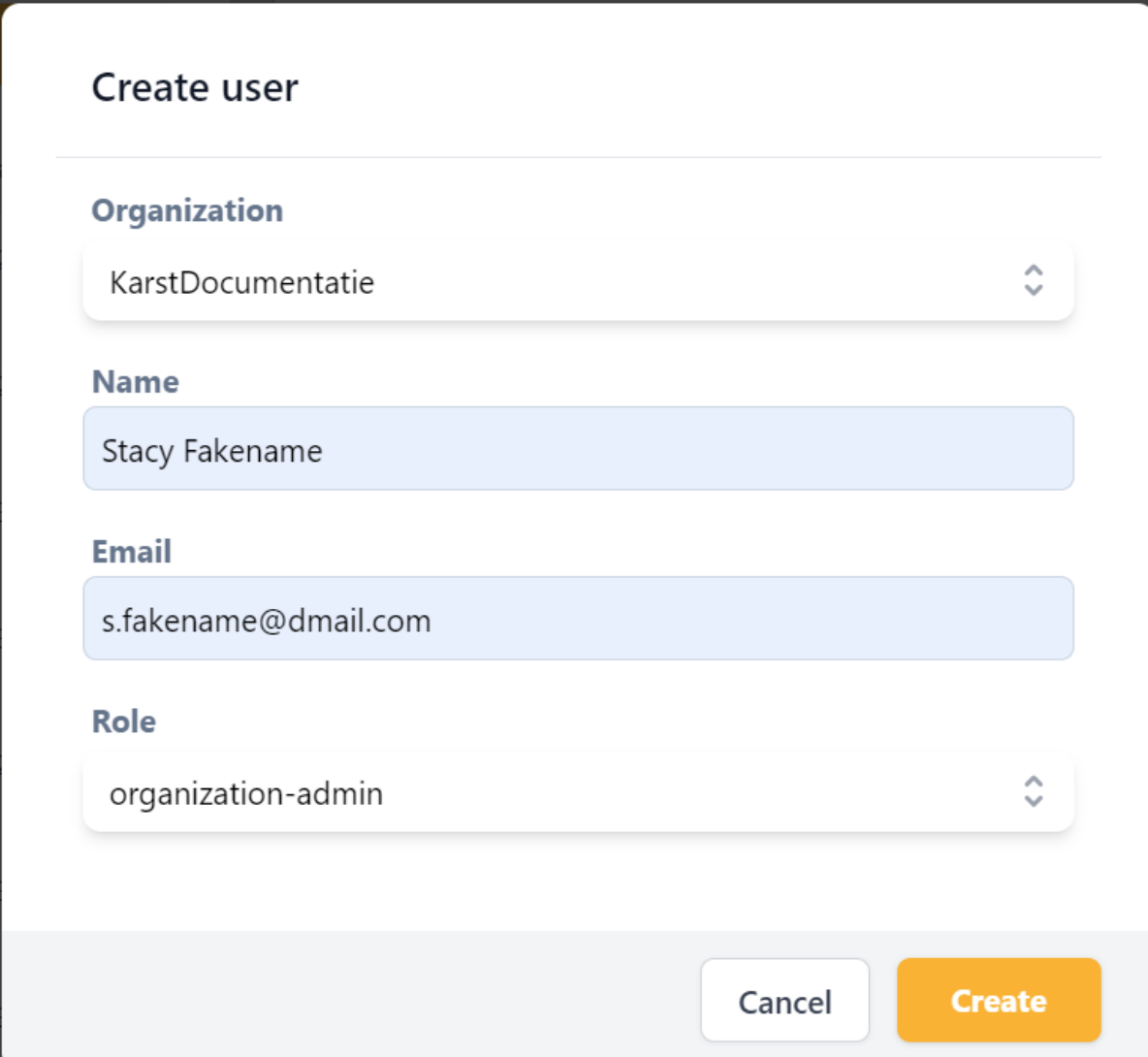
4.3 Users and their roles

For information on how to create users and managing roles, see [the general overview Admin section \(see page 17\)](#).

4.3.1 Managing users

Users can interact with the Yres environment at different levels. At the SuperAdmin home panel, users and their role within the organization can be added (see example below). Specify the organization created in the previous step and enter their name, email and role.

- **Users** cannot manage either the Admin or SuperAdmin panels.
- **Organization Admins** have administrator privileges within the specified organization and can access the Admin panel.
- **System Admins** have administrator privileges over every organization and can access the SystemAdmin panel.



The image shows a 'Create user' form with a light gray background and rounded corners. It contains four input fields: 'Organization' (a dropdown menu with 'KarstDocumentatie' selected), 'Name' (a text field with 'Stacy Fakename'), 'Email' (a text field with 's.fakename@dmil.com'), and 'Role' (a dropdown menu with 'organization-admin' selected). At the bottom right, there are two buttons: a white 'Cancel' button and an orange 'Create' button.

Create user

Organization

KarstDocumentatie

Name

Stacy Fakename

Email

s.fakename@dmil.com

Role

organization-admin

Cancel Create

Created users will also appear at the Admin panel of the specified organization.

4.3.2 Managing roles

At the [Admin panel \(see page 25\)](#), roles can be created to manage user permissions. These can be used to manage more specific permissions and can be assigned to users once created by clicking the green pencil icon next to the appropriate user.

Create role

Name

ExampleRole

☒

Read organization users

☒

Create organization users

☒

Update own user

☒

Update organization users

☐

Delete organization users

☒

Read organization adfs

☐

Create organization adfs

☒

Update organization adfs

☐

Delete organization adfs

☒

Read organization environments

☐

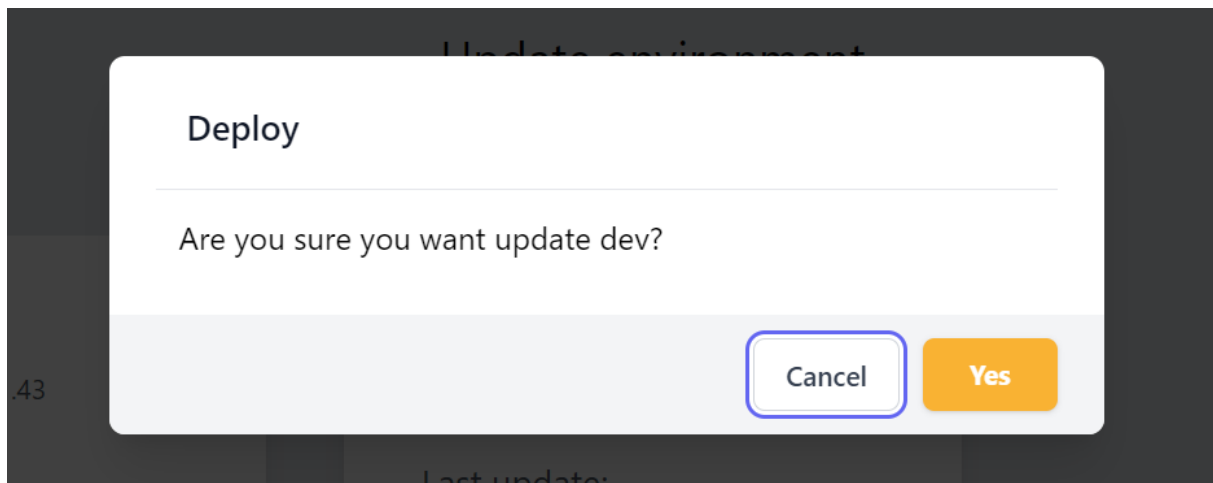
Create organization environments

4.4 Utilizing different environments

Environments are contained versions of an organization and can be utilized to page changes on one version without affecting others. When setting up a new organization, Yres generates a **dev** and **prod** environment by default.

4.4.1 Updating environments

Updating an environment can be done via the Update environment tab at the Admin panel. This also shows the Yres data warehouse version, status, results and when it last ran.



4.4.2 Utilizing projects and changes

Projects are used to categorize any work done on an environment. Changes can be added under a project to allow for new content to be transferred from the development to production environment (dev to prod).

4.5 Linking data sources



For information on how to manage data sources, see [the general overview data sources section](#). (see [page 36](#))

Yres offers a variety of options for sources, like Oracle, Azure or Mysql. The requirements of connecting a data source depend on the source type and can be found at the [Data Source Requirements](#) (see [page 81](#)) section.

Create source

✓

2

Name

Type

SQL Server

MySQL

PostgreSql

Oracle

AFAS

yes

Cancel

Next Step

For the purposes of demonstration, an SQL server will be added. This will require the host, port, database name, username and password.



After creating a new data source, please allow for some time for the source to be published to Azure Data Factory

Create source

✓

✓

SQL Server

Host

Port

Db_name

User_name

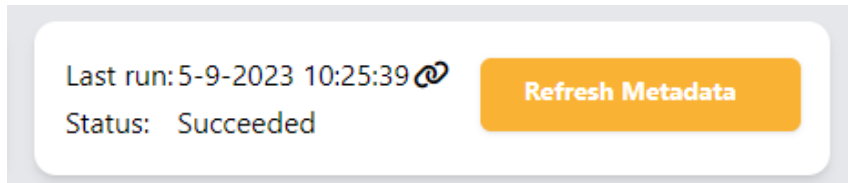
Password

CancelBackCreate

4.6 Loading data sources

4.6.1 Refreshing data source metadata

Yres allows for users to refresh data sources to allow for up-to-date data. To refresh the metadata of a given source, click on the Refresh Metadata button when viewing a data source. Click the link button to view the progress of refreshing metadata.



For more information on how to automate this process, regard the [Setting up pipelines and triggers \(see page 76\)](#) section.

4.6.2 Load types

Below are a list of load types which can be used with loading data sources.

Load type	Description
FULL	Replaces existing records with new records
DELTA	Compares the old and new data and adds changed records.
OVERWRITE	Deletes existing records and adds all new records
RELOAD	Similar to FULL, but uses optimized updates to prevent replacing unchanged data
IMAGE	Does not replace or reload data, but keeps a record of the older data
ADDITIONAL	Adds new records to the dataset but does not replace existing records

4.6.3 Key columns

Key columns are used to uniquely identify columns in a table where column placement might be subject to change. These columns can be specified when adding a data source.

4.7 Generating views

4.7.1 Generating persisted views



Generating a persisted view requires a connection to the Yres SQL database via a program such as [SQL Server Management Studio](https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-ver16)¹¹.

For more information on persisted views, regard the [View Persistence](#) (see page 45) section.

A persisted view requires a view to be created first. In this example, we've created a view named **DM.Vw7602** in SSMS under the sqlDb-Yres-dev database.

```

7 CREATE VIEW [DM].[Vw7602] AS
8 SELECT
9     [anzsic06]
10    , [Area]
11    , [year]
12    , [geo_count]
13    , [ec_count]
14    , [ETL_Date]
15    , [ETL_EndDate]
16    , [KeyHash]
17    , [RowHash]
18    , [Delta]
19    , [IsCurrent]
20 FROM [ODS].[DUMMYDATA_DBO_DATA7602DESCENDINGYEARORDER]
21 GO
22
23

```

After creating a view, it should show up under the specified source schema name in the Create persisted view dialog.

Modal.create.persistView

SourceSchemaName: DM

SourceViewName: Vw7602

Next, specify the destination schema- and table name, as well as the level, source delta object type and load type.

11. <https://learn.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms?view=sql-server-ver16>

Modal.create.persistView

✓

2

SourceSchemaName

DM

SourceViewName

Vw7602

DestinationSchemaName

DM

DestinationTableName

Record7602

Level

0

SourceDeltaObjectType

View

Cancel

Next Step

4.8 Setting up pipelines and triggers

At the [Load Management panel](#) (see page 39), pipelines can be manually run and their triggers can be set up. Additionally, users can monitor ran pipelines.

The “Refresh All Metadata” pipeline can be ran via the Start Pipeline button once selected on the left side. In order to run the Dynamic Workflow Yres pipeline, users must set the source, source schema and source table, as well as the running tier and which tier to revert to.

Set parameters

Source

ALL

SourceSchema

SourceTable

RunningTier

Current

RevertToTier

Previous

Cancel

Start Pipeline

Additionally, users can create triggers to automate running pipelines. These are configured to run on regular intervals like every week, as shown in the example.

Form.modal.createTrigger

✓

2

Timezone UTC(+1) will be used

Name

Every

Frequency

On

Time

Cancel

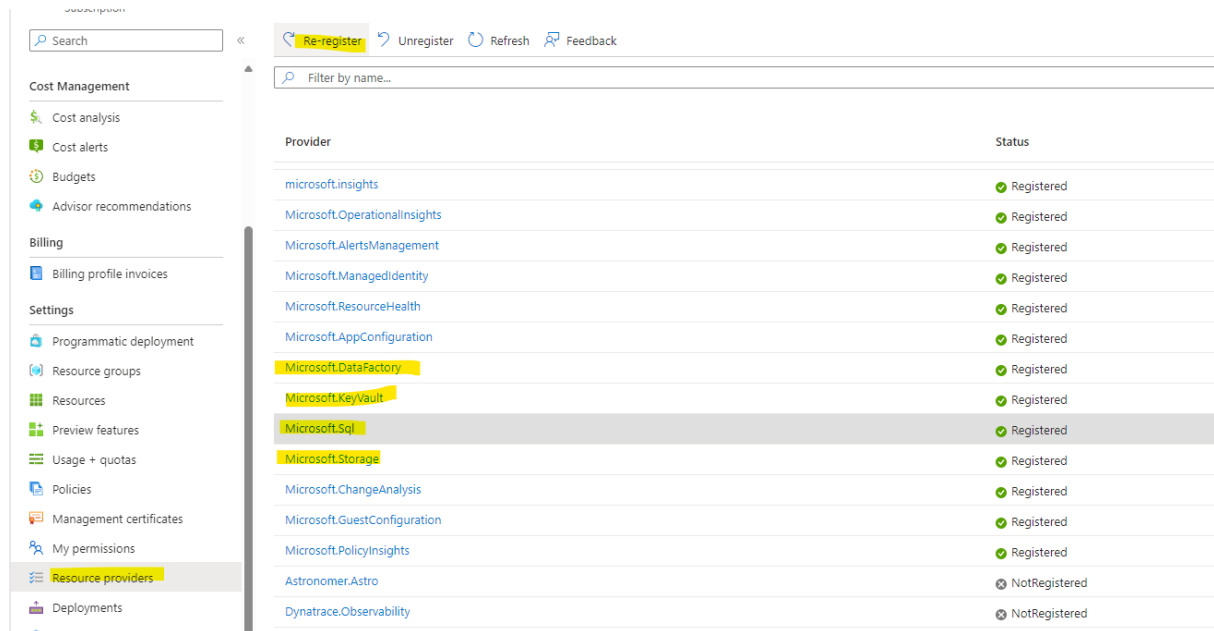
Next Step

5 Troubleshooting

Welcome to the troubleshooting section for Yres. This section should guide the user when it comes to fixing mistakes, as well as give information on error codes if needed.

5.1 1. Deployment rights error

If on an Azure subscription a resource has not been created by a user, an API call cannot make it aswell. In this case, register the resources:



5.2 2. Unable to load local files

When trying to load local files, ADF returns this error:

The value of the property " is invalid: 'Access to serverName is denied, resolved IP address is ::1, network type is OnPremise'. Access to serverName is denied, resolved IP address is ::1, network type is OnPremise

To load local files using ADF Integrated Runtime you need to set -EnableLocalMachineAccess. To do so run the statement below in CMD or Powershell on the Windows server running the IR software

```
cd "C:\Program Files\Microsoft Integration Runtime\5.0\Shared\  
.\dmgcmd.exe -EnableLocalMachineAccess
```

6 Data Source Requirements

Welcome to the data source requirements section. This section describes requirements and interactions of all compatible data sources within Yres.

6.1 Databases

6.1.1 MySQL

6.1.1.1 Requirements for connecting with Yres

MySQL requires the following:

- Host
- Port
- Database name
- Username
- Password

Create source



Host

Database name

Port

User name

Password

Cancel

Back

Create

6.1.2 DB2

6.1.2.1 Setting up the data source to link with Yres

6.1.2.2 Requirements for connecting with Yres

DB2 requires the following:

- Host
- Port
- Database name
- Username
- Password

Create source



DB2

Host

somedatabase.iris.dev

Port

1433

Db_name

Top2000

User_name

s.fakename@dmil.com

Password

.....

Cancel

Back

Create

6.1.3 SQL Server

6.1.3.1 Setting up the data source to link with Yres

6.1.3.2 Requirements for connecting with Yres

SQL Server requires the following:

- Host
- Port
- Database name
- Username
- Password

Create source

✓

✓

SQL Server

Host

Port

Db_name

User_name

Password

CancelBackCreate

6.1.4 Azure SQL Database

6.1.4.1 Setting up the data source to link with Yres

6.1.4.2 Requirements for connecting with Yres

An Azure SQL Database requires the following:

- Host
- Port
- Database name
- Username
- Password

Create source

✓

✓

Azure SQL Database

Host

Port

Db_name

User_name

Password

CancelBackCreate

6.1.5 Oracle

6.1.5.1 Requirements for connecting with Yres

Oracle requires the following:

- Host
- Port
- Service name
- Username
- Password

Create source

✓

✓

Host

Port

Service name

User name

Password

Cancel

Back

Create

6.1.6 PostgreSQL

6.1.6.1 Setting up the data source to link with Yres

6.1.6.2 Requirements for connecting with Yres

PostgreSQL requires the following:

- Host

- Port
- Database name
- Username
- Password

Create source

✓

✓

PostgreSql

Host

Port

Db_name

User_name

Password

CancelBackCreate

6.1.7 Snowflake

6.1.7.1 Setting up the data source to link with Yres

6.1.7.2 Requirements for connecting with Yres

Snowflake requires the following:

- Account name
- Username
- Password
- Database
- Warehouse
- Role (optional)

Create source

✓

✓

Snowflake

Account name

User name

Password

Database

Warehouse

Role

Cancel

Back

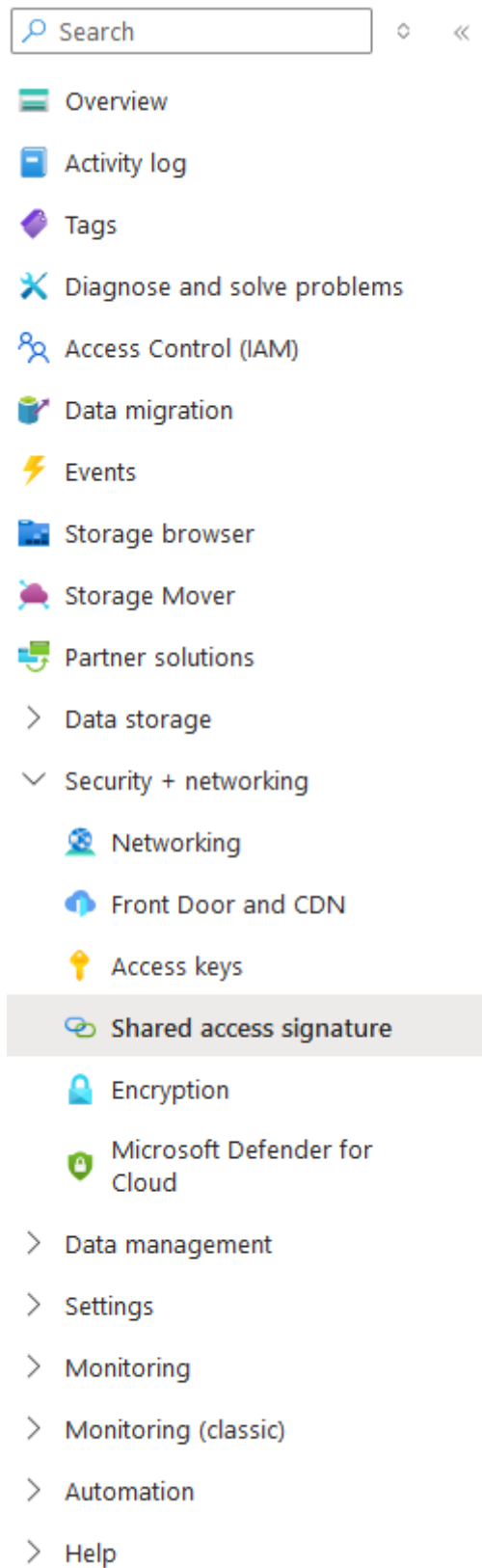
Create

6.2 Azure

6.2.1 Azure Blob Storage

6.2.1.1 Setting up the data source to link with Yres

A SAS token is required. Create one by going to the specified blob storage account and navigating to Security + Networking → Shared Access Signature.

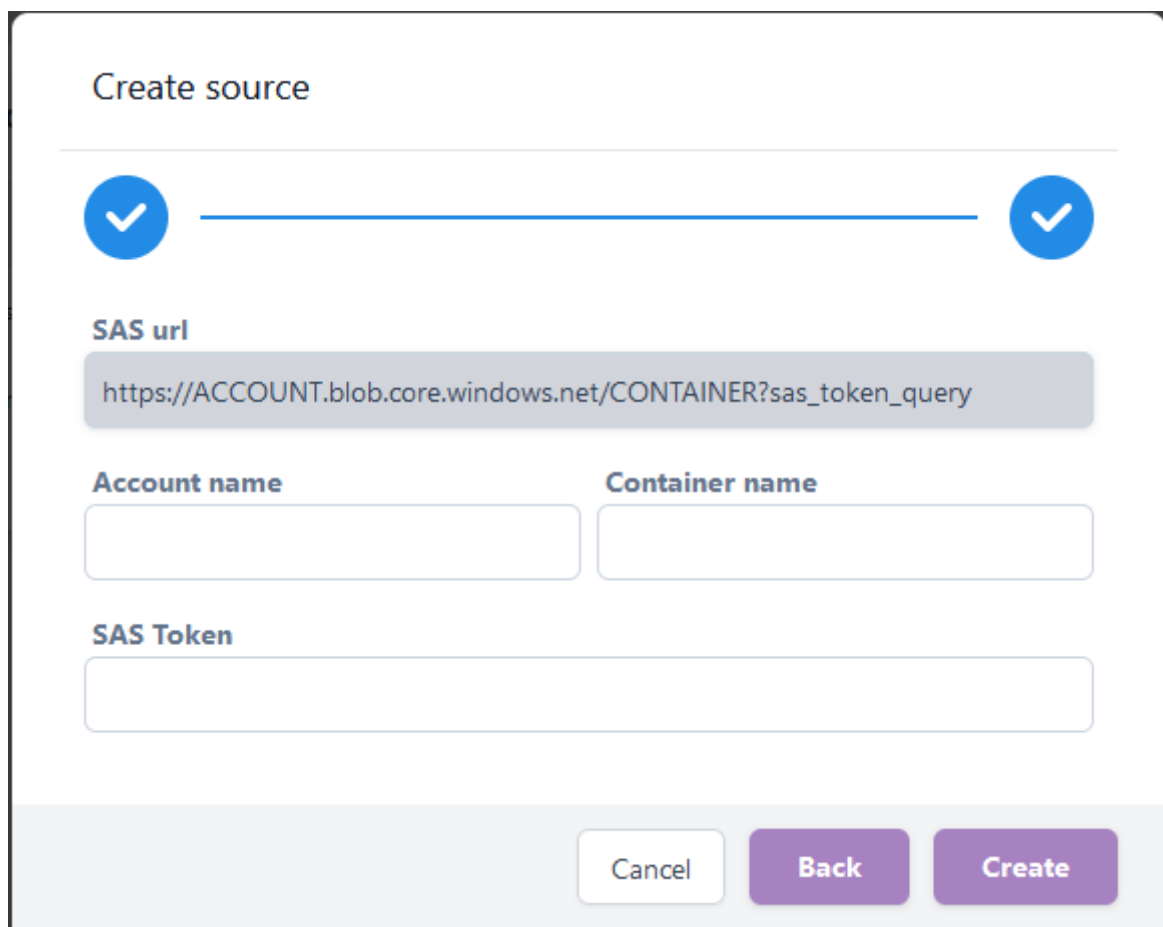


6.2.1.2 Requirements for connecting with Yres

Azure Blob Storage requires the following:

- Account name
- Container name
- SAS token

The created SAS url is shown above the form.



The screenshot shows a 'Create source' form. At the top, there is a title 'Create source'. Below it is a progress bar with two blue circles, each containing a white checkmark. Under the progress bar, the label 'SAS url' is followed by a text box containing the URL: `https://ACCOUNT.blob.core.windows.net/CONTAINER?sas_token_query`. Below this, there are two input fields: 'Account name' and 'Container name'. Below these is a single input field for 'SAS Token'. At the bottom right, there are three buttons: 'Cancel', 'Back', and 'Create'.

6.2.2 SharePoint

6.2.2.1 Setting up the data source to link with Yres

Use your Microsoft account in the Azure Portal with permissions to:

- Register application in Azure AD

- Create and read secrets in Azure Key Vault
- Assign permissions to user to read secrets in Azure Key Vault

Use Microsoft account with owner permissions in Microsoft Teams team / SharePoint list

Make sure to use Azure Key Vault in your resource group

Follow the steps below:

1. Register application in Azure AD

- Copy App ID, Tenant ID and paste into Notepad
- Create Client Secret under Certificates & Secrets in the new app
- Copy Secret value and paste into Notepad
- Create two secrets in App ID and Secret value into two new secrets in Azure Key Vault

2. Add App to in Sharepoint site / Teams team using https://bisolved.sharepoint.com/sites/IRIS/_layouts/15/appinv.aspx

3. Fill in the next fields:

Fill in 1 with App ID / Client ID

Fill in 2 with localhost.com¹²

Fill in 3 with:

```
<AppPermissionRequests AllowAppOnlyPolicy="true"> <AppPermissionRequest Scope="http://sharepoint/content/sitecollection/web" Right="FullControl" /> </AppPermissionRequests>
```

1. Click on Create and click on Trust

6.2.2.2 Requirements for connecting with Yres

SharePoint requires the following:

- Sharepoint site URL
- AD tenant name
- Postfix
- AD tenant ID
- Application ID / Service principal ID

12. <http://localhost.com>

- Application secret / Service principal key

Create source

✓

✓

Sharepoint site url

AD tenant name

Postfix

AD tenant id

Application id / service principal id

Application secret / service principal key

Cancel

Back

Create

6.3 File

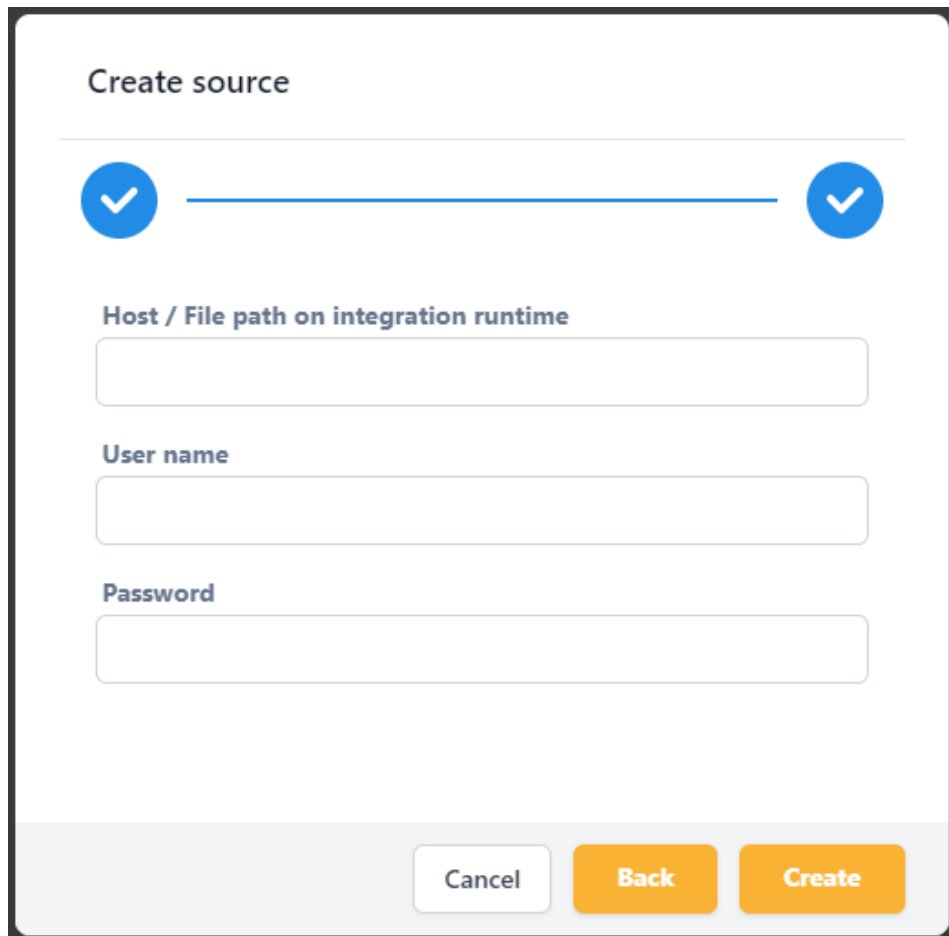
6.3.1 File Server

6.3.1.1 Setting up the data source to link with Yres

6.3.1.2 Requirements for connecting with Yres

File Server requires the following:

- Host / File path on integration runtime
- Username
- Password



The screenshot shows a 'Create source' dialog box. At the top, the title 'Create source' is displayed. Below the title, there is a progress bar with two blue circular markers, each containing a white checkmark. The first marker is on the left, and the second is on the right, connected by a horizontal line. Below the progress bar, there are three input fields, each with a label above it: 'Host / File path on integration runtime', 'User name', and 'Password'. Each input field is empty. At the bottom of the dialog box, there are three buttons: 'Cancel' (white with a grey border), 'Back' (orange), and 'Create' (orange).

6.4 Services & Apps

6.4.1 Monday

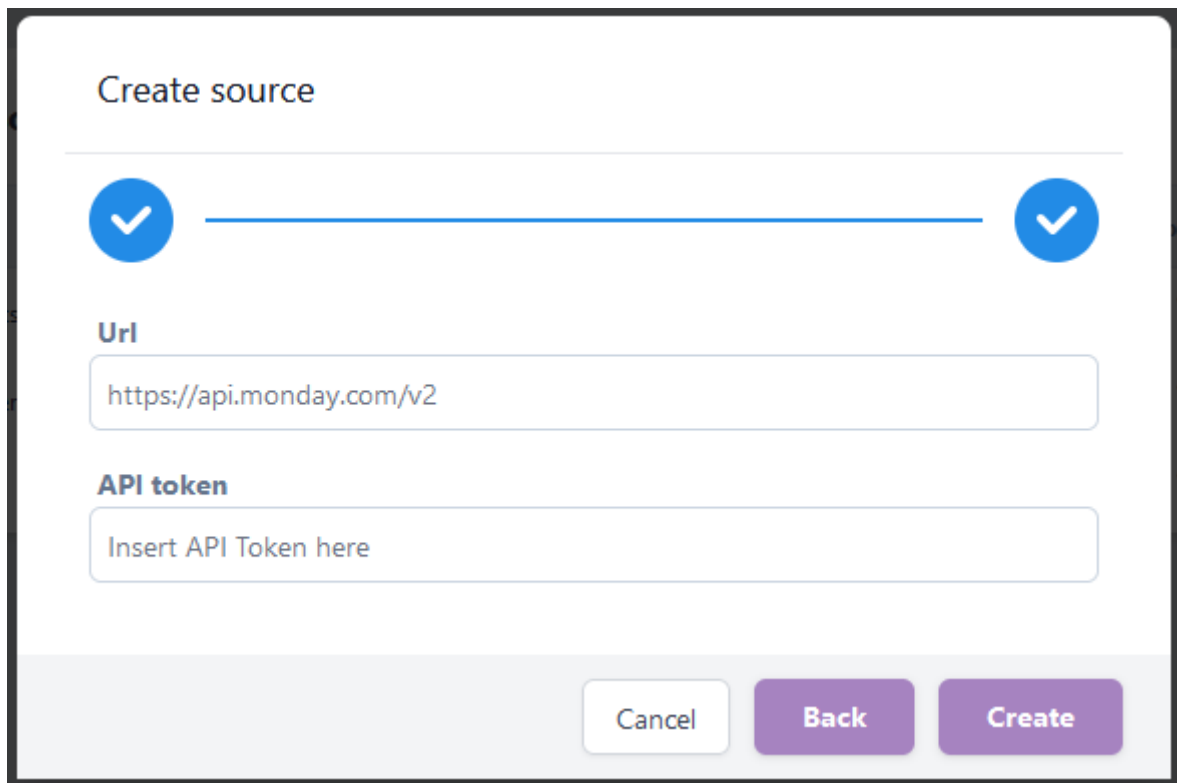
6.4.1.1 Setting up the data source to link with Yres

In order to link a Monday data source, an API token must be created. Please check [the Monday documentation](#)¹³ to learn more.

6.4.1.2 Requirements for connecting with Yres

Monday requires the following:

- Url
- API token



The screenshot shows a 'Create source' dialog box. At the top, the title 'Create source' is displayed. Below the title is a progress bar with two blue circular checkmarks at each end. The first section is labeled 'Url' and contains a text input field with the value 'https://api.monday.com/v2'. The second section is labeled 'API token' and contains a text input field with the placeholder text 'Insert API Token here'. At the bottom right of the dialog are three buttons: 'Cancel' (white with a grey border), 'Back' (purple), and 'Create' (purple).

6.4.2 AFAS

6.4.2.1 Setting up the data source to link with Yres

Documentation on how to utilize the app connector service can be found at [this link](#).¹⁴

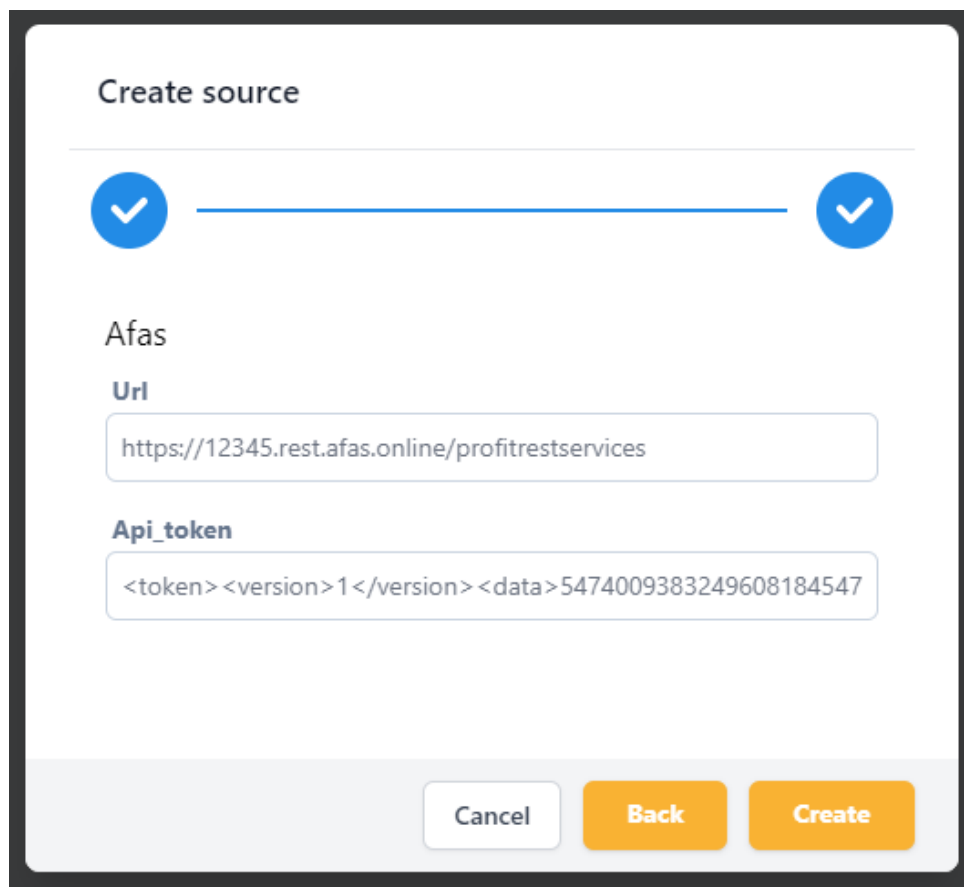
6.4.2.2 Requirements for connecting with Yres

AFAS requires the following:

- URL
- API token

13. <https://developer.monday.com/api-reference/docs/authentication>

14. <https://help.afas.nl/help/NL/SE/120718.htm>



Create source

Afas

Url

https://12345.rest.afas.online/profitrestservices

Api_token

<token> <version> 1 </version> <data> 5474009383249608184547

Cancel Back Create

6.4.3 Exact Online

6.4.3.1 Setting up the data source to link with Yres

6.4.3.2 Requirements for connecting with Yres

Exact Online requires the following:

- Exact Online apps for the dev and prod environment. Set the redirect URL to <https://backend.Yres-web.dev/exactonline/callback>¹⁵

15. <https://backend.yres-web.dev/exactonline/callback>

Create source

✓ — ✓ — ✓ — ✓ — 5

prod

Create a new Exact Online app for the prod environment via the exact app center. Set the redirect URI to:
<https://backend.iris-web.dev/exactonline/callback>

Client_id

Client_secret

Cancel Back Next Step

Create source

✓ — ✓ — ✓ — 4 — 5

dev

Authenticate ExactOnline App made for dev environment

Test

Login With Exact

Administration

Cancel Back Next Step

- tbd

6.4.4 Simplicate

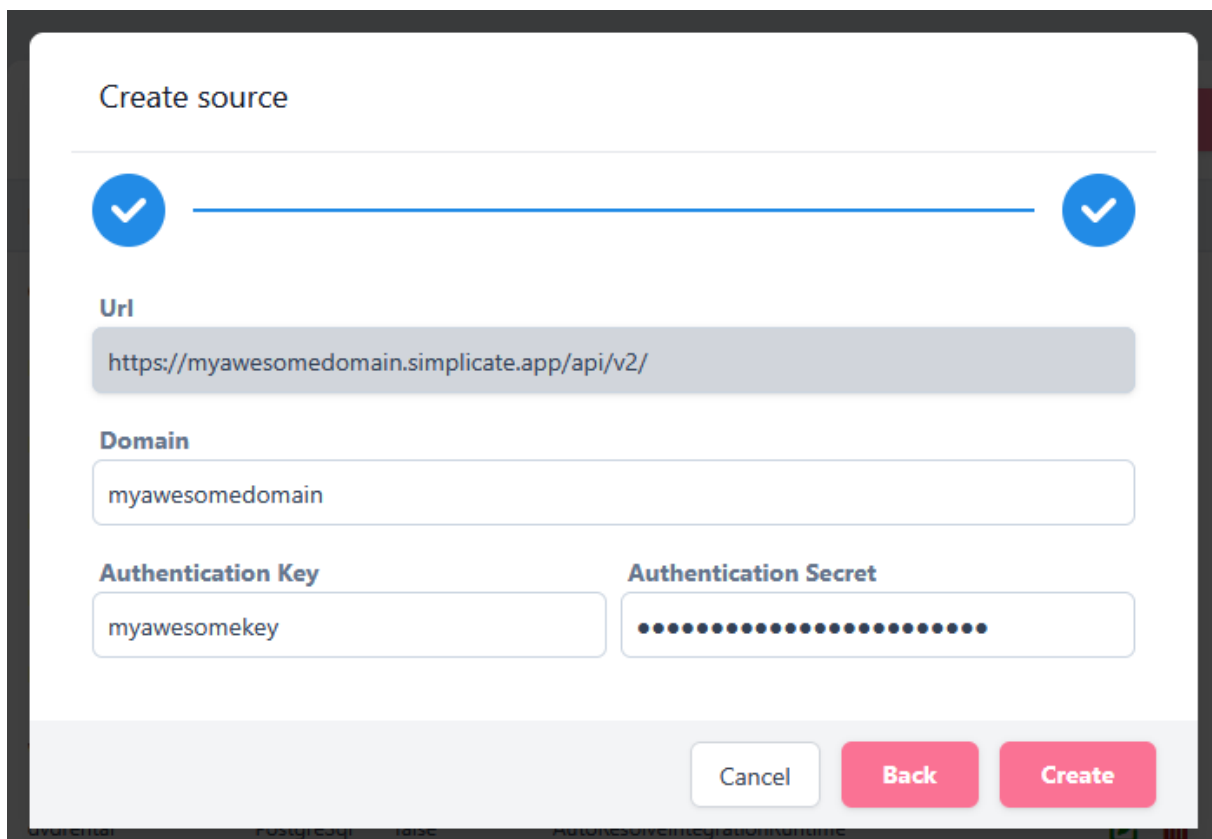
6.4.4.1 Setting up the data source to link with Yres

Connecting with your simplicate domain requires an API key and secret. Details on how to create these are found at the [Simplicate documentation](#).¹⁶

6.4.4.2 Requirements for connecting with Yres

Simplicate requires the following:

- Domain name
- Authentication key
- Authentication secret



The screenshot shows a 'Create source' form with a progress bar at the top, flanked by two blue checkmark icons. The form contains the following fields:

- Url:** A text field containing the value `https://myawesomedomain.simplicate.app/api/v2/`.
- Domain:** A text field containing the value `myawesomedomain`.
- Authentication Key:** A text field containing the value `myawesomekey`.
- Authentication Secret:** A text field filled with dots, representing a masked secret.

At the bottom right of the form, there are three buttons: 'Cancel' (white), 'Back' (pink), and 'Create' (pink).

6.4.5 Salesforce

6.4.5.1 Setting up the data source to link with Yres

Linking a Salesforce source requires a Consumer ID and secret, which can be found at the App Manager.

¹⁶. <https://support.simplicate.nl/nl/articles/6693108-api-key-aanmaken-en-verwijderen>

	19.09.2023, 08:...	Connected	
	13.01.2022, 10:05	Connected (Managed	
Channel Manager...	13.01.2022, 09:27	Lightning	View
ter Desktop is an...	13.01.2022, 09:27	Connected (Managed	Edit
Salesforce.com C...	13.01.2022, 09:27	Connected (Managed	Manage

6.4.5.2 Requirements for connecting with Yres

SalesForce requires the following:

- Environment URL
- Client ID
- Client secret

Create source

✓

✓

Environment url

https://tenant.my.salesforce.com

Client Id

id

Client Secret

.....

Cancel

Back

Create

6.4.6 SAC

6.4.6.1 Setting up the data source to link with Yres

A SAC source requires you to set up an OAuth client on the SAP Analytics Cloud. Documentation on how to do this can be found at the [SAP documentation](https://help.sap.com/docs/SAP_ANALYTICS_CLOUD/14cac91febef464dbb1efce20e3f1613/).¹⁷

17. https://help.sap.com/docs/SAP_ANALYTICS_CLOUD/14cac91febef464dbb1efce20e3f1613/

6.4.6.2 Requirements for connecting with Yres

SAC requires the following:

- URL
- Authentication URL
- Client ID
- Client secret

Create source

Url

https://tenant-name.eu10.hcs.cloud.sap/api/v1/dataexport/

Authentication url

https://tenant-name.authentication.eu10.hana.ondemand.com/oauth/token

Client Id

clientId

Client Secret

.....

Cancel Back Create

6.4.7 OneStream (preview)



This source type is currently a **preview**, and may be unstable.

6.4.7.1 Requirements for connecting with Yres

OneStream requires the following:

- URL

e4914540f5c04fd2b5b8be5675248be2.html

- Client ID
- Client secret
- Access token URL
- Application

Create source

Url

Client id

Client secret

Access token url

Application

Cancel Back Create

6.4.8 PowerBI

6.4.8.1 Setting up the data source to link with Yres

6.4.8.2 Requirements for connecting with Yres

PowerBI requires the following:

- Tenant ID
- Client ID

- Client secret

Create source

Tenant Id

Tenant id used to get a access token

Client Id

Client secret

Cancel Back Create

6.5 Generic protocol

6.5.1 OData

6.5.1.1 Setting up the data source to link with Yres

6.5.1.2 Requirements for connecting with Yres

OData requires the following:

- URL
- Authentication type
 - The basic authentication type requires a username and password
- HTTP headers

Create source

✓

✓

OData

Url

AuthenticationType

Basic

User_name

Password

Add HTTP header

Remove

Header

Value

Header

Value

Cancel

Back

Create

6.5.2 OData OAuth

6.5.2.1 Setting up the data source to link with Yres

6.5.2.2 Requirements for connecting with Yres

OData OAuth requires the following:

- URL
- Body URL
- Client ID
- Client secret
- Access token URL
- Scope
- Grant Type
 - The “Authorization Code” grant type requires a refresh token

Create source

✓

✓

Url

PaginationType
BodyUrl

Body url ?
@odata.nextlink

Client id

Client secret

Access token url

Scope

Grand Type
Authorization Code

Refresh token

Cancel

Back

Create

6.5.3 RestService

6.5.3.1 Setting up the data source to link with Yres

6.5.3.2 Requirements for connecting with Yres

RestService requires the following:

- API specification URL
- Base URL
- Pagination type
 - Body URL requires a body URL
 - Offset, Paging and OffsetPage require an offset object and a limit object
- Authentication type
 - Basic requires a username and password
 - OAuth2ClientCredential requires a token endpoint, client ID and secret, a scope and a resource
- Optional extra headers

✓

✓

Api Specification url (for example: OpenAPI 3.0)

Base URL

paginationType

Paging

Offset Object ?

Limit Object ?

AuthenticationType

OAuth2ClientCredential

Token endpoint

Client ID

Client Secret

Scope

Resource

Header 1

Value 1

Cancel

Back

Create

7 SQL Interaction

Yres databases can be managed for a SQL endpoint (like SSMS or Azure data studio). We advice to stick to the web frontend for most features but some features are only available for an SQL endpoint.

Additionally, when you no longer possess a subscription to the Yres frontend, using the SQL endpoint ensures availability for most features on existing sources. Adding new sources is not possible from the SQL endpoint.

7.1 StoredProcedures

7.1.1 Procedure: [Change].[spAddScriptedObject]

Purpose:

This stored procedure is designed to add a scripted object (such as a stored procedure, function, trigger, view, or table) to a change within the change management system. It ensures that the object and its dependencies are properly tracked and managed within the specified change.

Inputs:

- @ChangeID (INT): The ID of the change to which the object should be added.
- @ObjectType (NVARCHAR(5)): The type of the object (e.g., 'P' for stored procedure, 'FN' for scalar function).
- @ObjectName (NVARCHAR(4000)): The name of the object to be added.
- @Delete (BIT, default 0): Indicates whether the object should be marked for deletion.
- @AddDependencies (BIT, default 1): Indicates whether to automatically add dependencies of the object.
- @IgnoreYresObjects (BIT, default 1): Indicates whether to ignore Yres-generated objects.
- @appuser (NVARCHAR(512)): The username of the person executing the procedure.
- @OverwriteProcessID (UNIQUEIDENTIFIER, default NULL): Optional parameter to overwrite an existing process ID.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the addition of the scripted object and updates the change management tables accordingly.

7.1.2 Procedure: [Change].[spAddTable]

Purpose:

This stored procedure adds a table and its related metadata (such as columns, keys, and data types)

to an existing change within the change management system. It also manages dependencies and ensures that no active change conflicts exist.

Inputs:

- `@ChangeID` (INT): The ID of the change to which the table should be added.
- `@Source` (NVARCHAR(1024)): The source system name.
- `@Schema` (NVARCHAR(1024)): The schema name of the table.
- `@Table` (NVARCHAR(1024)): The table name.
- `@appUser` (NVARCHAR(MAX), default 'Unknown'): The username of the person executing the procedure.
- `@OverwriteProcessID` (UNIQUEIDENTIFIER, default NULL): Optional parameter to overwrite an existing process ID.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the addition of the table and updates the change management tables accordingly.

7.1.3 Procedure: `[Change].[spCopyTableContent]`

Purpose:

This stored procedure copies data from a source table to a target table, with the ability to map and cast columns as necessary. It is useful for transferring data between tables with similar but not identical structures.

Inputs:

- `@SourceTable` (NVARCHAR(256)): The name of the source table (including schema) from which data will be copied.
- `@TargetTable` (NVARCHAR(256)): The name of the target table (including schema) where data will be inserted.
- `@Execute` (BIT, default 0): Indicates whether to execute the copy operation or just generate the SQL command.

Outputs:

This procedure does not return any direct outputs. It either executes the data transfer or logs the generated SQL command.

7.1.4 Procedure: `[Change].[spImport]`

Purpose:

This stored procedure imports changes described in a JSON format into the change management

system. It handles the addition or updating of changes, projects, content, and dependencies based on the provided JSON data.

Inputs:

- `@JSON` (NVARCHAR(MAX)): A JSON string containing the change details to be imported.
- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person performing the import.

Outputs:

This procedure logs the results of the import process and updates the change management tables with the imported data.

7.1.5 Procedure: `[Change].[spInstall]`

Purpose:

This stored procedure is designed to import a change into the target system, allowing for execution, printing of SQL, or impact analysis. It processes the changes specified by `@ChangeID` and handles various object types within the change.

Inputs:

- `@ChangeID` (NVARCHAR(1024)): The ID of the change to be installed.
- `@Execute` (INT, default 2): Execution mode—1 for execution, 0 for printing SQL, 2 for impact analysis.
- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.
- `@CommitPartial` (BIT, default 0): Indicates whether to commit partial results in case of failure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the installation process and updates the relevant change management tables.

7.1.6 Procedure: `[Change].[spMaintain]`

Purpose:

This stored procedure is designed to maintain a change by adding, updating, or deleting records in the change management system. It ensures that changes are properly recorded and managed.

Inputs:

- `@Action` (NVARCHAR(256), default 'ADD'): The action to perform (ADD, UPDATE, DELETE).

- `@Name` (NVARCHAR(1024), default ""): The name of the change.
- `@Description` (NVARCHAR(MAX), default 'No description'): A description of the change.
- `@Project` (INT, default NULL): The project ID associated with the change.
- `@DueDate` (DATE, default NULL): The due date for the change.
- `@Status` (INT, default 1): The status of the change.
- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.
- `@Id` (INT, default -1): The ID of the change for update or delete actions.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the maintenance process and updates the relevant change management tables.

7.1.7 Procedure: `[Change].[spMaintainProject]`

Purpose:

This stored procedure is designed to maintain a project by adding, updating, or deleting records in the project management system. It ensures that projects are properly recorded and managed.

Inputs:

- `@Action` (NVARCHAR(256), default 'ADD'): The action to perform (ADD, UPDATE, DELETE).
- `@Name` (NVARCHAR(1024), default ""): The name of the project.
- `@Description` (NVARCHAR(MAX), default 'No description'): A description of the project.
- `@DueDate` (DATE, default NULL): The due date for the project.
- `@Status` (INT, default 1): The status of the project.
- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.
- `@Id` (INT, default -1): The ID of the project for update or delete actions.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the maintenance process and updates the relevant project management tables.

7.1.8 Procedure: `[Change].[spRelease]`

Purpose:

This stored procedure is designed to release a change in the change management system. It validates dependencies, updates the status of the change, and logs the release process.

Inputs:

- `@ChangeId` (INT): The ID of the change to be released.
- `@UseLapageVersion` (BIT, default 1): Indicates whether to use the lapage version of objects in the change.
- `@AppUser` (NVARCHAR(1024)): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the release process and updates the relevant change management tables.

7.1.9 Procedure: `[Config].[spAddFrameWorkColumns]`

Purpose:

This stored procedure adds framework columns to a stage table in the data warehouse. It includes the addition of ETL metadata such as ETL date, key hash, and row hash columns.

Inputs:

- `@Execute` (BIT, default 0): Indicates whether to execute the changes or just print the SQL script.
- `@SelectedTable` (NVARCHAR(MAX), default 'NoneSelected'): The table to which the framework columns should be added.
- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the addition of framework columns and updates the relevant data warehouse tables.

7.1.10 Procedure: `[Config].[spCompareMetadata]`

Purpose:

This stored procedure refreshes the stored metadata comparison by comparing the current metadata with the stored metadata in the data warehouse. It identifies mismatches such as missing columns, deleted columns, or type mismatches.

Inputs:

This procedure does not require any inputs.

Outputs:

This procedure updates the `Config.MetadataComparison` table with the results of the metadata comparison.

7.1.11 Procedure: [Config].

[spCreateExternalTablesFromDictionary]

Purpose:

This stored procedure creates external tables in the data warehouse based on the dictionary metadata. It is used for generating external table definitions for data integration purposes.

Inputs:

- @Execute (BIT, default 0): Indicates whether to execute the changes or just print the SQL script.
- @SelectedTable (NVARCHAR(MAX), default 'NoneSelected'): The table for which the external table should be created.
- @AppUser (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the creation of external tables and updates the relevant metadata tables.

7.1.12 Procedure: [Config].[spCreateTablesFromDictionary]

Purpose:

This stored procedure creates tables in the database based on the metadata from a dictionary view. It is used to automate the creation of tables for data integration and management purposes.

Inputs:

- @Execute (BIT, default 0): Indicates whether to execute the changes or just print the SQL script.
- @SelectedSource (NVARCHAR(MAX), default 'NoneSelected'): The source system for which tables should be created.
- @SelectedSchema (NVARCHAR(MAX), default 'NoneSelected'): The schema for which tables should be created.
- @SelectedTable (NVARCHAR(MAX), default 'NoneSelected'): The specific table to be created.
- @AppUser (NVARCHAR(MAX), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the creation of tables and updates the relevant metadata tables.

7.1.13 Procedure: [Config].[spDeleteTablesFromDB]

Purpose:

This stored procedure deletes tables from the database based on specified source, schema, and table names. It is used to remove tables that are no longer active or required.

Inputs:

- `@Execute` (BIT, default 0): Indicates whether to execute the deletion or just print the SQL script.
- `@SelectedSource` (NVARCHAR(MAX), default 'NoneSelected'): The source system for which tables should be deleted.
- `@SelectedSchema` (NVARCHAR(MAX), default 'NoneSelected'): The schema for which tables should be deleted.
- `@SelectedTable` (NVARCHAR(MAX), default 'NoneSelected'): The specific table to be deleted.
- `@AppUser` (NVARCHAR(MAX), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the deletion of tables and updates the relevant metadata tables.

7.1.14 Procedure: [Config].[spEnableColumnstore]

Purpose:

This stored procedure enables a ColumnStore index on existing tables in the database. It is used to improve the performance of analytical queries by converting tables to ColumnStore indexes.

Inputs:

- `@Execute` (BIT): Indicates whether to execute the changes or just print the SQL script.
- `@Schema` (NVARCHAR(50), default 'BOTH'): Specifies the schema (HIS or STAGE) where the ColumnStore index should be enabled.
- `@Table` (NVARCHAR(1024)): The name of the table for which the ColumnStore index should be enabled.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the enabling of ColumnStore indexes and updates the relevant metadata tables.

7.1.15 Procedure: [Config].[spEnableMemoryOptimization]

Purpose:

This stored procedure enables memory optimization in the current database by adding a MEMORY_OPTIMIZED_DATA filegroup and configuring necessary database settings. It is used to improve transaction performance by using in-memory OLTP features.

Inputs:

- @Execute (BIT, default 1): Indicates whether to execute the changes or just print the SQL script.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to enabling memory optimization and updates the relevant metadata tables.

7.1.16 Procedure: [Config].[spFillServices_SAC]

Purpose:

This stored procedure imports SAC (SAP Analytics Cloud) providers into the configuration services table. It is used to populate the services table with SAC providers' metadata.

Inputs:

- @input (ttServices_SAC READONLY): A table type containing SAC providers' metadata.
- @source (NVARCHAR(1024)): The source system from which the providers are imported.
- @Execute (BIT, default 1): Indicates whether to execute the insertion or just print the SQL script.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to the insertion of SAC providers and updates the relevant metadata tables.

7.1.17 Procedure: [Config].[spGetDependenciesSQL]

Purpose:

This stored procedure analyzes a SQL SELECT statement to determine its dependencies within the database. It is used to identify the tables, views, and other objects that a SQL query depends on.

Inputs:

- @SQL (NVARCHAR(MAX)): The SQL SELECT statement to be analyzed.

Outputs:

This procedure returns the dependencies found within the provided SQL statement. It also logs messages and errors related to the dependency analysis.

7.1.18 Procedure: `[Config].[spSetDatabaseParameter]`

Purpose:

This stored procedure sets a specified database parameter to a given value. It is used to modify database settings dynamically.

Inputs:

- `@Execute` (BIT, default 0): Indicates whether to execute the change or just print the SQL script.
- `@Parameter` (NVARCHAR(256)): The database parameter to be set.
- `@Setting` (NVARCHAR(256)): The value to which the database parameter should be set.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to setting the database parameter.

7.1.19 Procedure: `[Config].[spSetDatabaseServiceTier]`

Purpose:

This stored procedure changes the service tier of the database to a specified tier. It is used to scale the database's performance and resources by altering its service tier.

Inputs:

- `@toTier` (NVARCHAR(250), default 'Default'): The target service tier to which the database should be set.
- `@Execute` (BIT, default 1): Indicates whether to execute the change or just print the SQL script.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to changing the service tier and updates the relevant metadata tables.

7.1.20 Procedure: `[Config].[spUpdateRefreshToken]`

Purpose:

This stored procedure updates the refresh and access tokens in the configuration tokens table. It is used to maintain the validity of tokens used for API authentication.

Inputs:

- `@Source` (NVARCHAR(4000)): The source system for which the tokens are being updated.
- `@RefreshToken` (NVARCHAR(4000)): The new refresh token.
- `@AccessToken` (NVARCHAR(4000)): The new access token.
- `@ExpirationTime` (INT, default 10): The expiration time in minutes for the access token.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to updating the tokens and updates the relevant metadata tables.

7.1.21 Procedure: `[Config].[spUpdateTablesFromDictionary]`

Purpose:

This stored procedure updates existing tables in the database based on metadata from a dictionary view. It is used to modify table structures (add, remove, update columns) for data management purposes.

Inputs:

- `@Execute` (BIT, default 0): Indicates whether to execute the changes or just print the SQL script.
- `@SelectedSource` (NVARCHAR(MAX), default 'NoneSelected'): The source system for which tables should be updated.
- `@SelectedSchema` (NVARCHAR(MAX), default 'NoneSelected'): The schema for which tables should be updated.
- `@SelectedTable` (NVARCHAR(MAX), default 'NoneSelected'): The specific table to be updated.
- `@Methods` (NVARCHAR(MAX)): Specifies the actions to be taken (ADD, REMOVE, UPDATE).
- `@Stage` (BIT, default 0): Indicates whether to apply changes to the staging tables.
- `@HIS` (BIT, default 0): Indicates whether to apply changes to the history tables.
- `@AppUser` (NVARCHAR(MAX), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It logs messages and errors related to updating tables and updates the relevant metadata tables.

7.1.22 Procedure: [Config].[spWriteCrash]

Purpose:

This stored procedure logs crash information to the `Config.ProcessLog` table. It is used to capture detailed error information when a procedure fails.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process that encountered an error.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure that encountered the error.
- `@SpStep` (NVARCHAR(4000)): The step in the stored procedure where the error occurred.
- `@SpCall` (NVARCHAR(MAX)): The SQL call that was being executed when the error occurred.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@DbRequest` (NVARCHAR(MAX), optional): The database request that led to the error.
- `@ErrorLine` (INT): The line number in the SQL script where the error occurred.
- `@ErrorMessage` (NVARCHAR(4000)): The error message generated by SQL Server.
- `@ErrorNumber` (INT): The error number generated by SQL Server.
- `@ErrorProcedure` (NVARCHAR(4000)): The name of the stored procedure where the error occurred.
- `@ErrorSeverity` (INT): The severity of the error.
- `@ErrorState` (INT): The state of the error.

Outputs:

This procedure does not return any direct outputs. It logs detailed error information to the `Config.ProcessLog` table.

7.1.23 Procedure: [Config].[spWriteDump]

Purpose:

This stored procedure logs detailed dump information to the `Config.ProcessLog` table. It is used for logging system errors that require more detailed analysis.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process that encountered an error.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure that encountered the error.
- `@SpStep` (NVARCHAR(4000)): The step in the stored procedure where the error occurred.

- `@SpCall` (NVARCHAR(MAX)): The SQL call that was being executed when the error occurred.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@DbRequest` (NVARCHAR(MAX), optional): The database request that led to the error.
- `@ErrorLine` (INT): The line number in the SQL script where the error occurred.
- `@ErrorMessage` (NVARCHAR(4000)): The error message generated by SQL Server.
- `@ErrorNumber` (INT): The error number generated by SQL Server.
- `@ErrorProcedure` (NVARCHAR(4000)): The name of the stored procedure where the error occurred.
- `@ErrorSeverity` (INT): The severity of the error.
- `@ErrorState` (INT): The state of the error.

Outputs:

This procedure does not return any direct outputs. It logs detailed dump information to the `Config.ProcessLog` table.

7.1.24 Procedure: `[Config].[spWriteError]`

Purpose:

This stored procedure logs error information to the `Config.ProcessLog` table. It is used to capture detailed error information when a procedure encounters a problem.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process that encountered an error.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure that encountered the error.
- `@SpStep` (NVARCHAR(4000)): The step in the stored procedure where the error occurred.
- `@SpCall` (NVARCHAR(MAX)): The SQL call that was being executed when the error occurred.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@Message1` (NVARCHAR(4000)): The primary error message.
- `@Message2` (NVARCHAR(4000), optional): Additional error message information.
- `@Message3` (NVARCHAR(4000), optional): Additional error message information.
- `@Message4` (NVARCHAR(4000), optional): Additional error message information.
- `@DbRequest` (NVARCHAR(MAX), optional): The database request that led to the error.
- `@ErrorLine` (INT, optional): The line number in the SQL script where the error occurred.
- `@ErrorMessage` (NVARCHAR(4000), optional): The error message generated by SQL Server.

- `@ErrorNumber` (INT, optional): The error number generated by SQL Server.
- `@ErrorProcedure` (NVARCHAR(4000), optional): The name of the stored procedure where the error occurred.
- `@ErrorSeverity` (INT, optional): The severity of the error.
- `@ErrorState` (INT, optional): The state of the error.

Outputs:

This procedure does not return any direct outputs. It logs error information to the `Config.ProcessLog` table.

7.1.25 Procedure: `[Config].[spWriteFullLog]`

Purpose:

This stored procedure writes full log entries to the `Config.ProcessLog` table. It is used for logging various levels of information, including process steps, messages, and errors.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process being logged.
- `@CallSource` (NVARCHAR(4000)): The source system or hostname making the call.
- `@CallSourceType` (NVARCHAR(4000)): The type of application making the call.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure being logged.
- `@SpCall` (NVARCHAR(MAX)): The SQL call being executed.
- `@SpStep` (NVARCHAR(4000)): The step in the process being logged.
- `@ReturnCode` (INT): The return code of the stored procedure.
- `@Message1` (NVARCHAR(4000)): The primary message to be logged.
- `@Message2` (NVARCHAR(4000)): Additional message information.
- `@Message3` (NVARCHAR(4000)): Additional message information.
- `@Message4` (NVARCHAR(4000)): Additional message information.
- `@DbUser` (NVARCHAR(4000)): The database user executing the procedure.
- `@DbServer` (NVARCHAR(4000)): The database server name.
- `@DbName` (NVARCHAR(4000)): The database name.
- `@DbRequest` (NVARCHAR(MAX)): The database request or query being logged.
- `@ErrorLine` (INT): The line number in the SQL script where the error occurred.
- `@ErrorMessage` (NVARCHAR(4000)): The error message generated by SQL Server.

- `@ErrorNumber` (INT): The error number generated by SQL Server.
- `@ErrorProcedure` (NVARCHAR(4000)): The name of the stored procedure where the error occurred.
- `@ErrorSeverity` (INT): The severity of the error.
- `@ErrorState` (INT): The state of the error.

Outputs:

This procedure does not return any direct outputs. It logs detailed process information to the `Config.ProcessLog` table.

7.1.26 Procedure: `[Config].[spWriteLog]`

Purpose:

This stored procedure writes log entries to the `Config.ProcessLog` table. It is used for logging messages, warnings, and errors during the execution of other stored procedures.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process being logged.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure being logged.
- `@SpStep` (NVARCHAR(4000)): The step in the process being logged.
- `@SpCall` (NVARCHAR(MAX)): The SQL call being executed.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@ReturnCode` (INT): The return code of the stored procedure.
- `@Message1` (NVARCHAR(4000)): The primary message to be logged.
- `@Message2` (NVARCHAR(4000), optional): Additional message information.
- `@Message3` (NVARCHAR(4000), optional): Additional message information.
- `@Message4` (NVARCHAR(4000), optional): Additional message information.
- `@DbRequest` (NVARCHAR(MAX), optional): The database request or query being logged.
- `@ErrorLine` (INT, optional): The line number in the SQL script where the error occurred.
- `@ErrorMessage` (NVARCHAR(4000), optional): The error message generated by SQL Server.
- `@ErrorNumber` (INT, optional): The error number generated by SQL Server.
- `@ErrorProcedure` (NVARCHAR(4000), optional): The name of the stored procedure where the error occurred.
- `@ErrorSeverity` (INT, optional): The severity of the error.
- `@ErrorState` (INT, optional): The state of the error.

Outputs:

This procedure does not return any direct outputs. It logs messages, warnings, and errors to the `Config.ProcessLog` table.

7.1.27 Procedure: `[Config].[spWriteMessage]`**Purpose:**

This stored procedure logs informational messages to the `Config.ProcessLog` table. It is used to record process steps and actions without errors.

Inputs:

- `@ProcessID` (UNIQUEIDENTIFIER): The unique identifier for the process being logged.
- `@SpName` (NVARCHAR(4000)): The name of the stored procedure being logged.
- `@SpStep` (NVARCHAR(4000)): The step in the process being logged.
- `@SpCall` (NVARCHAR(MAX)): The SQL call being executed.
- `@AppUser` (NVARCHAR(4000)): The username of the person executing the procedure.
- `@Message1` (NVARCHAR(4000)): The primary message to be logged.

Outputs:

This procedure does not return any direct outputs. It logs informational messages to the `Config.ProcessLog` table.

7.1.28 Procedure: `[Config].[spWriteMessage]`**Purpose:**

This stored procedure writes a log message to the `Config.ProcessLog` table, including all relevant fields for tracking and debugging purposes.

Inputs:

- `@processID` (uniqueidentifier): The unique identifier for the process being logged.
- `@spName` (nvarchar(4000)): The name of the stored procedure generating the log.
- `@spStep` (nvarchar(4000)): The step within the stored procedure being executed.
- `@spCall` (nvarchar(max)): The SQL call being executed.
- `@appUser` (nvarchar(4000)): The username of the person executing the procedure.
- `@message1` (nvarchar(4000)): The primary log message.
- `@message2` (nvarchar(4000), optional): Additional log message information.

- `@message3` (nvarchar(4000), optional): Additional log message information.
- `@message4` (nvarchar(4000), optional): Additional log message information.
- `@dbRequest` (nvarchar(max), optional): The database request or query being logged.

Outputs:

This procedure does not return any direct outputs. It logs detailed information to the `Config.ProcessLog` table.

7.1.29 Procedure: `[Config].[spWriteWarning]`

Purpose:

This stored procedure writes a warning log to the `Config.ProcessLog` table, capturing all relevant fields for monitoring and debugging purposes.

Inputs:

- `@processID` (uniqueidentifier): The unique identifier for the process being logged.
- `@spName` (nvarchar(4000)): The name of the stored procedure generating the log.
- `@spStep` (nvarchar(4000)): The step within the stored procedure where the warning occurred.
- `@spCall` (nvarchar(max)): The SQL call being executed.
- `@appUser` (nvarchar(4000)): The username of the person executing the procedure.
- `@message1` (nvarchar(4000)): The primary warning message.
- `@message2` (nvarchar(4000), optional): Additional warning message information.
- `@message3` (nvarchar(4000), optional): Additional warning message information.
- `@message4` (nvarchar(4000), optional): Additional warning message information.
- `@dbRequest` (nvarchar(max), optional): The database request or query being logged.

Outputs:

This procedure does not return any direct outputs. It logs detailed warning information to the `Config.ProcessLog` table.

7.1.30 Procedure: `[dbo].[spAdaptiveIndexDefrag_CurrentExecStats]`

Purpose:

This stored procedure provides a report on the current execution status of index defragmentation tasks. It monitors what has been completed and what is still pending.

Inputs:

- `@dbname` (NVARCHAR(255), optional): Specifies the database to monitor. If not provided, all databases are monitored.

Outputs:

This procedure returns a set of results that detail the progress of index and statistics defragmentation, showing both completed and pending tasks.

7.1.31 Procedure: `[dbo].[spAdaptiveIndexDefrag_Exceptions]`

Purpose:

This stored procedure manages exceptions for index defragmentation, allowing the exclusion of specific databases, days, tables, or indexes from the defragmentation process.

Inputs:

- `@exceptionMask_DB` (NVARCHAR(255), optional): The database to be excluded from defragmentation.
- `@exceptionMask_days` (NVARCHAR(27), optional): Days of the week to exclude from defragmentation, provided in short form (e.g., 'Mon,Wed').
- `@exceptionMask_tables` (NVARCHAR(500), optional): Tables to be excluded from defragmentation.
- `@exceptionMask_indexes` (NVARCHAR(500), optional): Indexes to be excluded from defragmentation.

Outputs:

This procedure does not return any direct outputs. It logs actions and results related to the management of defragmentation exceptions.

7.1.32 Procedure: `[dbo].[spAdaptiveIndexDefrag_PurgeLogs]`

Purpose:

This stored procedure purges old logs related to index defragmentation to prevent indefinite growth of the log tables.

Inputs:

- `@daystokeep` (smallint, default 90): The number of days of logs to retain. Logs older than this will be deleted.

Outputs:

This procedure does not return any direct outputs. It deletes old log entries from the maintenance tables.

7.1.33 Procedure: [dbo].[spCopyDB]

Purpose:

This stored procedure creates a copy of an existing database, with options to drop the target database if it exists and to specify the service tier of the new database.

Inputs:

- @sourceDB (NVARCHAR(1024)): The name of the source database to be copied.
- @targetDB (NVARCHAR(1024)): The name of the target database to be created.
- @targetTier (NVARCHAR(256), default 'GP_Gen5_2'): The service tier for the new database.
- @dropIfExists (BIT, default 0): Indicates whether to drop the target database if it already exists.

Outputs:

This procedure does not return any direct outputs. It logs messages indicating success or failure in creating the database copy.

7.1.34 Procedure: [dbo].[spJsonToTable]

Purpose:

This stored procedure converts JSON data into a relational table structure in SQL Server, allowing for easier data manipulation and querying.

Inputs:

- @Collection (NVARCHAR(1024), optional): Specifies the JSON collection to be processed.
- @json (NVARCHAR(max), default '{}'): The JSON data to be converted.
- @targetSchema (NVARCHAR(1024), optional): The schema where the table will be created.
- @targetTable (NVARCHAR(1024), optional): The name of the target table to be created.

Outputs:

This procedure dynamically generates and executes a SQL script to convert the provided JSON data into a table. The resulting table structure is based on the JSON keys and values.

7.1.35 Procedure: [dbo].[spLongPrint]

Purpose:

This stored procedure prints long strings of text in chunks, allowing for better handling of large messages that exceed the default length limits in SQL Server.

Inputs:

- `@String` (NVARCHAR(MAX)): The long string of text to be printed.

Outputs:

This procedure does not return any direct outputs. It prints the provided string in manageable chunks to the console or log.

7.1.36 Procedure: `[dbo].[spMSForEachTable]`

Purpose:

This stored procedure executes a specified command against each table in the database, allowing for batch operations across multiple tables.

Inputs:

- `@command1` (NVARCHAR(2000)): The primary command to be executed against each table.
- `@replacechar` (NCHAR(1), default '?'): The character in `@command1` that will be replaced with the table name.
- `@command2` (NVARCHAR(2000), optional): An additional command to be executed.
- `@command3` (NVARCHAR(2000), optional): Another additional command to be executed.
- `@whereand` (NVARCHAR(2000), optional): Additional conditions to apply to the table selection.
- `@precommand` (NVARCHAR(2000), optional): A command to be executed before the main commands.
- `@postcommand` (NVARCHAR(2000), optional): A command to be executed after the main commands.

Outputs:

This procedure does not return any direct outputs. It executes the specified commands against all relevant tables in the database.

7.1.37 Procedure: `[dbo].[spMSForEachWorker]`

Purpose:

This stored procedure is a worker procedure that assists `[dbo].[spMSForEachTable]` in executing commands against each table in the database.

Inputs:

- `@command1` (NVARCHAR(2000)): The primary command to be executed.
- `@replacechar` (NCHAR(1), default '?'): The character in `@command1` that will be replaced.

- `@command2` (NVARCHAR(2000), optional): An additional command to be executed.
- `@command3` (NVARCHAR(2000), optional): Another additional command to be executed.
- `@worker_type` (INT, default 1): Indicates the type of worker process.

Outputs:

This procedure does not return any direct outputs. It assists in executing commands against tables as part of the batch process initiated by `[dbo].[spMSForEachTable]`.

7.1.38 Procedure: `[dbo].[spRunSQL]`

Purpose:

This stored procedure runs any SQL statement provided as input, allowing for dynamic execution of SQL commands.

Inputs:

- `@SQL` (NVARCHAR(max)): The SQL statement to be executed.

Outputs:

This procedure does not return any direct outputs. It executes the provided SQL statement.

7.1.39 Procedure: `[LoadManagement].[spCheckKeyAndRowHash]`

Purpose:

This stored procedure checks the integrity of key and row hashes for specified tables, verifying the consistency of data between staging and history tables.

Inputs:

- `@EXECUTE` (BIT, default 0): Indicates whether to execute the checks or just generate the SQL script.
- `@Source` (NVARCHAR(MAX)): The source system of the tables to be checked.
- `@Schema` (NVARCHAR(MAX)): The schema of the tables to be

checked.

- `@Table` (NVARCHAR(MAX)): The name of the table to be checked.
- `@sampleSize` (NVARCHAR(MAX), default 10): The sample size of rows to check.

Outputs:

This procedure returns a report on the integrity of key and row hashes, indicating whether the data is consistent.

7.1.40 Procedure: `[LoadManagement].[spFillDictionary_AFAS]`

Purpose:

This stored procedure imports metadata from an AFAS data source into a SQL Server dictionary table, mapping columns and data types.

Inputs:

- `@input` (ttDictionary_AFAS READONLY): The table type containing AFAS dictionary data.
- `@source` (NVARCHAR(1024)): The source name of the data.
- `@EXECUTE` (BIT, default 1): Indicates whether to execute the import or just generate the SQL script.

Outputs:

This procedure imports metadata into the `LoadManagement.Dictionary` table or generates an SQL script for review.

7.1.41 Procedure: `[LoadManagement].[spFillDictionary_oDATA]`

Purpose:

This stored procedure imports metadata from an oData data source into a SQL Server dictionary table, mapping columns and data types.

Inputs:

- `@input` (ttDictionary_oDataADF READONLY): The table type containing oData dictionary data.
- `@source` (NVARCHAR(1024)): The source name of the data.
- `@schema` (NVARCHAR(1024), default 'API'): The schema for the data.
- `@EXECUTE` (BIT, default 1): Indicates whether to execute the import or just generate the SQL script.

Outputs:

This procedure imports metadata into the `LoadManagement.Dictionary` table or generates an SQL script for review.

7.1.42 Procedure: `[LoadManagement].[spFillTable_Json]`

Purpose:

This stored procedure populates a table with data from a JSON file, converting the JSON structure into a tabular format.

Inputs:

- `@Collection` (NVARCHAR(1024), optional): Specifies the JSON collection to be processed.
- `@jsonTable` (ttTable_JsonAdf READONLY): The table type containing JSON data.
- `@targetSchema` (NVARCHAR(1024), optional): The schema where the target table will be created.
- `@targetTable` (NVARCHAR(1024), optional): The name of the target table to be created.

Outputs:

This procedure dynamically creates a table based on the JSON structure and populates it with the data.

7.1.43 Procedure: `[LoadManagement].[spFillTable_Monday]`

Purpose:

This stored procedure processes and maps data from <http://Monday.com>, transposing columns as necessary and inserting the data into a specified target table.

Inputs:

- `@Table` (ttTable_Monday READONLY): The table type containing <http://Monday.com> data.

Outputs:

This procedure transposes the <http://Monday.com> data and inserts it into the specified target table.

7.1.44 Procedure: `[LoadManagement].[spFindTablesBehindSQL]`

Purpose:

This stored procedure identifies all tables referenced by a given SQL query, useful for dependency analysis.

Inputs:

- `@SQL` (NVARCHAR(MAX)): The SQL query to analyze.

Outputs:

This procedure returns a list of tables that are referenced by the given SQL query.

7.1.45 Procedure: `[LoadManagement].[spGenerateTypeMapping]`

Purpose:

This stored procedure generates type mappings for source systems, adding them to the `LoadManagement.TypeMapping` table if they do not already exist.

Inputs:

- `@AppUser` (NVARCHAR(4000), default 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure does not return any direct outputs. It updates the `LoadManagement.TypeMapping` table with any new type mappings found.

7.1.46 Procedure: `[LoadManagement].[spGetColumnMapping]`

Purpose:

This stored procedure generates a JSON object representing the column mappings between a source and target system, useful for ETL processes.

Inputs:

- `@tablename` (VARCHAR(256), optional): The name of the source table.
- `@schemaname` (VARCHAR(256), optional): The schema name of the source table.
- `@source` (VARCHAR(256), optional): The name of the source system.
- `@Target` (NVARCHAR(MAX), optional): The target system for the mapping.
- `@Pipeline_ID` (NVARCHAR(MAX), default 1): The ID of the ETL pipeline.

Outputs:

This procedure returns a JSON object representing the field mappings for the specified source and target systems.

7.1.47 Procedure: `[LoadManagement].[spGetRowCount]`

Purpose:

This stored procedure returns the row count of a specified table, with options to get an exact count or a simple check for non-empty status.

Inputs:

- `@table` (NVARCHAR(1024)): The name of the table to count rows in.
- `@Exact` (BIT): Indicates whether to get an exact row count (1) or just check for non-empty status (0).
- `@Active` (NCHAR(1), default 0): Indicates whether to count only records where `isCurrent = 1`.

Outputs:

This procedure returns the row count of the specified table.

7.1.48 Procedure: [LoadManagement].[spHIS_InsertAndUpdate]

Purpose:

This stored procedure inserts new records and updates existing records in a History (HIS) table, managing delta loads and maintaining data integrity.

Inputs:

- @Target (NVARCHAR(MAX), optional): The target table for the operation.
- @Pipeline_ID (NVARCHAR(MAX), default 1): The ID of the ETL pipeline.
- @Execute (BIT, default 0): Indicates whether to execute the operation or just generate the SQL script.
- @DeltaColumn (NVARCHAR(MAX), optional): The column used to identify delta changes.
- @TableLoadType (NVARCHAR(MAX), optional): The type of table load operation (e.g., DELTA , OVERWRITE , RELOAD).

Outputs:

This procedure updates the target HIS table with new or updated records and logs the operation's status.

7.1.49 Procedure: [LoadManagement].[spHIS_TruncateTable]

Purpose:

This stored procedure truncates or deletes all records from a history table (HIS) in a SQL Server database. The specific action depends on whether the table is memory optimized.

Inputs:

- @Target (NVARCHAR(MAX)): The name of the target table to truncate.
- @EXECUTE (BIT, default = 1): Indicates whether to execute the truncation (1) or just print the SQL script (0).

Outputs:

This procedure either truncates or deletes records from the specified history table or prints the corresponding SQL script.

7.1.50 Procedure: [LoadManagement].[spLoadDWH]

Purpose:

This stored procedure triggers the data warehouse loading process by invoking specific procedures for inserting and updating data and updating the ETL end date.

Inputs:

- @Target (NVARCHAR(MAX), optional): The target table for the data load.
- @Pipeline_ID (NVARCHAR(MAX), default = '1'): The ID of the ETL pipeline.
- @Execute (BIT, default = 0): Indicates whether to execute the operations or just generate the SQL script.
- @DeltaColumn (NVARCHAR(MAX), optional): The column used for delta changes.
- @TableLoadType (NVARCHAR(MAX), optional): The type of table load operation (e.g., DELTA, FULL).

Outputs:

This procedure calls the [LoadManagement].[spHIS_InsertAndUpdate] and [LoadManagement].[spUpdateETL_EndDate] procedures, executing or printing their results based on the input parameters.

7.1.51 Procedure: [LoadManagement].[spMaintainFiles]

Purpose:

This stored procedure manages files for import into the data warehouse, supporting actions such as adding, updating, deleting, or deactivating file entries.

Inputs:

- @action (NVARCHAR(256), default = 'ADD'): The action to perform (ADD, UPDATE, DELETE, DEACTIVATE).
- @source (NVARCHAR(256)): The source of the file data.
- @schema (NVARCHAR(256)): The schema name of the file data.
- @table (NVARCHAR(256)): The table name associated with the file data.
- Additional parameters such as @DataPlatform, @loadType, @deltaColumn, etc., define the specific configuration of the file import process.

Outputs:

This procedure modifies entries in the LoadManagement.UsedTables and LoadManagement.UsedColumns tables based on the specified action.

7.1.52 Procedure: [LoadManagement].

[spMaintainFilesInDictionary]

Purpose:

This stored procedure manages metadata for files in the SQL Server dictionary, including actions to add, update, or delete entries.

Inputs:

- @EXECUTE (BIT, default = 0): Indicates whether to execute the SQL script or just print it.
- @action (NVARCHAR(256)): The action to perform (ADD, UPDATE, DELETE).
- @source (NVARCHAR(4000), default = 'FILE'): The source of the data.
- @category (NVARCHAR(4000)): The category of the file data.
- @table (NVARCHAR(4000)): The table name associated with the file data.

Outputs:

This procedure adds, updates, or deletes entries in the LoadManagement.Dictionary and LoadManagement.UsedColumns tables based on the specified action.

7.1.53 Procedure: [LoadManagement].[spMaintainPersistView]

Purpose:

This stored procedure manages the persistence of views in the data warehouse, including actions to add, update, or delete view persistence settings.

Inputs:

- @action (NVARCHAR(256), default = 'ADD'): The action to perform (ADD, UPDATE, DELETE).
- @SourceDeltaObjectDB (NVARCHAR(128), optional): The database of the source delta object.
- @SourceDeltaObjectSchemaName (NVARCHAR(128), optional): The schema name of the source delta object.
- @SourceDeltaObjectName (NVARCHAR(128), optional): The name of the source delta object.
- @SourceDeltaObjectType (NVARCHAR(1), optional): The type of the source delta object (V = View, T = Table).
- Additional parameters define the view and table configurations for persistence.

Outputs:

This procedure modifies entries in the LoadManagement.ViewPersistence table based on the specified action.

7.1.54 Procedure: `[LoadManagement].[spMaintainRestService]`

Purpose:

This stored procedure manages REST services for import into the data warehouse, supporting actions such as adding, updating, deleting, or deactivating service entries.

Inputs:

- `@action` (NVARCHAR(256), default = 'ADD'): The action to perform (ADD, UPDATE, DELETE, DEACTIVATE).
- `@source` (NVARCHAR(256)): The source of the REST service.
- `@service` (NVARCHAR(256)): The name of the REST service.
- `@endpoint` (NVARCHAR(256)): The endpoint of the REST service.
- Additional parameters such as `@DataPlatform`, `@loadType`, `@keepStage`, etc., define the specific configuration of the REST service import process.

Outputs:

This procedure modifies entries in the `LoadManagement.UsedTables` and `LoadManagement.UsedColumns` tables based on the specified action.

7.1.55 Procedure: `[LoadManagement].[spMaintainSource]`

Purpose:

This stored procedure manages data sources in the data warehouse, supporting actions such as adding, updating, deleting, or deactivating source entries.

Inputs:

- `@action` (NVARCHAR(256), default = 'ADD'): The action to perform (ADD, UPDATE, DELETE, DEACTIVATE).
- `@source` (NVARCHAR(256)): The name of the data source.
- `@sourceType` (NVARCHAR(256), optional): The type of the data source (e.g., database, API).
- `@AppUser` (NVARCHAR(4000), default = 'Unknown'): The username of the person executing the procedure.

Outputs:

This procedure modifies entries in the `LoadManagement.SourceSystems` table based on the specified action.

7.1.56 Procedure: `[LoadManagement].[spMaintainTable]`

Purpose:

This stored procedure manages tables in the data warehouse, supporting actions such as adding, updating, or deleting table entries.

Inputs:

- `@action` (NVARCHAR(256), default = 'ADD'): The action to perform (ADD, UPDATE, DELETE).
- `@source` (NVARCHAR(256)): The source of the table data.
- `@schema` (NVARCHAR(256)): The schema name of the table data.
- `@table` (NVARCHAR(256)): The table name associated with the data.
- Additional parameters such as `@DataPlatform`, `@fieldList`, `@loadType`, etc., define the specific configuration of the table management process.

Outputs:

This procedure modifies entries in the `LoadManagement.UsedTables`, `LoadManagement.UsedColumns`, and other related tables based on the specified action.

7.1.57 Procedure: `[LoadManagement].[spMaintainTrigger]`

Purpose:

The specific purpose of this stored procedure is not provided, but typically a `spMaintainTrigger` procedure would be used to manage database triggers (adding, updating, or deleting them) based on specific requirements.

Inputs:

The inputs for this procedure are not provided in the script.

Outputs:

The outputs for this procedure are not provided in the script.

7.1.58 Procedure: `[LoadManagement].[spMaintainTrigger]`

Purpose:

To manage triggers for tables within the `LoadManagement` schema. This includes adding new triggers or deleting existing ones based on the provided parameters.

Inputs:

- `@action`: The action to perform (`ADD` or `DELETE`).
- `@source`: The source name (can specify "all" for all sources).

- `@schema` : The schema name (can specify "all" for all schemas).
- `@Table` : The table name (can specify "all" for all tables).
- `@Trigger` : The trigger name (used with `DELETE` action; "ALL" can be used to delete all triggers for a specific table).
- `@AppUser` : The user performing the action.

Outputs:

- No explicit outputs, but the procedure logs actions and errors using `[Config].[spWriteMessage]`, `[Config].[spWriteError]`, and `[Config].[spWriteWarning]`.

7.1.59 Procedure: `[LoadManagement].[spMaterializeViews]`**Purpose:**

To load or materialize views into tables according to the trigger mapping defined in `[LoadManagement].[TriggerMapping]`. The procedure automates the data loading process based on the triggers defined.

Inputs:

- `@WorkFlow` : Identifier for the workflow.
- `@PipelineName` : Name of the pipeline.
- `@Trigger` : Name of the trigger that initiates the load.
- `@ISource` : Source name (can be 'AUTO' to auto-detect or 'ALL').
- `@ISchema` : Schema name (can be 'ALL').
- `@IView` : View name (can be 'ALL').
- `@EXECUTE` : Flag (1 or 0) indicating whether to execute the load or just print the SQL commands.

Outputs:

- No direct outputs. The procedure logs the status and progress of the load using `[Monitoring].[spWriteLoadStatus]`.

7.1.60 Procedure: [LoadManagement].[spMaterializeViewToTable]

Purpose:

To materialize a specific view into a corresponding table. It handles the transfer of data from the view to the table and supports both full and delta loads.

Inputs:

- @SourceSchemaName : The schema of the source view.
- @SourceViewName : The name of the source view.
- @execute : Flag (1 or 0) indicating whether to execute the materialization.
- @rows : Output parameter to capture the number of rows affected.

Outputs:

- Outputs the number of rows affected by the materialization process.
-

7.1.61 Procedure: [LoadManagement].[spPrepareCopy]

Purpose:

To prepare for the copying or loading of data by truncating staging tables, looking up mappings, and initiating the load process.

Inputs:

- Various parameters for pipeline identification, process steps, and target tables such as @PipelineID, @Process, @Step, @Status, @Rows, @WorkflowID, @PipelineName, @Started_by, @Target, @Source_system, @Table, @Schema, @LoadType, @LapageRecord, and @ETL_Date.

Outputs:

- No direct outputs. Logs load status and any errors encountered during the preparation process using [Monitoring].[spWriteLoadStatus].
-

7.1.62 Procedure: [LoadManagement].[spRegenerateSurrogateKeys]

Purpose:

To regenerate surrogate keys for a specified source table. This procedure is typically used when there

are changes in the source data that require re-generating the unique identifiers (surrogate keys) for historical tables.

Inputs:

- `@Source` : The name of the data source.
- `@SourceSchema` : The schema of the source table.
- `@SourceTable` : The name of the source table.
- `@Execute` : Flag (1 or 0) indicating whether to execute the regeneration.
- `@AppUser` : The user initiating the procedure.

Outputs:

- No direct outputs. The procedure generates and logs messages related to the regeneration process.

7.1.63 Procedure: `[Monitoring].[spWriteLoadStatus]`

Purpose:

This stored procedure is designed to log the status and details of data load processes within the system. It helps in tracking the progress of ETL (Extract, Transform, Load) workflows by recording key steps, their status, and any relevant messages or errors.

Inputs:

- `@PipelineID` (NVARCHAR(255)): A unique identifier for the pipeline being executed.
- `@Process` (NVARCHAR(255), default NULL): The name or description of the process within the pipeline.
- `@Step` (NVARCHAR(255), default NULL): The specific step within the process being logged.
- `@Status` (NVARCHAR(255), default NULL): The current status of the step (e.g., 'Running', 'Completed', 'Failed').
- `@Rows` (BIGINT, default NULL): The number of rows affected or processed in this step.
- `@WorkflowID` (NVARCHAR(255), default NULL): An identifier for the overall workflow being executed.
- `@PipelineName` (NVARCHAR(255), default NULL): The name of the pipeline.
- `@Started_by` (NVARCHAR(255), default NULL): The user or system that initiated the process.
- `@Target` (NVARCHAR(255), default NULL): The target system or table being loaded.
- `@Source_system` (NVARCHAR(255), default NULL): The source system from which data is being extracted.
- `@Table` (NVARCHAR(255), default NULL): The specific table being loaded.

- `@Schema` (NVARCHAR(255), default NULL): The schema of the table being loaded.
- `@LoadType` (NVARCHAR(255), default NULL): The type of load being performed (e.g., 'Full', 'Delta').
- `@LapageRecord` (NVARCHAR(255), default NULL): The identifier or timestamp of the lapage record processed.
- `@ETL_Date` (DATETIME, default NULL): The date and time when the ETL process occurred.
- `@Log` (NVARCHAR(max), default 'No details provided'): A log message or additional details about the process or step.

Outputs:

- No direct output. The procedure inserts logs into `LS_Pipeline` and `LS_Trans` tables to record the process details and status.

Additional Logic:

- If the `@Step` is 'Start workflow' or 'Start load', it inserts a record into the `LS_Pipeline` table with basic details about the workflow or load.
- Every step is logged in the `LS_Trans` table.
- If the status is 'Failed', 'Fail', or 'Error', it raises an error with the logged message.

7.1.64 Procedure: `config.spGenerateDbreader`

Purpose:

This stored procedure dynamically generates and configures the `Yres_dbreader` database role in SQL Server. The role is designed to have specific read permissions while denying other actions on certain schemas and objects.

Detailed Description:

The procedure checks if the `Yres_dbreader` role already exists in the database. If it does, the role is dropped and recreated. The procedure then assigns this role to the `db_datareader` role, granting it select permissions on all objects by default. However, it explicitly denies select, insert, update, and delete permissions on various schemas and objects to ensure restricted access. Additionally, any members of existing roles are added to the `Yres_dbreader` role to maintain role memberships.

Inputs:

- There are no input parameters for this procedure.

Outputs:

The procedure dynamically generates and executes SQL commands that configure the `Yres_dbreader` role with specific permissions and memberships.

Procedure Logic:

1. Variable Declarations:

- `@SQL` (NVARCHAR(MAX)): Holds the dynamically generated SQL query.
- `@Schema` (NVARCHAR(1024)): Stores the schema name retrieved using the `Config.fxGetSetting` function with 'SchemaHIS' as the key.

2. Role Existence Check:

- The procedure checks if the `Yres_dbreader` role already exists using the `DATABASE_PRINCIPAL_ID` function.
- If the role exists, it appends a command to drop the existing role to the `@SQL` variable.

3. Role Creation and Configuration:

- The procedure appends commands to create the `Yres_dbreader` role and assigns it to the `db_datareader` role.
- The procedure then appends multiple `DENY` statements to restrict select permissions on specified schemas and objects.
- Additionally, `DENY INSERT`, `UPDATE`, and `DELETE` permissions are appended for the schema stored in `@Schema`.

4. Role Membership Configuration:

- The procedure dynamically adds existing role members to the `Yres_dbreader` role by querying `sys.database_role_members` and `sys.database_principals` to find relevant roles and members.

5. Execution of Dynamic SQL:

- The final SQL query stored in the `@SQL` variable is executed using the `EXEC` command.

Permissions Granted:

- The procedure grants select permissions on the `Loadmanagement.SurrogateKeys` object and denies select permissions on various other schemas and objects.
- It also denies execute permissions on specific stored procedures.

Role Membership:

- Any existing members of roles with the name `Yres_dbReader` are added to the newly created `Yres_dbreader` role.

This procedure ensures that the `Yres_dbreader` role is appropriately configured with the necessary permissions and maintains role membership consistency across the database.

7.2 Functions

7.2.1 Function: [Change].[fxGetChangelsOpen]

Purpose:

Determines if a specific change, identified by @ChangeId , is currently open.

Input Parameters:

- @ChangeId (int): The ID of the change to check.

Output:

- Returns 1 if the change is open, 0 if it is not open.

Description:

This function joins the Change.Changes and Change.Status tables to check if the status of the given change ID is marked as open. It also ensures the change originates from the correct server and database.

7.2.2 Function: [Change].[fxGetLapageChangeIdFor]

Purpose:

Retrieves the lapage change ID that includes a specific object type and name.

Input Parameters:

- @objectType (int): The type of the object (e.g., table, view).
- @objectName (nvarchar(1024)): The name of the object.

Output:

- Returns the lapage change ID (int) for the specified object.

Description:

The function searches the Change.ChangeContent table for the lapage change entry related to the specified object type and name. It handles specific conditions where table content or definitions are involved and returns the relevant change ID.

7.2.3 Function: [Change].[fxGetLapageOpenChange]

Purpose:

Fetches the lapage open change for a specified object type and name.

Input Parameters:

- @ObjectType (nvarchar(10)): The type of the object.
- @ObjectName (nvarchar(1024)): The name of the object.

Output:

- Returns the lapage open change ID (int).

Description:

This function combines the functionalities of `[Change].[fxGetLaspageChangeIdFor]` and `[Change].[fxGetChangeIsOpen]` to find and return the lapage open change for the given object type and name.

7.2.4 Function: [Change].[fxGetReleasedJson]**Purpose:**

Retrieves the JSON data associated with a specific change ID.

Input Parameters:

- `@ChangeID` (int): The ID of the change.

Output:

- Returns the JSON data (nvarchar(max)) related to the change.

Description:

The function queries the `Change.Changes` table to obtain the JSON data stored for a particular change ID.

7.2.5 Function: [Change].[fxGetTableDefinition]**Purpose:**

Generates a SQL script to recreate a table definition, including columns, indexes, constraints, and optionally the content as JSON.

Input Parameters:

- Various parameters to control the script generation, such as `@ObjectName`, `@UseTransaction`, `@GenerateFKs`, etc.

Output:

- Returns the generated SQL script (nvarchar(max)).

Description:

This comprehensive function creates a detailed SQL script to recreate a table, including its structure and content. The generated script can optionally include foreign keys, identity columns, constraints, and table content as JSON.

7.2.6 Function: [Change].[fxGetTableTypeDefinition]**Purpose:**

Generates the SQL script to define a table type based on its name.

Input Parameters:

- `@ObjectName` (nvarchar(1024)): The name of the table type.

Output:

- Returns the table type definition script (nvarchar(max)).

Description:

The function constructs the SQL script needed to recreate a user-defined table type, including its columns and their properties.

7.2.7 Function: [Config].[fxAddTryCatch]

Purpose:

Wraps a given SQL script in a try-catch block for error handling and logging.

Input Parameters:

- `@SQL` (nvarchar(max)): The SQL script to be executed.
- `@process` (uniqueidentifier): The process ID for logging.
- `@spName` (nvarchar(max)): The stored procedure name for logging.
- `@spStep` (nvarchar(max)): The step name within the stored procedure.
- `@spCall` (nvarchar(max)): The stored procedure call information.
- `@appUser` (nvarchar(max)): The application user executing the script.

Output:

- Returns the wrapped SQL script (nvarchar(max)).

Description:

The function adds error handling to a given SQL script by wrapping it in a try-catch block. It logs the process and any errors using predefined stored procedures.

7.2.8 Function: [Config].[fxGetSchemaName]

Purpose:

Retrieves the schema name for a specific target and type, considering configuration settings and overrides.

Input Parameters:

- `@Target` (nvarchar(max)): The target for which the schema name is required.
- `@Type` (nvarchar(max)): The type of schema (e.g., 'HIS', 'STAGE').

Output:

- Returns the schema name (nvarchar(max)).

Description:

The function determines the schema name based on configuration settings and potential overrides in the `Loadmanagement.UsedTables` table.

7.2.9 Function: [Config].[fxGetSession]**Purpose:**

Returns session details for active connections, formatted as JSON.

Input Parameters:

- None.

Output:

- Returns session details (nvarchar(max)) as JSON.

Description:

This function queries the `sys.dm_exec_connections` and `sys.dm_exec_sessions` system views to retrieve details of active sessions, formatting the results as JSON.

7.2.10 Function: [Config].[fxGetSetting]**Purpose:**

Fetches a configuration setting value by name, with special handling for 'GodMode'.

Input Parameters:

- `@SettingName` (nvarchar(max)): The name of the setting to retrieve.

Output:

- Returns the setting value (nvarchar(max)).

Description:

The function retrieves the value of a specified setting from the `Config.Settings` table. It includes special handling for 'GodMode', which can override certain settings.

7.2.11 Function: [dbo].[fxGeneratePassword]**Purpose:**

Generates a random password of a specified length.

Input Parameters:

- `@Length` (int): The desired length of the password (default is 16).

Output:

- Returns the generated password (nvarchar(1024)).

Description:

This function generates a random password containing characters between ASCII values 48 and 122, ensuring a mix of alphanumeric characters.

7.2.12 Function: [dbo].[fxGetJsonCollection]

Purpose:

Retrieves a JSON collection from a given JSON string.

Input Parameters:

- `@Json` (nvarchar(max)): The input JSON string.

Output:

- Returns the JSON collection (nvarchar(1024)).

Description:

The function extracts a JSON collection from an input JSON string by querying the `dbo.fxGetJsonCollections` function.

7.2.13 Function: [dbo].[fxRemoveNonAlphaCharacters]

Purpose:

Removes non-alphabetic characters from a given string.

Input Parameters:

- `@Temp` (varchar(1000)): The input string.

Output:

- Returns the cleaned string (varchar(1000)) with only alphabetic characters.

Description:

The function iterates through the input string and removes any non-alphabetic characters, returning a string that contains only alphabetic characters.

7.2.14 Function: [dbo].[fxStripCharacters]

Purpose:

Strips specified characters from a string based on a match expression.

Input Parameters:

- `@String` (nvarchar(max)): The input string.
- `@MatchExpression` (varchar(255)): The match expression defining characters to be stripped.

Output:

- Returns the cleaned string (nvarchar(max)) with specified characters removed.

Description:

The function removes characters from the input string that match the provided match expression, useful for cleaning up strings based on custom criteria.

7.2.15 Function: [dbo].[fxToDecimal]**Purpose:**

Converts a string to a decimal, handling various formats and potential negative values.

Input Parameters:

- `@string` (nvarchar(1024)): The input string to be converted.
- `@decimals` (int, default 0): The number of decimal places.
- `@findNegative` (bit, default 1): Indicates whether to check for negative values.

Output:

- Returns the converted decimal value (decimal(25,8)).

Description:

This function attempts to convert a string to a decimal, handling different formats and considering negative values if specified.

7.2.16 Function: [dbo].[fxToProper]**Purpose:**

Converts a string to proper case (first letter uppercase, rest lowercase).

Input Parameters:

- `@string` (nvarchar(4000)): The input string.

Output:

- Returns the converted string (nvarchar(4000)).

Description:

The function converts the input string to proper case, making the first letter uppercase and the rest of the letters lowercase.

7.2.17 Function: [dbo].[fxUNIXtoDateTime]**Purpose:**

Converts a UNIX timestamp (epoch) to a datetime value.

Input Parameters:

- `@epochStringDate` (nvarchar(50)): The input UNIX timestamp as a string.

Output:

- Returns the converted datetime value (datetime2(0)).

Description:

The function converts a UNIX timestamp (epoch) into a SQL Server datetime value, making it easier to work with in SQL queries.

7.2.18 Function: [dbo].[fxUTC2

CET]

Purpose:

Converts a UTC datetime value to Central European Time (CET).

Input Parameters:

- @UTC_DateTime (datetime): The input UTC datetime value.

Output:

- Returns the converted CET datetime value (datetime).

Description:

The function converts a given UTC datetime value to the corresponding Central European Time (CET) datetime value.

These functions provide a variety of utility operations, from checking the status of changes, generating SQL scripts, handling configurations, to manipulating and converting data types.

7.2.19 Function: [LoadManagement].[fxGetActualTablename]**Purpose:**

Fetches the actual table name for a given source, schema, and table.

Input Parameters:

- @Source (nvarchar(1024)): The source system name.
- @Schema (nvarchar(1024)): The schema name.
- @Table (nvarchar(1024)): The table name.

Output:

- Returns the actual table name (nvarchar(4000)).

Description:

This function queries the LoadManagement.UsedTables table to find the actual table name corresponding to the given source, schema, and table. If not found, it looks up in the CustomYres.Extractor table.

7.2.20 Function: [LoadManagement].[fxGetDataType]

Purpose:

Determines the full target column data type based on various input parameters.

Input Parameters:

- `@source` (nvarchar(256)): The source system name.
- `@dataType` (nvarchar(256)): The source data type.
- `@sourceLength` (int): The length of the source data type.
- `@sourcePrecision` (int): The precision of the source data type.
- `@sourceScale` (int): The scale of the source data type.
- `@schema` (nvarchar(256), default 'N/A'): The schema name.
- `@table` (nvarchar(256), default 'N/A'): The table name.
- `@column` (nvarchar(256), default 'N/A'): The column name.

Output:

- Returns the full target data type (nvarchar(256)).

Description:

This function maps the source data type to a target data type using the `LoadManagement.TypeMapping` and `LoadManagement.GlobalTypeMapping` tables. It considers length, precision, and scale attributes, and includes any necessary overrides.

7.2.21 Function: [LoadManagement].[fxGetKeyColumns]

Purpose:

Retrieves key columns for a specified table, including options for additional formatting.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.
- `@withType` (bit, default 0): Flag to include data type.
- `@enclosureChar` (nvarchar(1), default ''): Character to enclose column names.
- `@separatorChar` (nvarchar(1), default ','): Character to separate columns.

Output:

- Returns the key columns (nvarchar(max)).

Description:

This function fetches the key columns for the specified table from `LoadManagement.vwDictionary`. If no key columns are found, it defaults to using all columns.

7.2.22 Function: [LoadManagement].[fxGetKeyHashColumns]**Purpose:**

Generates a hash of key columns for a specified table.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.

Output:

- Returns the key hash columns (nvarchar(max)).

Description:

This function constructs a concatenation of key columns to generate a hash value for the specified table. If no key columns are found, it defaults to using all columns.

7.2.23 Function: [LoadManagement].[fxGetOptimized]**Purpose:**

Determines if a table is memory-optimized based on settings or actual status.

Input Parameters:

- `@Type` (nvarchar(50)): The type of optimization ('STAGE' or 'HIS').
- `@Table` (nvarchar(250)): The table name.
- `@Status` (nvarchar(50), default 'Intended'): Specifies whether to check intended or actual status.

Output:

- Returns the optimization status (nvarchar(20)).

Description:

This function checks if a table is memory-optimized based on its settings in `LoadManagement.UsedTables` or `Config.Settings`. It can return either the intended or actual status.

7.2.24 Function: [LoadManagement].[fxGetRowColumns]**Purpose:**

Retrieves the row columns for a specified table, with options for additional formatting.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.
- `@withType` (bit, default 0): Flag to include data type.
- `@enclosureChar` (nvarchar(1), default ''): Character to enclose column names.
- `@separatorChar` (nvarchar(1), default ','): Character to separate columns.

Output:

- Returns the row columns (nvarchar(max)).

Description:

This function fetches the row columns for the specified table from

`LoadManagement.vwDictionary`, including optional data type and formatting based on input parameters.

7.2.25 Function: [LoadManagement].[fxGetRowHashColumns]

Purpose:

Generates a hash of row columns for a specified table.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.

Output:

- Returns the row hash columns (nvarchar(max)).

Description:

This function constructs a concatenation of row columns to generate a hash value for the specified table.

7.2.26 Function: [LoadManagement].[fxGetStoreType]

Purpose:

Determines the storage type (row or column) for a table based on settings or actual status.

Input Parameters:

- `@Table` (nvarchar(250)): The table name.
- `@Status` (nvarchar(50), default 'Intended'): Specifies whether to check intended or actual status.

Output:

- Returns the storage type (nvarchar(20)).

Description:

This function checks if a table uses row or column storage based on its settings in

`LoadManagement.UsedTables` or `Config.Settings`. It can return either the intended or actual status.

7.2.27 Function: [LoadManagement].[fxGetSurrogate]**Purpose:**

Retrieves the surrogate key setting for a specified table.

Input Parameters:

- `@Table` (nvarchar(4000)): The table name.

Output:

- Returns the surrogate key setting (int).

Description:

This function checks the surrogate key setting for the specified table in

`LoadManagement.UsedTables`. If not found, it defaults to a global setting from `Config.Settings`.

7.2.28 Function: [LoadManagement].[fxPredictKeyColumns]**Purpose:**

Predicts key columns for a new table based on existing metadata.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.

Output:

- Returns the predicted key columns (nvarchar(max)).

Description:

This function predicts the key columns for a new table based on existing metadata in

`LoadManagement.Dictionary`. If no key columns are found, it defaults to using all non-nullable columns.

7.2.29 Function: [LoadManagement].[fxPredictTableName]

Purpose:

Predicts the table name based on source, schema, and table parameters.

Input Parameters:

- `@Source` (nvarchar(1024)): The source system name.
- `@Schema` (nvarchar(1024)): The schema name.
- `@Table` (nvarchar(1024)): The table name.

Output:

- Returns the predicted table name (nvarchar(4000)).

Description:

This function constructs a predicted table name using the source, schema, and table parameters, considering any overrides specified in `LoadManagement.UsedTables`.

7.2.30 Function: [oData].[fxBaseResponse]

Purpose:

Generates a base oData response URL for metadata.

Input Parameters:

- `@BaseUrl` (nvarchar(3000)): The base URL.

Output:

- Returns the oData base response (nvarchar(max)).

Description:

This function constructs a base oData response URL, appending the metadata endpoint and listing all tables and views available in the database.

7.2.31 Function: [oData].[fxMetadataResponse]

Purpose:

Generates oData metadata response.

Input Parameters:

- `@BaseUrl` (nvarchar(3000)): The base URL.

Output:

- Returns the oData metadata response (nvarchar(max)).

Description:

This function generates an oData metadata response, including details about entity types and sets. It

maps SQL types to corresponding oData EDM types and ensures the response adheres to oData standards.

7.2.32 Function: [Metadata].[fxGetViewSources]

Purpose:

Retrieves the source objects (tables or views) for a specified view.

Input Parameters:

- `@VIEW` (nvarchar(256)): The name of the view.

Output:

- Returns a table with the source objects and their details.

Description:

This function identifies the source objects used by a specified view by parsing the view definition and querying system tables to

find the related tables or views.

7.2.33 Function: [Metadata].[fxGetViewSourcesRecursive]

Purpose:

Recursively retrieves the source objects (tables or views) for a specified view.

Input Parameters:

- `@VIEW` (nvarchar(256)): The name of the view.

Output:

- Returns a table with the source objects and their details, including recursive sources.

Description:

This function recursively identifies the source objects used by a specified view and any views that those source objects depend on, providing a comprehensive list of all related tables or views.

These functions offer a variety of utility operations, from retrieving and predicting table names, generating metadata responses, to recursively identifying dependencies in views.

7.2.34 Function: [Monitoring].[fxGetTableLoads]

Purpose:

This function retrieves a paginated list of load records for a specific source and target, showing the ETL date and status. It is useful for monitoring and tracking the loading processes of different tables.

Inputs:

- `@offset` (int): The number of rows to skip before starting to return rows.
- `@top` (int): The number of rows to return.

- `@source` (nvarchar(1024)): The source system name.
- `@target` (nvarchar(1024)): The target table name.

Outputs:

- `[Source system]` (nvarchar(1024)): The source system name.
- `[Target]` (nvarchar(1024)): The target table name.
- `[ETL Date]` (datetime): The ETL date of the load.
- `[Status]` (nvarchar(50)): The status of the load.

7.2.35 Function: [LoadManagement].[fxExtractor]

Purpose:

This function generates a dynamic dataset of tables and their metadata for extraction. It includes source type, delta scripts, and load filters based on the provided parameters. This function is essential for preparing and managing data extraction processes in a data warehouse environment.

Inputs:

- `@LoadFilter` (nvarchar(max), default NULL): The filter to apply on the load.
- `@LoadType` (nvarchar(max), default NULL): The type of load (e.g., FULL, DELTA).
- `@PackageSize` (int, default NULL): The size of the package for loading.

Outputs:

- `sourceType` (nvarchar(50)): The type of the source system.
- `isFile` (bit): Indicates if the source is a file.
- `Source` (nvarchar(1024)): The source system name.
- `SourceSchema` (nvarchar(1024)): The source schema name.
- `SourceTable` (nvarchar(1024)): The source table name.
- `TechSchema` (nvarchar(1024)): The technical schema name.
- `TechTable` (nvarchar(1024)): The technical table name.
- `DataPlatform` (nvarchar(1024)): The name of the data platform.
- `LoadType` (nvarchar(50)): The type of load (FULL, DELTA, etc.).
- `DeltaColumn` (nvarchar(1024)): The delta column name used for incremental loads.
- `LapageRecord` (nvarchar(1024)): The lapage record identifier.
- `DeltaScript` (nvarchar(max)): The generated SQL script for extracting delta changes.
- `loadFilter` (nvarchar(max)): The load filter applied to the extraction.
- `Pipeline` (nvarchar(1024)): The associated pipeline for the load.

- `Trigger` (nvarchar(1024)): The trigger information.
- `fileLocation` (nvarchar(1024)): The location of the file source.
- `fileName` (nvarchar(1024)): The name of the file source.
- `TargetSource` (nvarchar(1024)): The target source system name.
- `TargetSchema` (nvarchar(1024)): The target schema name.
- `TargetTable` (nvarchar(1024)): The target table name.
- `Target` (nvarchar(1024)): The formatted target table name.
- `keepStage` (bit): Indicates if the staging data should be kept.
- `TargetSchemaHIS` (nvarchar(1024)): The target historical schema name.
- `TargetSchemaSTAGE` (nvarchar(1024)): The target staging schema name.
- `CellRange` (nvarchar(1024)): The cell range in the source.
- `Sheet` (nvarchar(1024)): The sheet name in the source.
- `FileType` (nvarchar(1024)): The type of the file.
- `ColumnDelimiter` (nvarchar(1)): The column delimiter in the source file.
- `RowDelimiter` (nvarchar(1)): The row delimiter in the source file.
- `QuoteCharacter` (nvarchar(1)): The quote character in the source file.
- `FirstRowHeader` (bit): Indicates if the first row is a header.
- `EscapeCharacter` (nvarchar(1)): The escape character in the source file.
- `CompressionType` (nvarchar(50)): The compression type.
- `LapageRuntime` (int): The lapage runtime for the pipeline in seconds.
- `noKey` (bit): Indicates if there is no key column.
- `deltaOverlap` (int): The delta overlap value.
- `deltaOverlapUnit` (nvarchar(50)): The unit for delta overlap.
- `PackageSize` (int): The size of the package for loading.
- `ArchivingScript` (nvarchar(max)): The generated SQL script for archiving data.

7.3 Logs

The database logs 4 main information flows:

1. The creation, altering and/or deletion of objects
2. Granting, Denying and Revoking user rights
3. Procedures and their results
4. Running and completed loads

These logs can be found in the following tables

- [Config].[EventLog]
- [Config].[ProcessLog]
- [Monitoring].[ls_trans]
- [Monitoring].[ls_pipeline]

Abstractions of these tables with more useful insights can be found in these views

User and access management

[Monitoring].[vwAccessManagement]

[Monitoring].[vwUserManagement]

Load monitoring

[Monitoring].[vwUsedTables]

[Monitoring].[vwLoadMonitor]

[Monitoring].[vwLoads]

[Monitoring].[vwMonitor]

Object manipulation

[Monitoring].[vwObjectAlterations]

7.4 Views

7.4.1 View: [Monitoring].[vwLoadMonitor]

Purpose:

To provide a comprehensive view of the load monitoring process, showing detailed load steps, runtime metrics, row counts, and statuses for each pipeline.

Output Columns:

- [Workflow ID] (WorkflowID)
- [Pipeline ID] (PipelineID)
- [Pipeline name] (PipelineName)
- [Source system] (Source_system)
- [Target] (Target)
- [DateTime] (DateTime)
- [Started by] (Started_by)
- [LoadType] (LoadType)

- [LapageRecord] (LapageRecord)
- [ETL Date] (ETLDate)
- [Start load] (Start load)
- [Runtime sec: write loadstatus] (Runtime in seconds for writing load status)
- [Start truncate table] (Start truncate table)
- [Runtime sec: truncate table] (Runtime in seconds for truncating table)
- [Start load data] (Start load data)
- [Runtime sec: load data] (Runtime in seconds for loading data)
- [Start upsert HIS] (Start upsert HIS)
- [Runtime sec: upsert HIS] (Runtime in seconds for upserting HIS)
- [Start update ETL_EndDate] (Start update ETL_EndDate)
- [Runtime sec: update ETL_EndDate] (Runtime in seconds for updating ETL_EndDate)
- [Start count rows] (Start count rows)
- [Runtime sec: update count rows] (Runtime in seconds for counting rows)
- [End load] (End load)
- [Runtime sec: Pipeline] (Total runtime in seconds for the pipeline)
- [Copied rows] (Number of copied rows)
- [New rows] (Number of new rows)
- [Delta rows] (Number of delta rows)
- [Deleted rows] (Number of deleted rows)
- [Status] (Status of the load)
- [LapageLoad] (Indicator if it is the lapage load)

7.4.2 View: [LoadManagement].[vwDictionary]

Purpose:

To provide a detailed dictionary view for the load management process, including source and target data types, key columns, nullability, and other metadata for each column.

Output Columns:

- RowId (Generated row number for partition)
- Source (Source system name)
- SourceSchema (Source schema name)
- SourceTable (Source table name)

- `SourceColumn` (Source column name)
- `SourceDataType` (Source data type)
- `TargetDataType` (Mapped target data type)
- `Nullable` (Nullability of the column)
- `KeyColumn` (Indicator if the column is a key column)
- `RowHashColumn` (Indicator if the column is used in row hash)
- `FactColumn` (Indicator if the column is a fact column)
- `Position` (Position of the column)
- `LoadType` (Load type for the table)
- `deltaColumn` (Indicator if the column is a delta column)
- `TargetSource` (Target source system name)
- `TargetSchema` (Target schema name)
- `TargetTable` (Target table name)
- `ActualTableName` (Actual table name)
- `ActualStageSchema` (Actual staging schema name)
- `ActualHisSchema` (Actual historical schema name)
- `keepStage` (Indicator if the staging data should be kept)

7.4.3 View: [LoadManagement].[vwUsedTables]

Purpose:

To provide a view of all used tables in the load management process, including metadata, row counts in DWH and staging, lapage load details, and change IDs.

Output Columns:

- `Source` (Source system name)
- `SourceSchema` (Source schema name)
- `SourceTable` (Source table name)
- `DataPlatform` (Formatted data platform name)
- `LoadType` (Load type for the table)
- `DeltaColumn` (Delta column name)
- `CellRange` (Cell range in the source)
- `Sheet` (Sheet name in the source)
- `LapageRecord` (Indicator for the lapage record)

- `PackageSize` (Size of the package)
- `StageMemOptimized` (Indicator if staging is memory optimized)
- `ODSMemOptimized` (Indicator if ODS is memory optimized)
- `Surrogate` (Surrogate key indicator)
- `StoreType` (Storage type)
- `DeltaOverlap` (Delta overlap value)
- `DeltaOverlapUnit` (Unit for delta overlap)
- `LoadFilter` (Load filter criteria)
- `fileName` (File name in the source)
- `fileLocation` (File location in the source)
- `firstRowHeader` (Indicator if the first row is a header)
- `ColumnDelimiter` (Column delimiter in the source file)
- `RowDelimiter` (Row delimiter in the source file)
- `QuoteCharacter` (Quote character in the source file)
- `EscapeCharacter` (Escape character in the source file)
- `ChangeId` (Lapage open change ID)
- `[Rows in DWH]` (Row count in DWH)
- `[Rows in Staging]` (Row count in staging)
- `[Lapage Load]` (Date of the lapage load)
- `[Status of lapage load]` (Status of the lapage load)
- `Columns` (JSON formatted columns metadata)
- `Loads30Days` (JSON formatted loads in the last 30 days)
- `Details` (JSON formatted details of the table)
- `TargetTable` (Target table name)
- `MetadataComparison` (JSON formatted metadata comparison)

7.4.4 View: `[Monitoring].[vwUsedTables]`

Purpose:

To provide a view of used tables in the monitoring process, showing the lapage load and status for each table.

Output Columns:

- `Source` (Source system name)

- `SourceSchema` (Source schema name)
- `SourceTable` (Source table name)
- `LoadType` (Load type for the table)
- `LapageLoad` (Date of the lapage load)
- `LapageStatus` (Status of the lapage load)

7.4.5 View: [Monitoring].[vwMonitor]

Purpose:

To provide a comprehensive view of the monitoring process, showing detailed load steps, runtime metrics, row counts, statuses, and deletion details for each pipeline.

Output Columns:

- `[Workflow ID]` (WorkflowID)
- `[Pipeline ID]` (PipelineID)
- `[Pipeline name]` (PipelineName)
- `[Source system]` (Source_system)
- `[Target]` (Formatted target name)
- `[DateTime]` (DateTime)
- `[Started by]` (Started_by)
- `[LoadType]` (LoadType)
- `[ETL Date]` (ETLDate)
- `[Start load]` (Start load)
- `[Start load data]` (Start load data)
- `[Runtime sec: load data]` (Runtime in seconds for loading data)
- `[Start upsert HIS]` (Start upsert HIS)
- `[Runtime sec: upsert HIS]` (Runtime in seconds for upserting HIS)
- `[Start update ETL_EndDate]` (Start update ETL_EndDate)
- `[Runtime sec: update ETL_EndDate]` (Runtime in seconds for updating ETL_EndDate)
- `[End load]` (End load)
- `[Runtime sec: Pipeline]` (Total runtime in seconds for the pipeline)
- `[Copied rows]` (Number of copied rows)
- `[New rows]` (Number of new rows)
- `[Delta rows]` (Number of delta rows)

- `[Deleted rows]` (Number of deleted rows)
- `[Status]` (Status of the load)
- `[LapageLoad]` (Indicator if it is the lapage load)
- `[Deleted]` (Indicator if the pipeline was deleted)
- `[Deleted by]` (User who deleted the pipeline)
- `[Deleted on]` (Date when the pipeline was deleted)

7.4.6 View: `[Config].[vwUserLog]`

Purpose:

This view is designed to provide a detailed log of user activities, including process steps, return codes, and messages. It is useful for auditing and tracking user actions within the system.

Outputs:

- `ProcessID` : Unique identifier for the process.
- `Date` : Date of the log entry.
- `Time` : Time of the log entry.
- `timestamp` : Timestamp of the log entry.
- `UserId` : User identifier extracted from the `appUser` field.
- `User` : User name extracted from the `appUser` field.
- `Process` : Name of the process.
- `Process step` : Specific step within the process.
- `returnCode` : Return code of the process step.
- `MessageType` : Type of message based on the return code.
- `Message` : Concatenated message from multiple columns.
- `dbRequest` : Database request associated with the log entry.

7.4.7 View: `[Config].[vwUserLogJSON]`

Purpose:

This view provides a JSON representation of user logs, aggregating process steps and messages for each process. It is useful for integrating with applications that consume JSON data.

Outputs:

- `process` : Name of the process.
- `returnCode` : Highest return code for the process.
- `messageType` : Type of message based on the highest return code.

- `timestamp` : Timestamp of the log entry.
- `dbRequest` : Database request associated with the log entry.
- `user` : User name.
- `userId` : User identifier.
- `processID` : Unique identifier for the process (used in web applications).
- `Steps` : JSON array of steps, messages, and timestamps for the process.

7.4.8 View: [Config].[vwDictionaryVsHis]

Purpose:

This view compares the dictionary definitions of tables and their historical counterparts to identify discrepancies such as missing columns, type mismatches, and deleted columns. It is useful for maintaining consistency between current and historical table schemas.

Outputs:

- Columns from the dictionary and used tables.
- `MissingColumns` : Indicates if columns are missing.
- `TypeMismatch` : Indicates if there is a data type mismatch.
- `DeletedColumns` : Indicates if columns have been deleted.

7.4.9 View: [Config].[vwDictionaryVsStage]

Purpose:

This view compares the dictionary definitions of tables and their staging counterparts to identify discrepancies such as missing columns, type mismatches, and deleted columns. It is useful for ensuring consistency between current and staging table schemas.

Outputs:

- Columns from the dictionary and used tables.
- `MissingColumns` : Indicates if columns are missing.
- `TypeMismatch` : Indicates if there is a data type mismatch.
- `DeletedColumns` : Indicates if columns have been deleted.

7.4.10 View: [LoadManagement].[vwExtractor]

Purpose:

This view provides a dynamic dataset of tables and their metadata for extraction, utilizing the `fxExtractor` function. It is useful for managing data extraction processes in a data warehouse environment.

Outputs:

- All columns from the `fxExtractor` function.

7.4.11 View: [LoadManagement].[vwArchivingExtractor]

Purpose:

This view generates SQL scripts for archiving data based on the load type and delta columns. It is useful for automating the archiving process of historical data.

Outputs:

- All columns from the archiving extractor function.
- `ArchivingDeltaScript` : Generated SQL script for archiving delta changes.

7.4.12 View: [LoadManagement].[vwUnusedTables]

Purpose:

This view identifies tables that are defined in the dictionary but not used in the active used tables. It is useful for cleaning up unused tables and maintaining a streamlined data warehouse.

Outputs:

- `Source` : Source system name.
- `SourceSchema` : Source schema name.
- `SourceTable` : Source table name.

7.4.13 View: [LoadManagement].[vwUsedColumns]

Purpose:

This view provides a list of columns that are actively used in the data warehouse. It is useful for managing and auditing the usage of table columns.

Outputs:

- `Source` : Source system name.
- `SourceSchema` : Source schema name.
- `SourceTable` : Source table name.
- `SourceColumn` : Source column name.

7.4.14 View: [LoadManagement].[vwUsedODSTablesAndColumns]

Purpose:

This view provides a JSON representation of ODS (Operational Data Store) tables and their columns, including target data types. It is useful for integrating with applications that consume JSON data and for managing ODS tables and columns.

Outputs:

- `ODS_Schema` : ODS schema name.
- `Source` : Source system name.
- `SourceSchema` : Source schema name.
- `SourceTable` : Source table name.
- `Columns` : JSON array of columns and their target data types.

7.4.15 View: [LoadManagement].[vwViewPersistence]

Purpose:

This view provides information about the persistence of views, including source and destination details, delta columns, and last persist dates. It is useful for managing the persistence and updates of views in a data warehouse environment.

Outputs:

- Various columns related to view persistence, including source and destination details, delta columns, and last persist dates.

7.4.16 View: [LoadManagement].[vwViewsAndColumns]

Purpose:

This view provides a JSON representation of views and their columns, including data types. It is useful for integrating with applications that consume JSON data and for managing view columns in a data warehouse environment.

Outputs:

- `TABLE_SCHEMA` : Schema name of the view.
- `TABLE_NAME` : Name of the view.
- `Columns` : JSON array of columns and their data types.

7.4.17 12. View: [Monitoring].[vwLoads]

Purpose:

This view provides detailed information about data loads, including timestamps, runtime metrics, row counts, and statuses. It is useful for monitoring and analyzing the performance of data loading processes.

Outputs:

- Various columns related to load processes, including pipeline and workflow IDs, timestamps, runtime metrics, row counts, and statuses.

7.4.18 View: [Monitoring].[vwWorkflow]

Purpose:

This view provides detailed information about workflows, including timestamps, runtime metrics, row counts, and statuses. It is useful for monitoring and analyzing the performance of workflows in a data warehouse environment.

Outputs:

- Various columns related to workflow processes, including pipeline and workflow IDs, timestamps, runtime metrics, row counts, and statuses.

8 Release notes

Welcome to the release notes section of Yres. This section contains release notes for current and previous versions of Yres. Do note that older versions listed here may no longer be supported, and are kept for archival purposes.

8.1 Version 1.55 - September 2025

8.1.1 Yres DWH – Version 1.55 (September 2025)

8.1.1.1 New Features

8.1.1.1.1 Data Processing & Loading

- Select columns that are used when comparing rows to determine changed records

8.1.1.1.2 Data Sources & Connectivity

- Yres now manages Integration Runtimes from the frontend allowing for a better managed experience when rebuilding and/or upgrading to a newer version
- Existing Integration runtimes can be “imported” into the new solution

8.1.1.1.3 User Interface

- Action buttons in the datasource menu’s are moved to a : menu to save space
- Rest datasources now allow users to choose default values for centrally managed settings like KeepStage and Row/Column store tables.
- Rest Sources are now named after their target instead of the source endpoint
- File and Rest sources now require Project and change at the start of the wizard

8.1.1.1.4 Life cycle management

- Environment comparison - Compare SQL definitions between environments
- Full diff view when comparing SQL script between versions or environments
- We have added a setting on Organization level that allows specific ADF objects to be excluded when publishing changes to ADF. This allows a more controllable way of handling custom adjustments to standardized objects.

8.1.1.1.5 Automation & Pipelines

- Main pipelines can now be copied to a new version

8.1.1.1.6 Security improvements

- Backend URL's are now based on UUID's to prevent predictable endpoints
- New installations require an additional property in preparation for subdomains per organization
- Client & Secrets used by Yres to access the Organizations Azure tenant can now be managed by Admins

8.1.1.2 Enhancements

- Textual improvements
- Issue with selecting released changed automatically is now solved
- Disappearing datasource tags now solved

8.2 Version 1.54 - July 2025

8.2.1 Yres DWH – Version 1.54 (July 2025)

8.2.1.1 New Features

8.2.1.1.1 Data Processing & Loading

- Option to deactivate tables so they can temporary be excluded from loads
- Start loads directly from the datasource menu
- Persist view directly from the Persist View menu

8.2.1.1.2 Data Sources & Connectivity

- Support for more than 80 sources in refresh metadata
- Added support for PAT tokens in OneStream

8.2.1.1.3 Monitoring & User Interface

- Add filter option in Pipeline monitor
- Extra time selections in monitor
 - 8 hours
 - 4 hours
 - 1 hours
- Paging in monitored Jobs
- Added Materialized Views to monitor

Projects and Changes

- New iterative process to handle complex dependencies
- Improved messages and error handling
- Add support for Naming overwrites between dev, page and prod
 - This allows for sources that have different names for source objects between the environments.
 - i.e. dev = ERP_DEV.Customers , page = ERP_TST.Customers & Prod = ERP.Customers

8.2.1.1.4 Automation & Pipelines

- Added support for Persist View from Main pipeline
- Allow to select Custom sources when starting loads
- Set parallelism for loads
- Alternative load is now fully available

8.2.1.1.5 Communication & Notifications

- Announcements are now available for organisation admins

8.2.1.2 Enhancements

- Redesign Trigger form
- Cleared dropdowns after switching screens
- Add copied records to monitor from stage2His
- Error when ServiceTier does not exist
- Fixed Main pipeline issue when adding objects
- Added missing texts
- PowerBI Menu not available for non-Admins
- Added logging in loads for better performance analysis
- Improvements in loading local files
- Fixed issues with deleting Paginated Rest and oData sources
- RestAPI supports more formats of JSON
- Fixed scaling issue when starting load from RunPipelines
- Fix for BDC source Duplicate handling
- Improvements in support for Dutch language
- General improvements in deleting and updating tables

8.3 Version 1.53 - May 2025

8.3.1 Yres DWH – Version 1.53 (May 2025)

8.3.1.1 🚀 New Features

8.3.1.1.1 Data Processing & Loading

- **Delta Image Load Mode**
Introduced a new 'Delta Image' load mode, allowing selective reloading of specific periods (e.g., the previous year) while removing outdated records from the target dataset and preserving historical data.
- **Enhanced Paging Support in REST Sources**
Implemented comprehensive paging capabilities in REST service sources, supporting RFC 5988, offset-based paging, looped page traversal, and body-result URL extraction.
- **Scalable Data Processing**
Data processing now supports paging mechanisms, enabling efficient handling of large datasets (100 million+ records) by segmenting loads based on system capacity.
- **PowerBI support**
Yres now supports refreshing PowerBI Models in loads. For more details, read the Masterpipeline update.

8.3.1.1.2 Data Sources & Connectivity

- **New Data Source Integrations**
 - **OneStream:** Added support for OneStream as a data source.
 - **SAP Business Data Cloud:** Integrated SAP Business Data Cloud as a new source system.
 - **Simplicate:** Simplicite is now available as a fully supported data source.
- **OpenAPI Support for REST Integrations**
REST API connections now support the **OpenAPI specification**. APIs that provide an `openapi.json` or `swagger.json` definition can be added much more easily. Users can visually select available endpoints directly from the frontend, significantly simplifying configuration.
- **Custom Database Deployment**
Users can now opt to deploy on their own existing Azure databases instead of the default embedded database.

8.3.1.1.3 Monitoring & User Interface

- **Centralized Environment Display**
The active environment (Development, Test, Production) is now prominently displayed in the top-left corner, simplifying environment switching.

- **New Monitoring Dashboard**
Introduced a comprehensive monitoring screen that provides easy access to historical data, including table sizes in MB and row counts.
- **Improved Upgrade & Rebuild Monitoring**
Monitoring of system upgrades and rebuilds has been improved, providing more transparent insights into these operations and their outcomes.
- **Adjustable Sidebar Menu**
The left-hand menu is now resizable, allowing users to customize their workspace layout.
- **Improved Modal Windows**
Enhanced the sizing and responsiveness of modal windows for a better user experience.
- **Refresh button to refresh in the webapp**
We have added a refresh button in the webapp so users can reload the shown data and items without having to reload the entire page.
- **Working deep links**
When you share a link to a specific page in Yres, the link will now actually open that specific page instead of routing users to the home screen.

8.3.1.1.4 Table & Schema Management

- **Revamped Table Creation Flow**
Overhauled the process for creating tables, allowing users to specify data types for each field during setup.
- **Enhanced Data Source Tagging**
Users can now add tags to data sources directly above the tables, eliminating the need to modify the data source itself.
- **Column Usage Analysis**
New functionality to identify unused columns within tables and trace their usage across the system, aiding in schema optimization.

Projects and Changes

- **Database Object Tree Change Inclusion**
Users can now select **existing database objects** directly from the object tree and include them in change sets. This simplifies the process of registering elements for versioning—but does not allow new object creation from this view.
- **Choose to include dependencies & content in changes**
You can now choose to include dependencies or just take the selected object to the next environment.
- **Bring along metadata from the Development environment**
When moving a source or table from Dev to Prod, Yres will now use the Metadata of Dev by default. This simplifies the installation process.
- **View scripted objects in a change**
The change content overview in the release screen now shows all objects instead of just the Yres managed objects.

8.3.1.1.5 Automation & Pipelines

- **Master Pipeline Configuration**
Introduced a master pipeline feature that allows users to sequence actions via the frontend. Actions can be conditionally executed based on the success or failure of preceding steps, facilitating complex load processes and conditional Power BI model refreshes.
- **Run full, Image, Overwrite or Reload Once**
You can now run or plan an alternative load type. This means you can run Delta's during the week while reloading all data in the weekend. Or just reload all data once without having to alter the table.
- **Garbage Collection Pipeline**
Implemented a garbage collection pipeline in ADF that retrieves the statuses of canceled pipelines, aiding in resource cleanup.
- **Trigger ADF Pipelines from Frontend**
Users can now trigger **custom Azure Data Factory (ADF) pipelines** directly from the frontend interface. This improves operational control without needing to navigate to Azure.
- **Improvements made in sorting load items for parallelism**
Tables that take long to load are now grouped with faster tables to improve the benefits of using parallelism in ADF

8.3.1.1.6 Communication & Notifications

- **Admin Messaging System**
Admins can now send centralized messages to all application users, improving communication for updates and announcements.

8.3.1.1.7 Installation & Configuration

- **Simplified Installation Process**
Users can now install Yres via a link received in an email, streamlining the setup process.
- **Azure Environment Variables in Config Settings**
The `config.settings` table now stores Azure environment variables, including ADF names, resource groups, and subscription details, enhancing transparency and configuration management.

8.3.1.2 Enhancements

- **UTC Time Standardization**
All timestamps are now stored in UTC, with users able to select their preferred time zone for display purposes.
- **Improved Error Messaging**
Enhanced error messages when attempting changes in projects with expired due dates, providing clearer guidance to users.

- **Stability and Change System Improvements**

Implemented various enhancements to system stability and the change management system, ensuring a more reliable and efficient user experience.

- **Release changes with only scripted objects**

A bug causing issues when releasing Changes with only scripted objects was solved.

- **Better filter options & performance in DWH logs**

DWH log screens can now be filtered on more details and will only show the first 100 results for better performance. To view missing logs, please filter more specific to your needs.

- **Fixed Load Issue with IMAGE load types**

Image load types that used to fail in the Staging step could cause all records in the target table to be closed. This has been solved

8.4 Version 1.52 - January 2025

8.4.1 Breaking change (reminder)

View [Monitoring].[LoadMonitor] has been replaced with [Monitoring].[Monitor] in release 1.51. These views will co-exist until version 1.53 after which the [Monitoring].[LoadMonitor] will be removed.

8.4.2 Improvements

- Make SSO required per user
 - This option allows admins to force users to sign in using SSO
 - The option can be found in the Users and Roles menu
- Database object viewer
 - The database object views allows users to display all database objects (also not created with or by Yres)
 - You can view the definition in SQL
 - You can compare definitions over time
 - You view dependencies
- Added Licensing options
 - A license is now added to the database. This allows Yres to limit usage if applicable.
 - All existing customers are granted a full license automatically
- Minor frontend improvements
 - Warning messages made Yellow
 - User settings links fixed in dashboard
 - Dropdown in dashboard linked to user timezone
 - Added new health checks
 - Update Tables redesigned
 - New Feedback form in user menu

- Added scrollbar for datasources
- Renamed Panel to Users & Roles
- Sourcefields with multiple "." are now displayed correctly
- Sorting applied to lists
- Allow reinstall without a reimport
- New feedback form
 - User can now provide feedback directly via the frontend with the new Feedback form
 - Feedback is split into 3 categories
 - Bug report
 - Report issues with the application
 - Feature request
 - Request new features or improved functions
 - Feedback
 - General feedback on the Yres application or processes
- Minor Improvements in Azure resources
 - ADF
 - Upgraded LinkedService in ADF
 - MySQL
 - Postgress
 - Snowflake
 - Custom dispatcher
 - Issues with upgrades are fixed
 - Added support for JSON objects in oData results
 - Added support for gZip compression in oData metadata (specifically required by SAP loads)
 - Database
 - DB Scaling is now better managed over multiple workstreams. This prevents one workstream to scale the server back down in another workstream is still running
 - JSON is handled better. This now also supports datatypes
 - General
 - The pipeline ID is now also picked up for custom sources, thus completing the logging if required.
 - Source table without a key are now better supported
 - Improvements to the changes system
 - Added support for datatypes used in table key
 - XML
 - TEXT
 - NTEXT

- IMAGE
- GEOGRAPHY
- GEOMETRY
- HIERARCHYID
- Log mismatches when updating a table
- Improvements on creating the Delta script for loads
- General stability improvements
- Improvements in Logging & Monitoring

8.5 Version 1.51 - September 2024

8.5.1 Breaking change

View [Monitoring].[LoadMonitor] has been replaced with [Monitoring].[Monitor]. These views will co-exist until version 1.52 after which the [Monitoring].[LoadMonitor] will be removed.

8.5.2 Improvements

- New homepage with monitors for Jobs, Errors and loads
- New Datasource picker
- Support for adding multiple Azure Blob sources
- Temporal tables for important settings
- Improvements to the stability of the changes system
- Generic Rest APIs as a new source
 - You can now extract data form any API that returns a JSON
 - Full support for headers per service, not per endpoint
 - Authentication supports
 - Anonymous
 - Header
 - Basic
 - oAuth
 - Query parameters are supported
 - Only GET endpoints are supported
- Salesforce as a new Source
- SAP Analytics Cloud as a new source
- oAuth support for oData
 - Client credential
 - Authorization Code

- Extended support for managing users and roles in the Expose scheme (only available with SQL interaction)
- Standard publishing of new role [Yres_dbreader]
 - This role allows users to read from the standard HIS schema (as set up in the settings) whilst hiding all stage and system tables.
- Added support for two Delta columns for all SQL based sources except for MySQL.
 - Right now this feature is only available by manually updating the deltaColumn field in the Loadmanagement.UsedTables table. By adding two columns separated by comma's (CreationDate,ChangeDate) the system will take the highest value out of these two fields.
 - Please be aware all fields need to be of the same data type
- Now using Keyvault API version 7.4
- Removed mandatory Orderby in oData, this allows for oData services that do not support OrderBy clauses
- Added security improvements
- General stability improvements
- Improvements in Logging & Monitoring

8.6 Version 1.50 - August 2024

- Standard support for surrogate keys
 - Can be set systemwide and per table
- Added table settings in Web frontend
 - Features that were only available from the SQL endpoint are now easier to use using the web-frontend
 - This includes features like table specific Columnstore and inMemory settings, Loadfilters, delta offsets, page limits and package sizes
- Yres now supports single environments (only prod)
- The entire process is now logged in UTC and can be shown in all possible timezones
- List orders by alphabet
- New data engineering menu

8.7 Version 1.49 - June 2024

- New top menu
- Better insight in active jobs
- Push messages for finished jobs, where ever in the application you are
- Strick database management preventing users to make unsolicited changes in settings logs or objects

8.8 Version 1.48 - April 2024

- New top menu
- Better insight in active jobs
- Notifications for finished jobs, wherever in the application you are
- Firewall specifically for the Web frontend
- Direct links to the existing Azure resources
- New role management
- Active monitoring on source changes that impact the data platform

8.9 Version 1.47 - Januari 2024

- New Audit log for web app usage
- PowerBI dashboard can be integrated in Yres, meant for extended load monitoring
- Allow users to be part of multiple organizations